TayChenJiangNguyenLi-stat443project

TayChenJiangNguyenLi

2026-03-16

Loading packages

```
library(readr)
library(dplyr)
library(lubridate)
```

Reading the data

```
# Covering 1981-01 to 2025-12
nhpi_full <- read.csv("18100205.csv", na.strings = c("..", "NA", "")) %>%
  mutate(REF_DATE = as.Date(paste0(REF_DATE, "-01")),
         REF_DATE_ym = format(REF_DATE, "%Y-%m"))
# head(nhpi_full)

# Covering 1981-01 to 2023-12
ir <- read_csv("IRSTCB01CAM156N.csv",show_col_types = FALSE) %>%
  rename(REF_DATE = observation_date,
         IR = IRSTCB01CAM156N) %>%
  mutate(REF_DATE = as.Date(REF_DATE),
         REF_DATE_ym = format(REF_DATE, "%Y-%m"),
         IR = as.numeric(IR)) %>%
  filter(!is.na(IR))
# head(ir)

# Combining the dataset for ARMAX (cover the overlap only)
nhpi_ir <- nhpi_full %>%
  left_join(ir %>% select(REF_DATE_ym, IR),
            by = "REF_DATE_ym") %>%
  filter(REF_DATE <= as.Date("2023-12-01"))
# head(nhpi_ir)

# Getting lagged interest rate
nhpi_ir <- nhpi_ir %>%
  group_by(GEO, New.housing.price.indexes) %>%
  arrange(REF_DATE) %>%
  mutate(IR_lag1 = lag(IR, 1),
         IR_lag2 = lag(IR, 2)) %>%
  ungroup()
# head(nhpi_ir)

# Checking date coverage
range(nhpi_full$REF_DATE)  # should include 1981-01-01 to 2025-12-01 (or close)
```

```r
range(ir$REF_DATE)          # should include 1981-01-01 to 2023-12-01
range(nhpi_ir$REF_DATE)     # should include 1981-01-01 to 2023-12-01

# Checking if there's any missing IR after merge? (should be basically none within overlap)
mean(is.na(nhpi_ir$IR))

# Checking lags (first month per series will be NA)
nhpi_ir %>%
  filter(GEO == "Canada", New.housing.price.indexes == "Total (house and land)") %>%
  select(REF_DATE, VALUE, IR, IR_lag1, IR_lag2) # %>%
  # head(5)
```

Defining functions

Baseline 1: Naive/Persistence

```r
persist_fc <- function(train, holdout, iprint = FALSE) {
  n <- length(train)
  n_holdout <- length(holdout)
  mse <- 0
  fcvec <- numeric(n_holdout)

  # first forecast
  fc <- train[n]
  yt <- holdout[1]
  fcvec[1] <- fc
  fcerror <- yt - fc
  mse <- mse + fcerror^2

  # remaining forecasts
  for (i in 2:n_holdout) {
    yt <- holdout[i]
    fc <- holdout[i-1]
    fcvec[i] <- fc
    fcerror <- yt - fc
    mse <- mse + fcerror^2
  }

  if (iprint)
    print(cbind(holdout, fcvec))

  rmse <- sqrt(mse / n_holdout)
  return(list(fc = fcvec, rmse = rmse))
}
```

Baseline 2: Training-Set Average

```r
iid_fc <- function(train, holdout, iprint = FALSE) {
  n <- length(train)
  n_holdout <- length(holdout)
  average <- mean(train)
  mse <- 0
  fcvec <- numeric(n_holdout)
```

```r
  for (i in 1:n_holdout) {
    yt <- holdout[i]
    fc <- average
    fcvec[i] <- fc
    fcerror <- yt - fc
    mse <- mse + fcerror^2
  }

  if (iprint)
    print(cbind(holdout, fcvec))

  rmse <- sqrt(mse / n_holdout)
  return(list(fc = fcvec, rmse = rmse))
}
```

Method 1: Best Exponential Smoothing (allowing for trend and seasonality, if relevant)

```r
# Simple Exponential Smoothing
esm_fc <- function(train, holdout, alpha, level, iprint = FALSE) {
  n_holdout <- length(holdout)
  sse <- 0
  fcvec <- numeric(n_holdout)

  fc <- level
  yt <- holdout[1]
  level_new <- level
  fcvec[1] <- fc
  fcerror <- yt - fc
  sse <- sse + fcerror^2

  for (i in 2:n_holdout) {
    level_new <- alpha * holdout[i-1] + (1 - alpha) * level_new

    fc <- level_new
    fcvec[i] <- fc
    yt <- holdout[i]
    fcerror <- yt - fc
    sse <- sse + fcerror^2

    if (iprint) {
      cat("i =", i, "| fc =", fc, "| y =", yt, "| level =", level_new, "\n")
    }
  }

  rmse <- sqrt(sse / n_holdout)
  return(list(fc = fcvec, rmse = rmse))
}
```

```r
# Linear Exponential Smoothing
lholt_fc <- function(train, holdout, alpha, beta, level, slope, iprint = FALSE) {
  n_holdout <- length(holdout)
  sse <- 0
  fcvec <- numeric(n_holdout)
```

```r
  # first forecast
  fc <- level + slope
  yt <- holdout[1]
  newfc <- fc
  fcvec[1] <- fc
  fcerror <- yt - fc
  sse <- sse + fcerror^2

  level_prev <- level
  slope_prev <- slope

  if (iprint) {
    cat("i = 1 | fc =", fc, "| y =", yt, "| level =", level_prev, "| slope =", slope_prev, "\n")
  }

  # remaining forecasts
  for (i in 2:n_holdout) {
    level_new <- alpha * holdout[i-1] + (1 - alpha) * fc
    slope_new <- beta * (level_new - level_prev) + (1 - beta) * slope_prev

    fc <- level_new + slope_new
    fcvec[i] <- fc
    yt <- holdout[i]
    fcerror <- yt - fc
    sse <- sse + fcerror^2

    if (iprint) {
      cat("i =", i, "| fc =", fc, "| y =", yt, "| level =", level_new, "| slope =", slope_new, "\n")
    }

    level_prev <- level_new
    slope_prev <- slope_new
  }

  rmse <- sqrt(sse / n_holdout)
  return(list(fc = fcvec, rmse = rmse))
}
```

```r
# Additive Seasonal Exponential Smoothing
aseason_fc <- function(train, holdout, alpha, beta, gamma, level, slope, season, iprint = FALSE) {
  n_holdout <- length(holdout)
  m <- length(season)
  sse <- 0
  fcvec <- numeric(n_holdout)

  # prev states from training end
  level_prev <- level
  slope_prev <- slope

  # helper: map i -> seasonal position in 1..d
  spos <- function(i) ((i-1) %% m) + 1

  # First forecast
```

```r
  fc <- level_prev + slope_prev + season[spos(1)]
  fcvec[1] <- fc
  yt <- holdout[1]
  fcerror <- yt - fc
  sse <- sse + fcerror^2

  if (iprint) {
    cat("i = 1 | fc =", fc, "| y =", yt, "| level =", level_prev, "| slope =", slope_prev, "| season ="
  }

  # Remaining forecast
  for (i in 2:n_holdout) {
    # Seasonal of previous observation (i-1)
    k_prev <- spos(i-1)
    s_prev <- season[k_prev]
    # Update using previous observation holdout[i-1]
    level_new <- alpha * (holdout[i-1] - s_prev) + (1 - alpha) * (level_prev + slope_prev)
    slope_new <- beta * (level_new - level_prev) + (1 - beta) * slope_prev
    s_new <- gamma * (holdout[i-1] - level_new) + (1 - gamma) * s_prev

    # store updated seasonal state one full cycle later
    season[k_prev] <- s_new

    # Seasonal of current forecast (i)
    k_cur <- spos(i)
    s_cur <- season[k_cur]

    # Forecast current observation using updated states + current seasonal
    fc <- level_new + slope_new + s_cur
    fcvec[i] <- fc
    yt <- holdout[i]
    fcerror <- yt - fc
    sse <- sse + fcerror^2

    if (iprint) {
      cat("i =", i, "| fc =", fc, "| y =", yt, "| level =", level_new, "| slope =", slope_new, "| seaso
    }

    # Update level and slope
    level_prev <- level_new
    slope_prev <- slope_new
  }

  rmse <- sqrt(sse / n_holdout)
  return(list(fc = fcvec, rmse = rmse))
}
```

Method 2: ARIMA

```r
arma_fc <- function(tsdata, ntrain, order, seasonal, method, traincoef, include.mean, iprint=F) {
  obj = arima(tsdata, order=order, seasonal = seasonal, init=traincoef, fixed=traincoef,
              method=method, include.mean=include.mean, optim.control=list(maxit=0)) # no log-likelihoo
  fc = tsdata - obj$residuals
```

```
  ntotal = length(tsdata)
  holdout_fc = fc[(ntrain+1):ntotal]
  holdout = tsdata[(ntrain+1):ntotal]

  if(iprint) print(cbind(holdout,holdout_fc))

  rmse = sqrt(mean((holdout-holdout_fc)^2))
  return(list(rmse=rmse, fc=holdout_fc, residuals = obj$residuals))
}
```

Method 3: ARIMAX with lagged interest

```
armax_fc <- function(tsdata, ntrain, order, seasonal, xreg, method, traincoef, include.mean, iprint=F) {
  obj = arima(tsdata, order=order, seasonal = seasonal, init=traincoef,
              fixed=traincoef, method=method, include.mean=include.mean,
              xreg=xreg, optim.control=list(maxit=0))

  fc = tsdata - obj$residuals

  ntotal = length(tsdata)
  holdout_fc = fc[(ntrain+1):ntotal]
  holdout = tsdata[(ntrain+1):ntotal]

  if(iprint) print(cbind(holdout,holdout_fc))

  rmse = sqrt(mean((holdout-holdout_fc)^2))
  return(list(rmse=rmse, fc=holdout_fc, residuals = obj$residuals))
}
```

Analyzing Montreal

Setting up

```
# Reading the data
montreal_nhpi_data <- read.csv("Full_Cities/montreal_full.csv")
montreal_nhpi_ir_data <- read.csv("Extracted_Cities/montreal.csv")

# Extracting the NHPI
montreal_nhpi <- montreal_nhpi_data$VALUE

# Extracting the one-month lag interest rate
montreal_irlag1 <- montreal_nhpi_ir_data$IR_lag1

# Setting the time window from 1981-2023
montreal_ts <- ts(montreal_nhpi, start = c(1981,1), end = c(2023, 12), frequency = 12)
montreal_ir_ts <- ts(montreal_irlag1, start = c(1981,1), end = c(2023, 12), frequency = 12)

# Applying a transformation for NHPI
montreal_transformed_nhpi <- diff(log(montreal_ts))

# Setting a training set from 1981-2021
montreal_train_ts <- window(montreal_transformed_nhpi, end = c(2021,12))
montreal_train_ir_xreg <- window(montreal_ir_ts, start = c(1981,2), end = c(2021,12))
```

```r
# Setting a holdout set from 2022-2023
montreal_holdout_ts <- window(montreal_transformed_nhpi, start = c(2022,1))
montreal_holdout_ir_xreg <- window(montreal_ir_ts, start = c(2022,1))
```

Doing the baseline forecasts

```r
# Baseline 1: Naive/Persistence
montreal_persist_fc <- persist_fc(montreal_train_ts, montreal_holdout_ts, iprint = F)

# Baseline 2: Training-Set Average
montreal_iid_fc <- iid_fc(montreal_train_ts, montreal_holdout_ts, iprint = F)
```

Doing the exponential smoothing forecasts

```r
# Method 1
ntrain <- length(montreal_train_ts)
ntotal <- length(montreal_ts)-1

montreal_transformed_nhpi_train <- montreal_transformed_nhpi[1:ntrain]
montreal_transformed_nhpi_holdout <- montreal_transformed_nhpi[(ntrain+1):ntotal]

# Simple Exponential Smoothing
fit_expsmo <- HoltWinters(montreal_train_ts, beta = F, gamma = F)
montreal_esm_fc <- esm_fc(montreal_transformed_nhpi_train, montreal_transformed_nhpi_holdout,
                          alpha = fit_expsmo$alpha, level = fit_expsmo$coefficients["a"],
                          iprint = F)

# Linear Exponential Smoothing
fit_hw <- HoltWinters(montreal_train_ts, gamma = F)
montreal_holt_fc <- lholt_fc(montreal_transformed_nhpi_train, montreal_transformed_nhpi_holdout,
                             alpha = fit_hw$alpha, beta = fit_hw$beta,
                             level = fit_hw$coefficients["a"],
                             slope = fit_hw$coefficients["b"], iprint = F)

# Additive Seasonal Exponential Smoothing
wafit <- HoltWinters(montreal_train_ts, seasonal = "additive")
montreal_aseason_fc <- aseason_fc(montreal_transformed_nhpi_train,
                                  montreal_transformed_nhpi_holdout,
                                  alpha = wafit$alpha, beta = wafit$beta,
                                  gamma = wafit$gamma, level = wafit$coef[1],
                                  slope = wafit$coef[2], season = wafit$coefficient[3:14],
                                  iprint = F)

# Comparing RMSE
rmse_esm <- cbind(montreal_esm_fc$rmse, montreal_holt_fc$rmse, montreal_aseason_fc$rmse)
colnames(rmse_esm) <- c("esm", "lholt", "aseason") # RMSE: 0.0079, 0.0077, 0.0088
rmse_esm
```

```
##           esm       lholt     aseason
## a 0.007918044 0.007662966 0.008806579
```

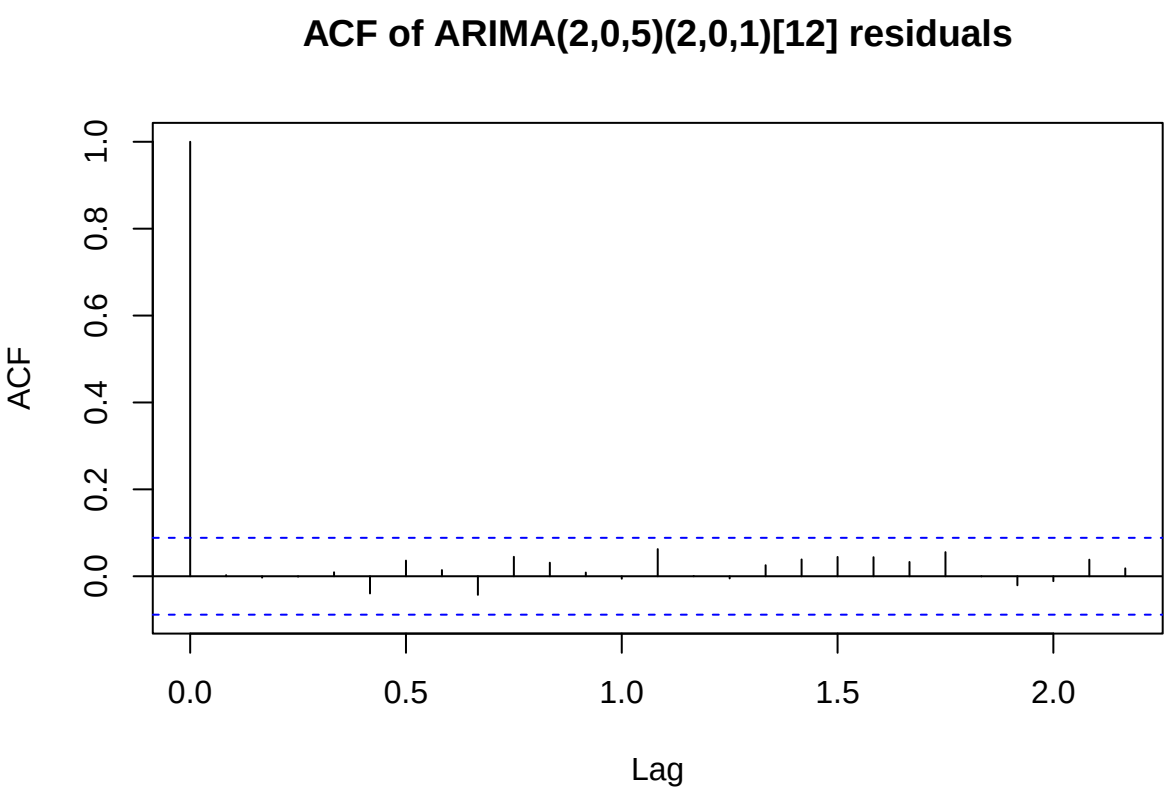
```r
# Best exponential smoothing method: Linear Exponential Smoothing
```

Doing the ARIMA forecasts

```r
# Method 2
montreal_transformed_nphi_auto <- forecast::auto.arima(montreal_train_ts)
summary(montreal_transformed_nphi_auto) # 0.0048783
```

```
## Series: montreal_train_ts
## ARIMA(2,0,5)(2,0,1)[12] with non-zero mean
##
## Coefficients:
##          ar1      ar2      ma1     ma2     ma3      ma4     ma5    sar1    sar2
##       1.2933  -0.4683  -1.1324  0.3576  0.0775  -0.0112  0.1526  0.7129  0.0270
## s.e.  0.1615   0.1643   0.1629  0.1522  0.0680   0.0678  0.0628  0.2571  0.0723
##          sma1    mean
##       -0.6604  0.0034
## s.e.   0.2622  0.0007
##
## sigma^2 = 2.434e-05:  log likelihood = 1916.27
## AIC=-3808.54   AICc=-3807.89   BIC=-3758.19
##
## Training set error measures:
##                        ME        RMSE         MAE MPE MAPE      MASE
## Training set -4.22165e-05 0.004878312 0.002960714 NaN  Inf 0.7689245
##                     ACF1
## Training set 0.002456102
```

```r
resid_fit <- residuals(montreal_transformed_nphi_auto)
acf(resid_fit, main = "ACF of ARIMA(2,0,5)(2,0,1)[12] residuals")
```



```r
# Comparing with the other ARIMA models
manual_fit <- arima(montreal_train_ts, order = c(2,0,5),
                    seasonal = list(order = c(2,0,1), period = 12), method = "ML")
```

```
## Warning in log(s2): NaNs produced
## Warning in log(s2): NaNs produced
```

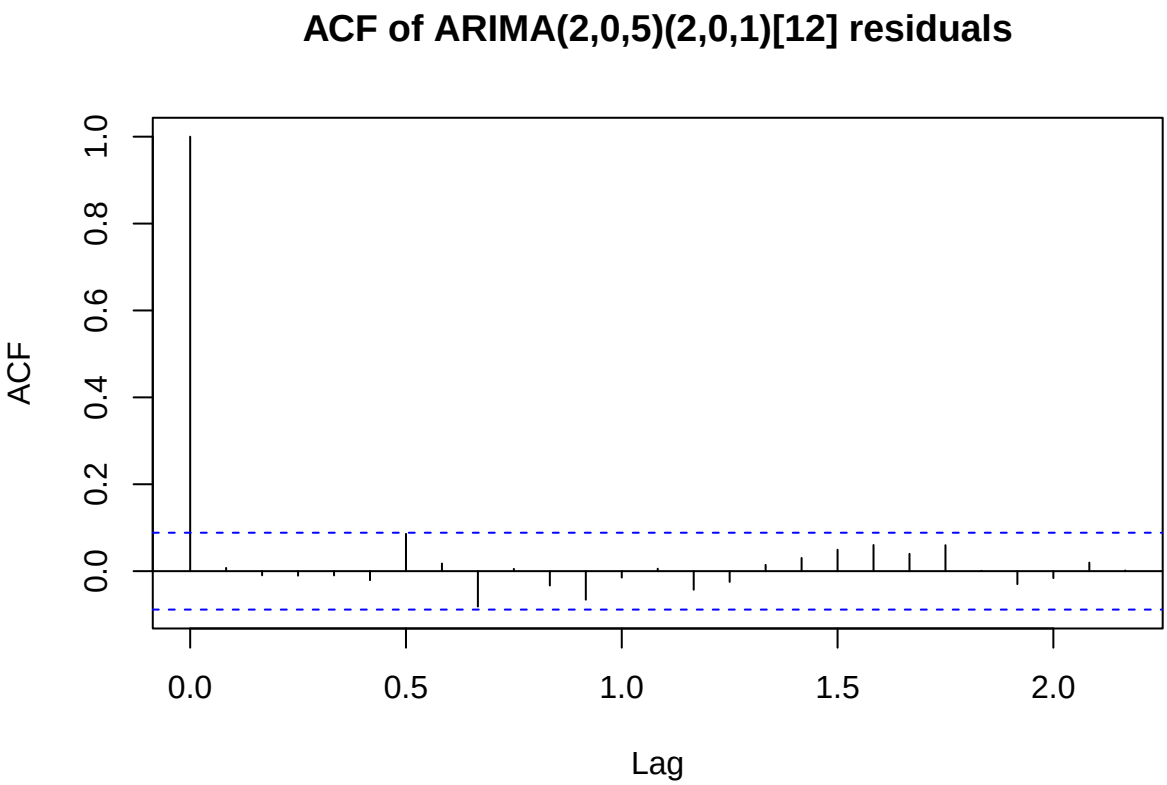
```r
summary(manual_fit) # RMSE: 0.004903117
```

```
##
## Call:
## arima(x = montreal_train_ts, order = c(2, 0, 5), seasonal = list(order = c(2,
##     0, 1), period = 12), method = "ML")
##
## Coefficients:
```

```
## Warning in sqrt(diag(x$var.coef)): NaNs produced
```

```
##          ar1     ar2      ma1      ma2    ma3      ma4     ma5    sar1    sar2
##       0.8764  0.0595  -0.7152  -0.1025  0.026  -0.0082  0.0892  0.7867  0.0841
## s.e.     NaN     NaN   0.0202      NaN    NaN   0.0262     NaN  0.0015  0.0022
##          sma1  intercept
##       -0.8241     0.0042
## s.e.   0.0429     0.0013
##
## sigma^2 estimated as 2.404e-05:  log likelihood = 1913.85,  aic = -3803.7
##
## Training set error measures:
##                        ME        RMSE         MAE MPE MAPE      MASE
## Training set -0.000121822 0.004903117 0.002922884 NaN  Inf 0.7590997
##                     ACF1
## Training set 0.007374584
```

```r
acf(residuals(manual_fit), main = "ACF of ARIMA(2,0,5)(2,0,1)[12] residuals")
```



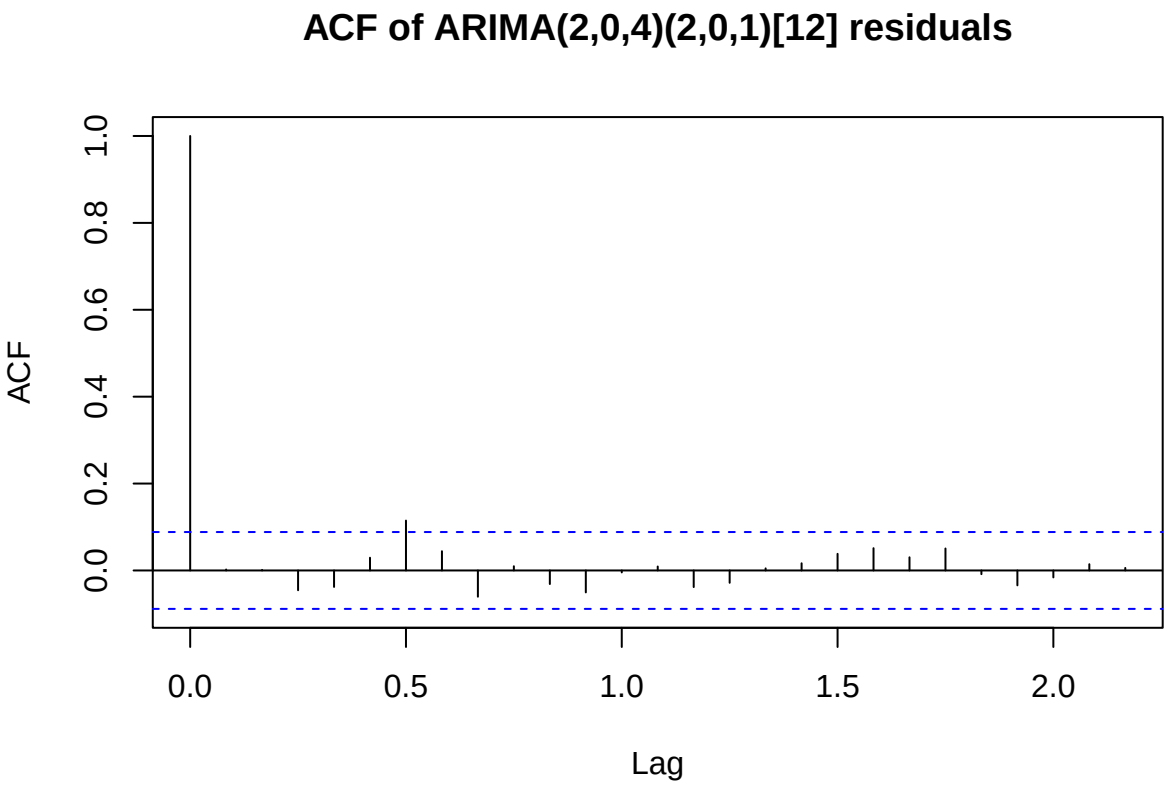

```r
test1 <- arima(montreal_train_ts, order = c(2,0,4),
               seasonal = list(order = c(2,0,1), period = 12), method = "ML")
summary(test1) # RMSE: 0.004919859
```

```
##
## Call:
## arima(x = montreal_train_ts, order = c(2, 0, 4), seasonal = list(order = c(2,
##     0, 1), period = 12), method = "ML")
##
## Coefficients:
```

```
## Warning in sqrt(diag(x$var.coef)): NaNs produced
```

```
##          ar1     ar2     ma1      ma2      ma3     ma4    sar1    sar2     sma1
##       0.1009  0.8396  0.0708  -0.7734  -0.0067  0.0520  0.8286  0.0792  -0.8625
## s.e.     NaN     NaN     NaN      NaN   0.0474  0.0536  0.1540  0.0648   0.1467
##       intercept
##          0.0043
## s.e.     0.0018
##
## sigma^2 estimated as 2.421e-05:  log likelihood = 1911.91,  aic = -3801.82
##
## Training set error measures:
##                        ME        RMSE         MAE MPE MAPE      MASE
## Training set -8.718343e-05 0.004919859 0.002886015 NaN  Inf 0.7495245
##                     ACF1
## Training set 0.00222566
```

```r
acf(residuals(test1), main = "ACF of ARIMA(2,0,4)(2,0,1)[12] residuals")
```



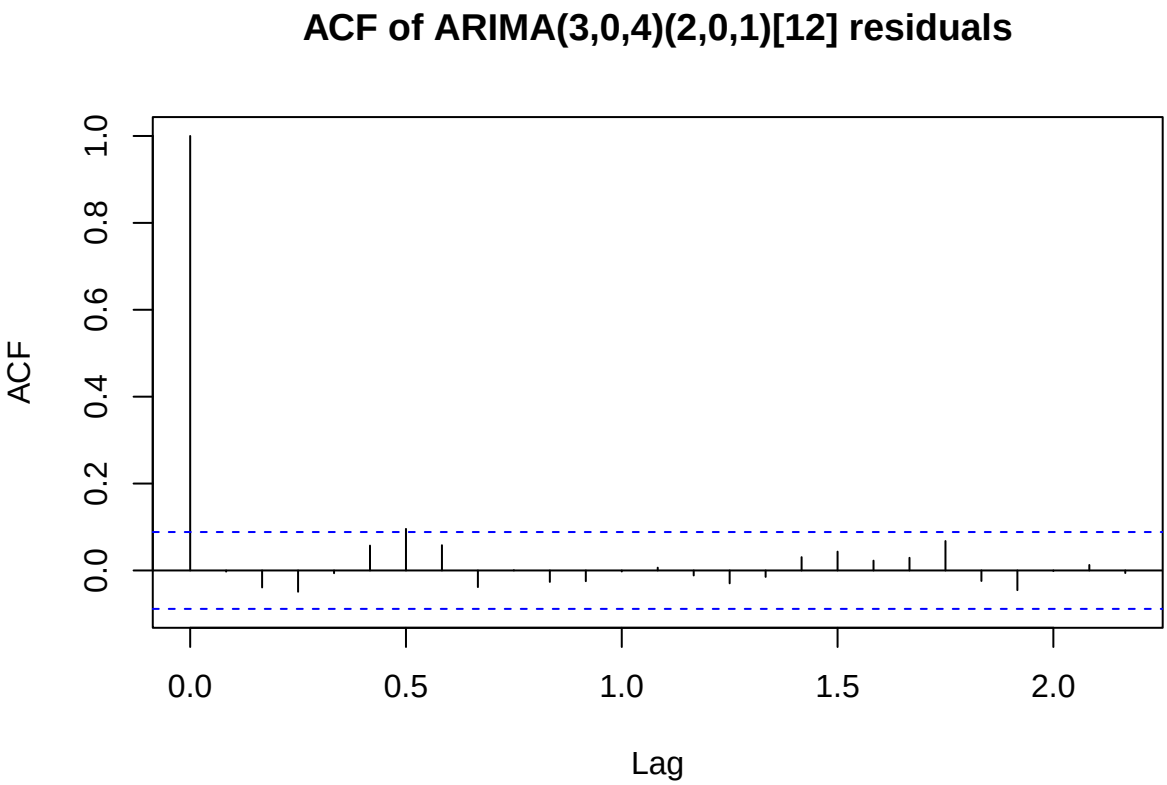

```r
test2 <- arima(montreal_train_ts, order = c(3,0,4),
               seasonal = list(order = c(2,0,1), period = 12), method = "ML")
```

```
## Warning in arima(montreal_train_ts, order = c(3, 0, 4), seasonal = list(order =
## c(2, : possible convergence problem: optim gave code = 1
```

```r
summary(test2) # RMSE: 0.004857711
```

```
##
## Call:
## arima(x = montreal_train_ts, order = c(3, 0, 4), seasonal = list(order = c(2,
##     0, 1), period = 12), method = "ML")
##
## Coefficients:
##          ar1      ar2     ar3      ma1     ma2      ma3      ma4    sar1
##       0.2633  -0.2965  0.9676  -0.0730  0.3999  -0.8139  -0.0725  0.8008
## s.e.  0.0164   0.0139  0.0167   0.0509  0.0328   0.0344   0.0486  0.1765
##         sar2     sma1  intercept
##       0.1004  -0.8507     0.0046
## s.e.  0.0677   0.1709     0.0021
##
## sigma^2 estimated as 2.36e-05:  log likelihood = 1915.58,  aic = -3807.16
##
## Training set error measures:
##                        ME        RMSE         MAE MPE MAPE      MASE
## Training set -9.30943e-05 0.004857711 0.002842068 NaN  Inf 0.7381111
##                      ACF1
## Training set -0.002624896
```

```r
acf(residuals(test2), main = "ACF of ARIMA(3,0,4)(2,0,1)[12] residuals")
```



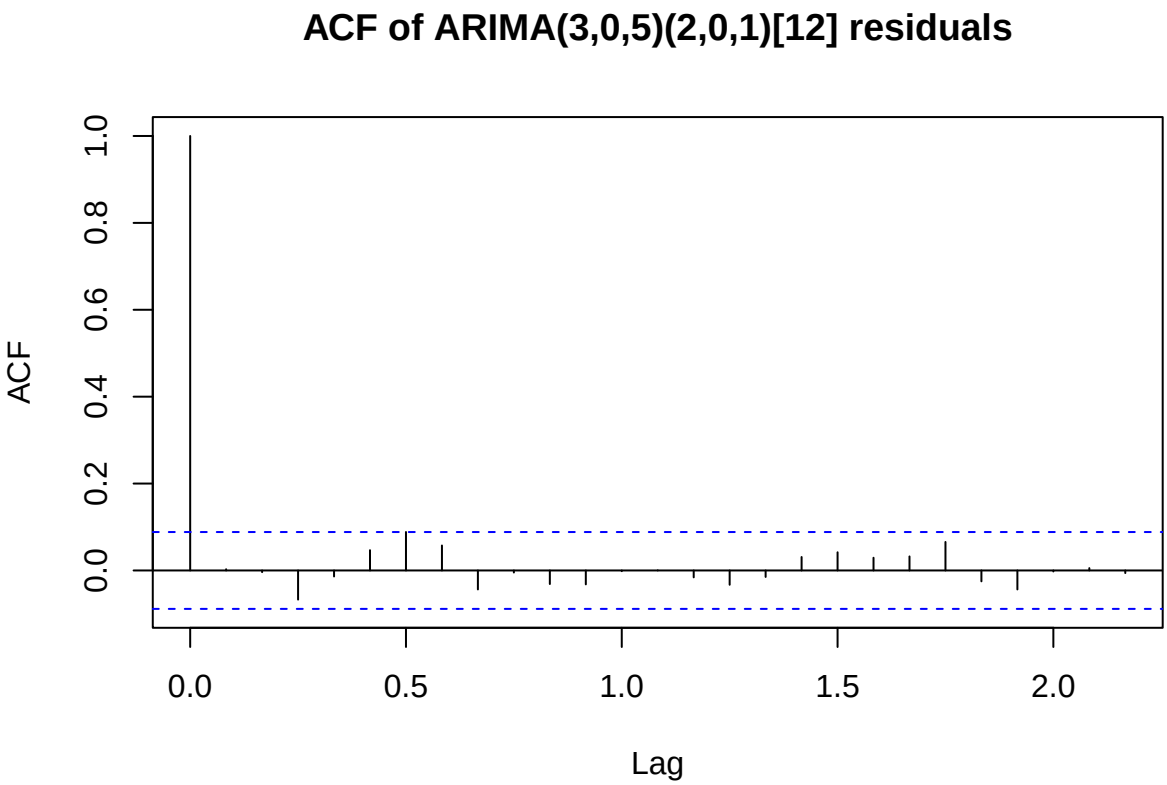

```r
test3 <- arima(montreal_train_ts, order = c(3,0,5),
               seasonal = list(order = c(2,0,1), period = 12), method = "ML")
```

```
## Warning in arima(montreal_train_ts, order = c(3, 0, 5), seasonal = list(order =
## c(2, : possible convergence problem: optim gave code = 1
```

```r
summary(test3) # RMSE: 0.004852676
```

```
##
## Call:
## arima(x = montreal_train_ts, order = c(3, 0, 5), seasonal = list(order = c(2,
##     0, 1), period = 12), method = "ML")
##
## Coefficients:
##          ar1      ar2     ar3      ma1     ma2      ma3      ma4     ma5
##       0.2642  -0.2936  0.9666  -0.0852  0.3585  -0.7956  -0.0706  0.0531
## s.e.  0.0164   0.0137  0.0184   0.0494  0.0486   0.0421   0.0472  0.0536
##         sar1    sar2     sma1  intercept
##       0.8061  0.1085  -0.8711     0.0046
## s.e.  0.1369  0.0645   0.1273     0.0022
##
## sigma^2 estimated as 2.355e-05:  log likelihood = 1916.07,  aic = -3806.15
##
## Training set error measures:
##                        ME        RMSE         MAE MPE MAPE     MASE
## Training set -8.857333e-05 0.004852676 0.002829046 NaN  Inf 0.734729
##                     ACF1
## Training set 0.002781505
```

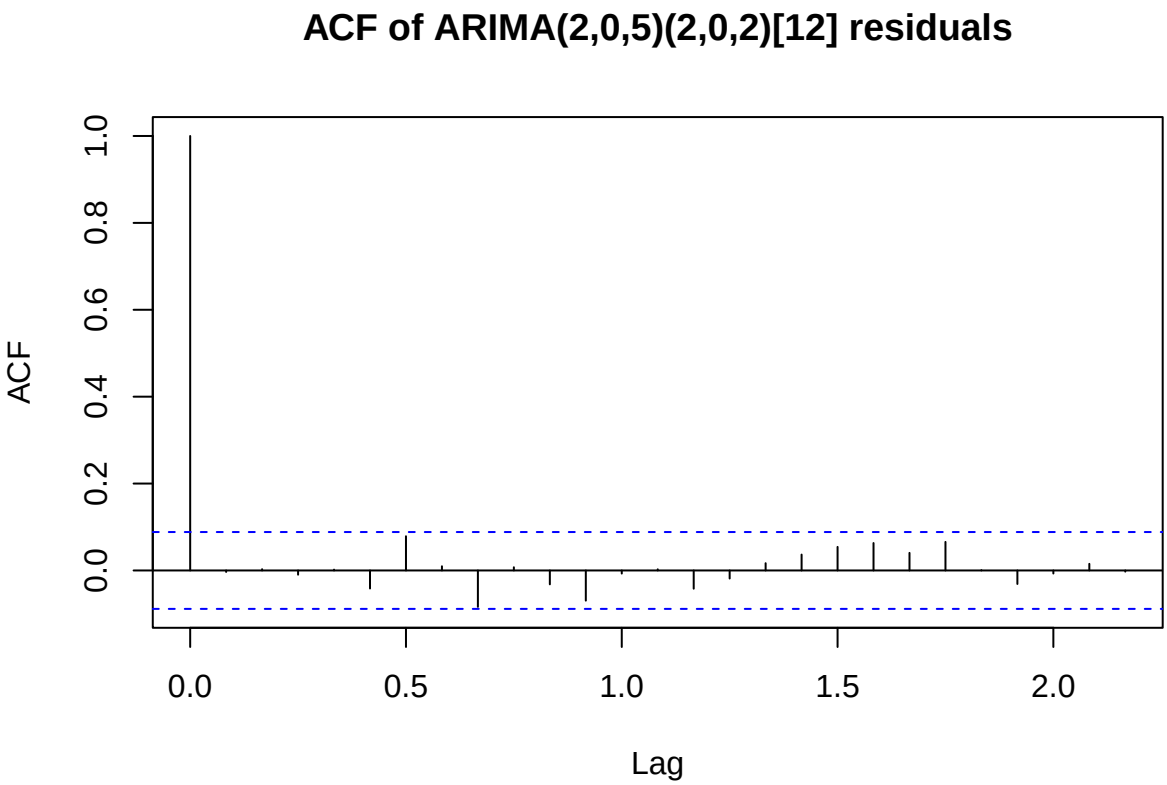
```r
acf(residuals(test3), main = "ACF of ARIMA(3,0,5)(2,0,1)[12] residuals")
```




```r
test4 <- arima(montreal_train_ts, order = c(2,0,5),
               seasonal = list(order = c(2,0,2), period = 12), method = "ML")
summary(test4) # RMSE: 0.004908301
```

```
##
## Call:
## arima(x = montreal_train_ts, order = c(2, 0, 5), seasonal = list(order = c(2,
##     0, 2), period = 12), method = "ML")
##
## Coefficients:

## Warning in sqrt(diag(x$var.coef)): NaNs produced

##          ar1     ar2      ma1      ma2     ma3      ma4     ma5    sar1    sar2
##       0.8518  0.0693  -0.6860  -0.1222  0.0409  -0.0209  0.1239  0.6627  0.2625
## s.e.  0.0011  0.0011   0.0028      NaN     NaN      NaN  0.0018  0.0014  0.0017
##          sma1     sma2  intercept
##       -0.7106  -0.1882     0.0043
## s.e.   0.0017   0.0025     0.0011
##
## sigma^2 estimated as 2.409e-05:  log likelihood = 1913.66,  aic = -3801.32
##
## Training set error measures:
##                        ME        RMSE         MAE MPE MAPE      MASE
## Training set -0.0001793343 0.004908301 0.002963063 NaN  Inf 0.7695344
##                      ACF1
## Training set -0.003464896
```

```r
acf(residuals(test4), main = "ACF of ARIMA(2,0,5)(2,0,2)[12] residuals")
```




```r
test5 <- arima(montreal_train_ts, order = c(2,0,4),
               seasonal = list(order = c(2,0,2), period = 12), method = "ML")
```

```
## Warning in arima(montreal_train_ts, order = c(2, 0, 4), seasonal = list(order =
## c(2, : possible convergence problem: optim gave code = 1
```

```r
summary(test5) # RMSE: 0.004891118
```

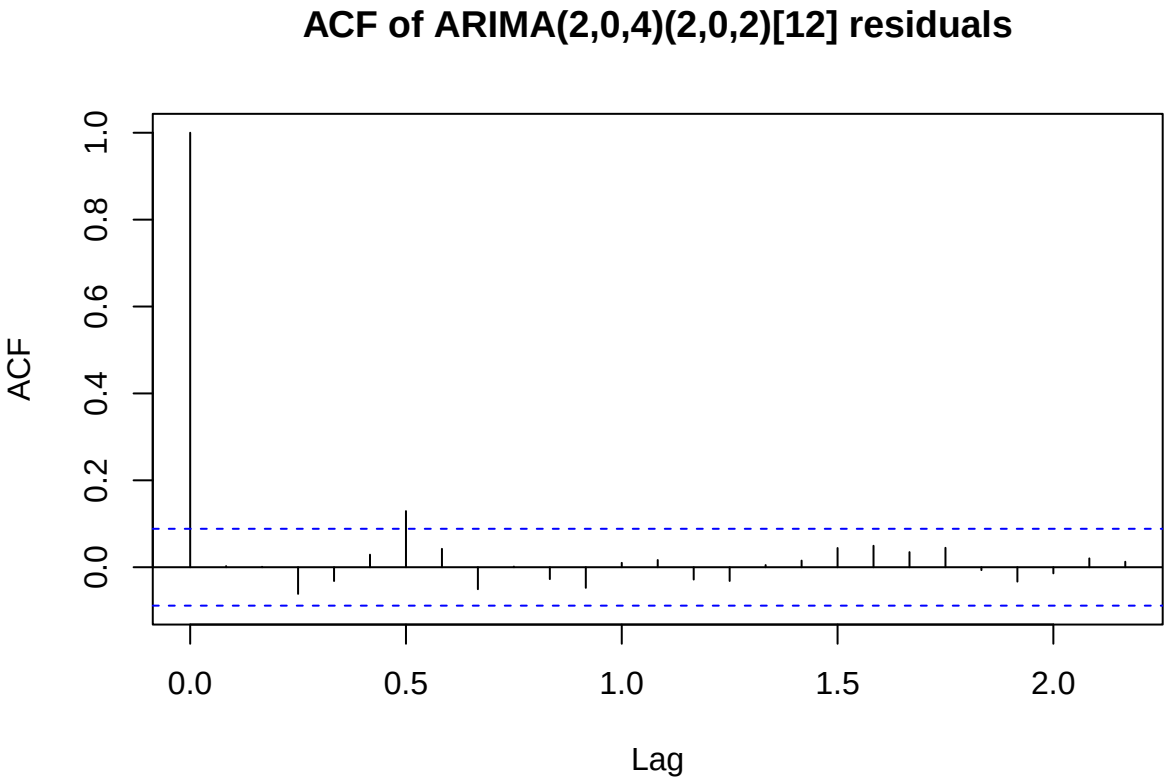
```
##
## Call:
## arima(x = montreal_train_ts, order = c(2, 0, 4), seasonal = list(order = c(2,
##     0, 2), period = 12), method = "ML")
##
## Coefficients:
```

```
## Warning in sqrt(diag(x$var.coef)): NaNs produced
```

```
##           ar1     ar2     ma1      ma2      ma3     ma4    sar1     sar2
##       -0.0227  0.9456  0.2107  -0.8658  -0.0184  0.0582  1.1899  -0.3936
## s.e.   0.0263  0.0249  0.0524   0.0561   0.0507  0.0541     NaN      NaN
##          sma1    sma2  intercept
##       -1.2437  0.5093     0.0045
## s.e.      NaN     NaN     0.0015
##
## sigma^2 estimated as 2.392e-05:  log likelihood = 1914.05,  aic = -3804.1
##
## Training set error measures:
##                         ME        RMSE         MAE MPE MAPE      MASE
## Training set -0.0001308868 0.004891118 0.002897664 NaN  Inf 0.7525497
```

```
##                      ACF1
## Training set 0.002438932
```

```r
acf(residuals(test5), main = "ACF of ARIMA(2,0,4)(2,0,2)[12] residuals")
```




```r
# Comparing AIC values
aic = cbind(manual_fit$aic, test1$aic, test2$aic, test3$aic, test4$aic, test5$aic)
colnames(aic) = c("manual_fit", "test1", "test2", "test3", "test4", "test5")
aic
```

```
##      manual_fit     test1     test2     test3    test4     test5
## [1,]  -3803.702 -3801.817 -3807.156 -3806.146 -3801.32 -3804.098
```

```r
# Best (based on lowest AIC and ACF plot): test3, ARIMA(3,0,5)(2,0,1)[12]

order <- c(3,0,5)
seasonal <- list(order = c(2,0,1), period = 12)
# Fit the model to the training data
train_arima_model <- arima(montreal_train_ts, order = order,
                           seasonal = seasonal, method = "ML", include.mean = T)
```

```
## Warning in arima(montreal_train_ts, order = order, seasonal = seasonal, :
## possible convergence problem: optim gave code = 1
```

```r
# Result the moving 1-step ahead forecast for holdout set
arima_results <- arma_fc(tsdata = montreal_transformed_nhpi,
                         ntrain = length(montreal_train_ts),
                         order = order, seasonal = seasonal, method = "ML",
                         traincoef = coef(train_arima_model), include.mean = T)
arima_results$rmse
```

```
## [1] 0.008251754
```

Doing the ARIMAX forecasts with lagged interest

```r
# Method 3
montreal_fit_arimax <- forecast::auto.arima(montreal_train_ts, xreg = montreal_train_ir_xreg)
summary(montreal_fit_arimax)
```

```
## Series: montreal_train_ts
## Regression with ARIMA(2,0,5)(2,0,1)[12] errors
##
## Coefficients:
##          ar1      ar2      ma1     ma2     ma3      ma4     ma5    sar1    sar2
##       1.2998  -0.4778  -1.1411  0.3668  0.0793  -0.0121  0.1545  0.7148  0.0216
## s.e.  0.1554   0.1577   0.1572  0.1477  0.0684   0.0681  0.0629  0.2831  0.0728
##          sma1  intercept   xreg
##       -0.6608     0.0028  1e-04
## s.e.   0.2890     0.0010  1e-04
##
## sigma^2 = 2.435e-05:  log likelihood = 1916.72
## AIC=-3807.45   AICc=-3806.69   BIC=-3752.9
##
## Training set error measures:
##                         ME        RMSE         MAE MPE MAPE     MASE
## Training set -2.873017e-05 0.004873876 0.002945387 NaN  Inf 0.764944
##                     ACF1
## Training set 0.003074693
```

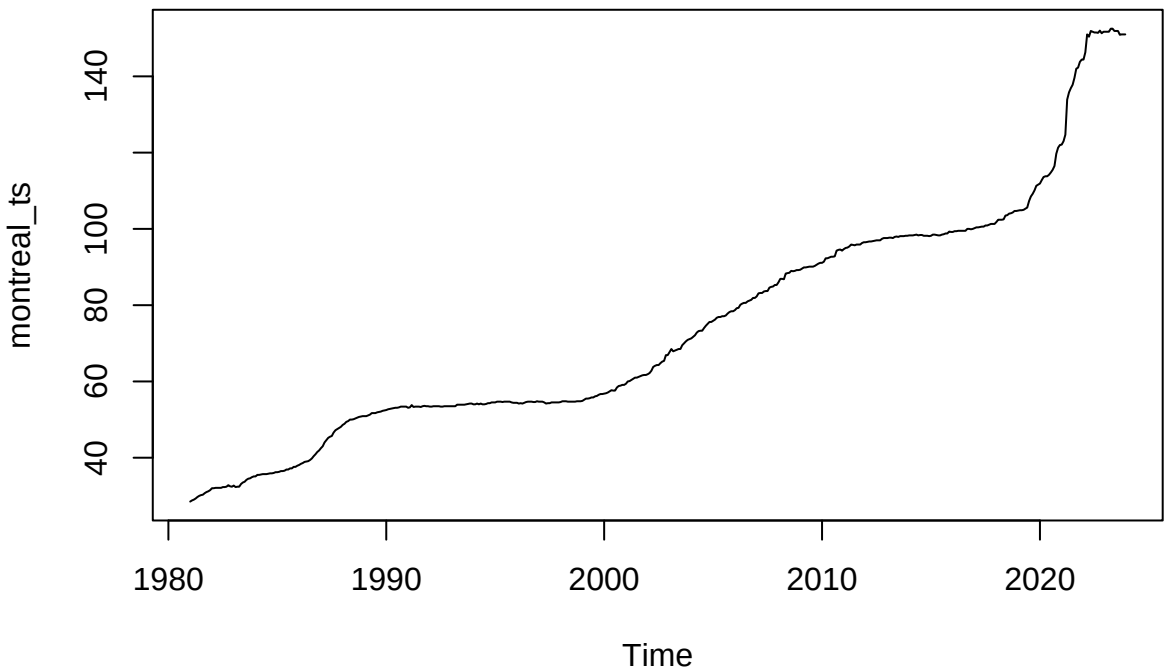
```r
order <- c(3,0,5)
seasonal <- list(order = c(2,0,1), period = 12)
# Fitting a model to the training data
train_arimax_model <- arima(montreal_train_ts, order = order, seasonal = seasonal,
                            xreg = montreal_train_ir_xreg, method = "ML", include.mean = T)
```

```
## Warning in arima(montreal_train_ts, order = order, seasonal = seasonal, :
## possible convergence problem: optim gave code = 1
```

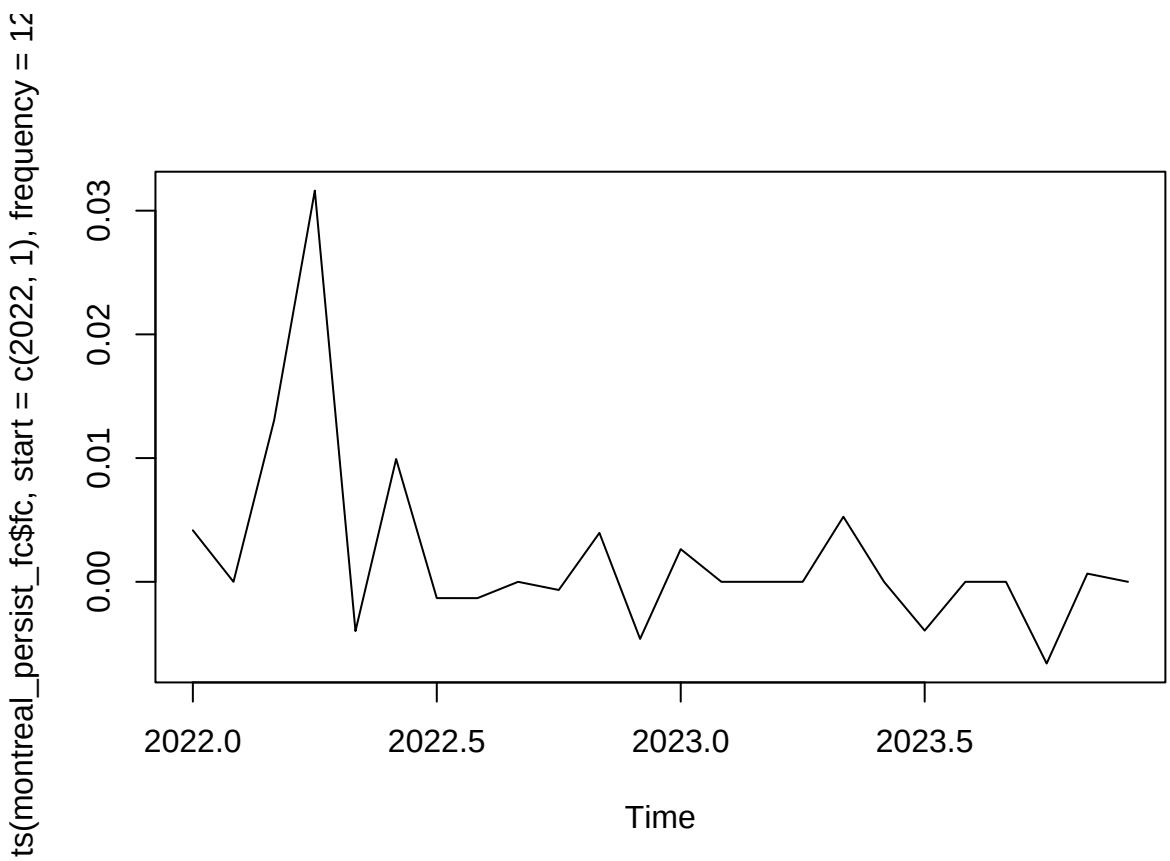
```r
# Getting the result of the moving 1-step ahead forecast for holdout set
arimax_results <- armax_fc(tsdata = montreal_transformed_nhpi,
                           ntrain = length(montreal_train_ts), order = order,
                           seasonal = seasonal, xreg = montreal_irlag1[-1],
                           method = "ML", traincoef = coef(train_arimax_model), include.mean = T)
montreal_arima_results <- arima_results
montreal_arimax_results <- arimax_results
```

Viewing the results of Montreal's analyses

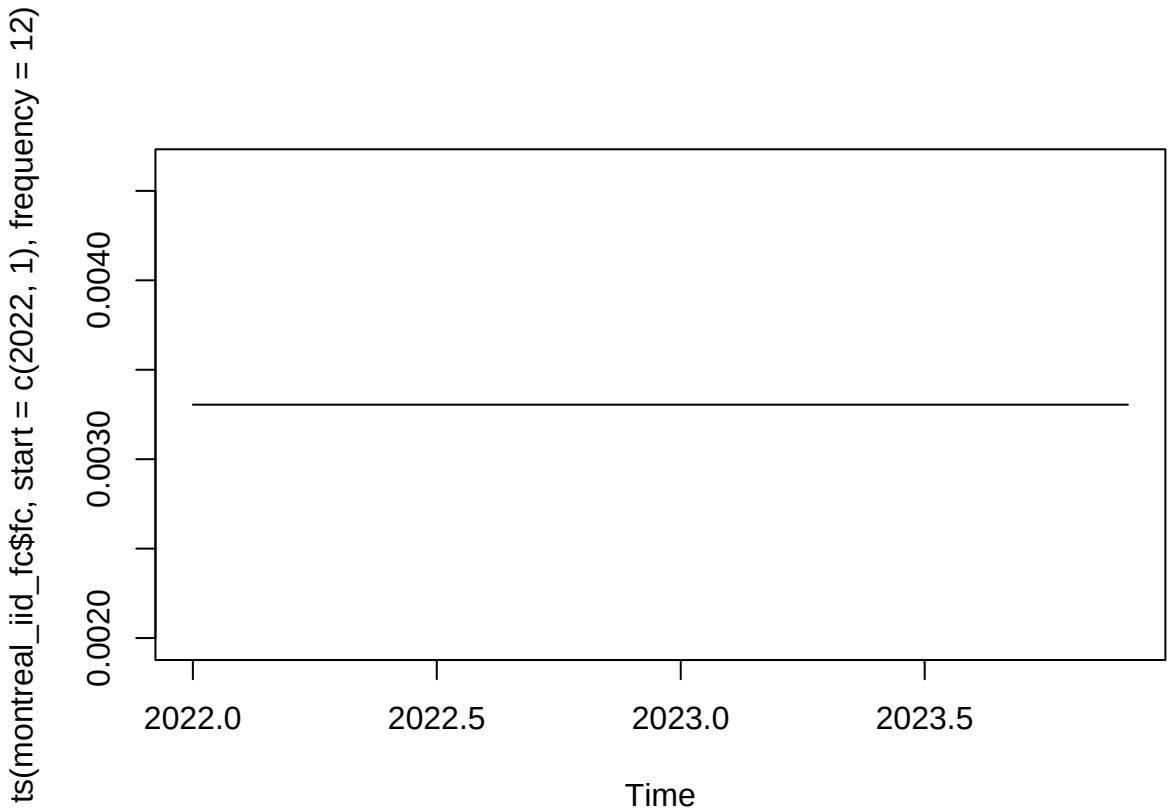
```r
# 1. TIME SERIES PLOT
plot(montreal_ts)
```



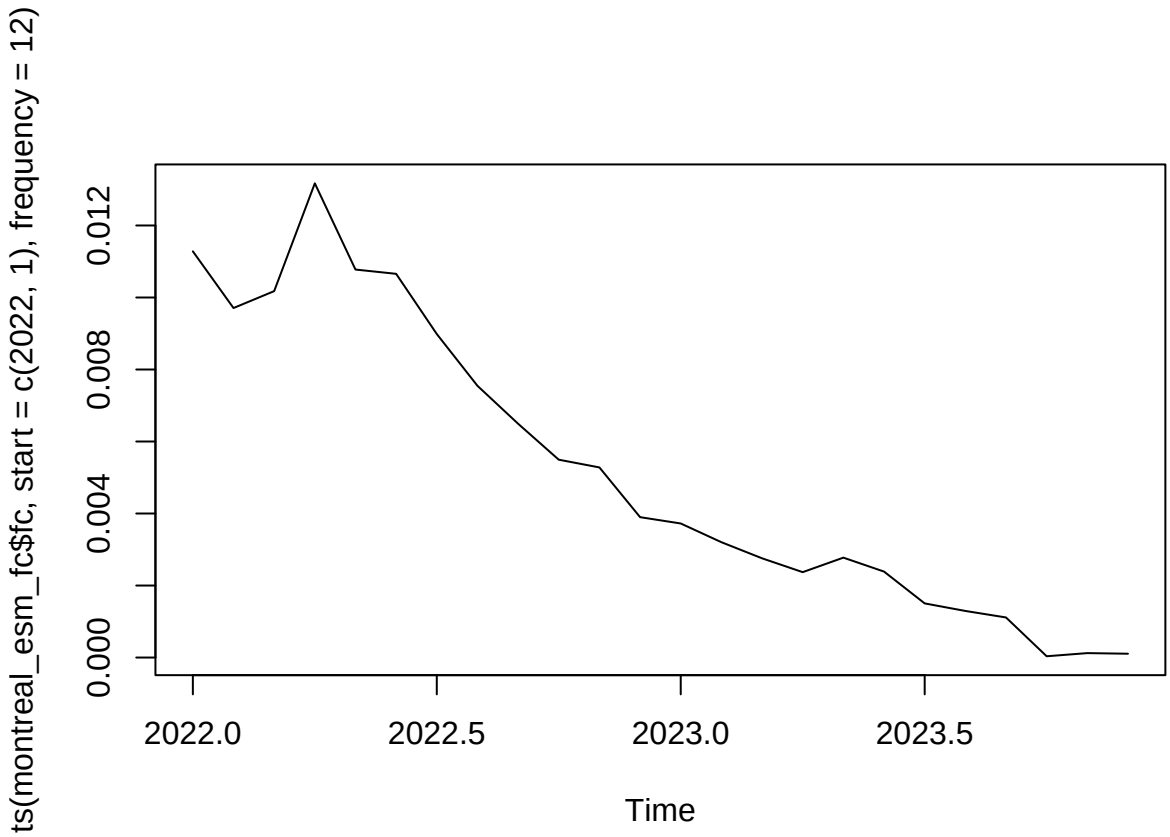

```r
# log-diff transformation
# plot(montreal_transformed_ts) # NOTE: this object doesn't exist
# ts plot from persistence
plot(ts(montreal_persist_fc$fc, start = c(2022, 1), frequency = 12))
```



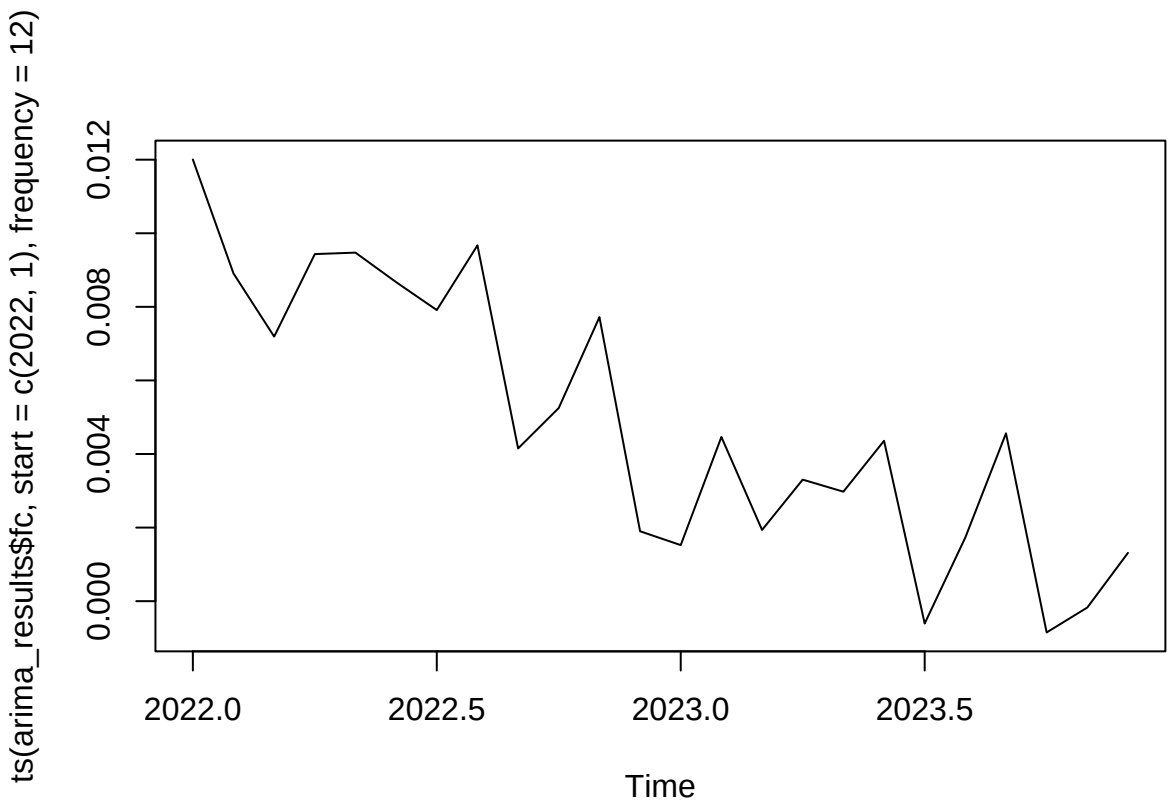
```r
# ts plot from average
plot(ts(montreal_iid_fc$fc, start = c(2022, 1), frequency = 12))
```



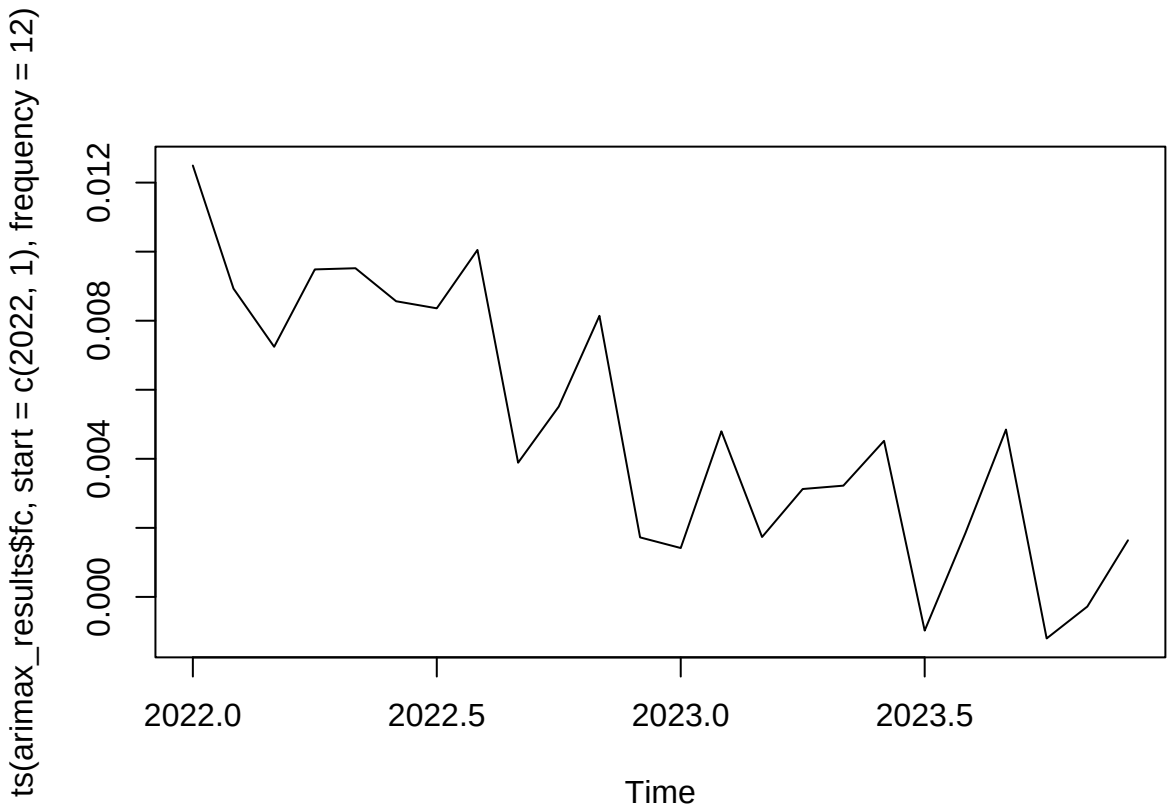

```r
# ts plot from simple exponential smoothing
plot(ts(montreal_esm_fc$fc, start = c(2022, 1), frequency = 12))
```



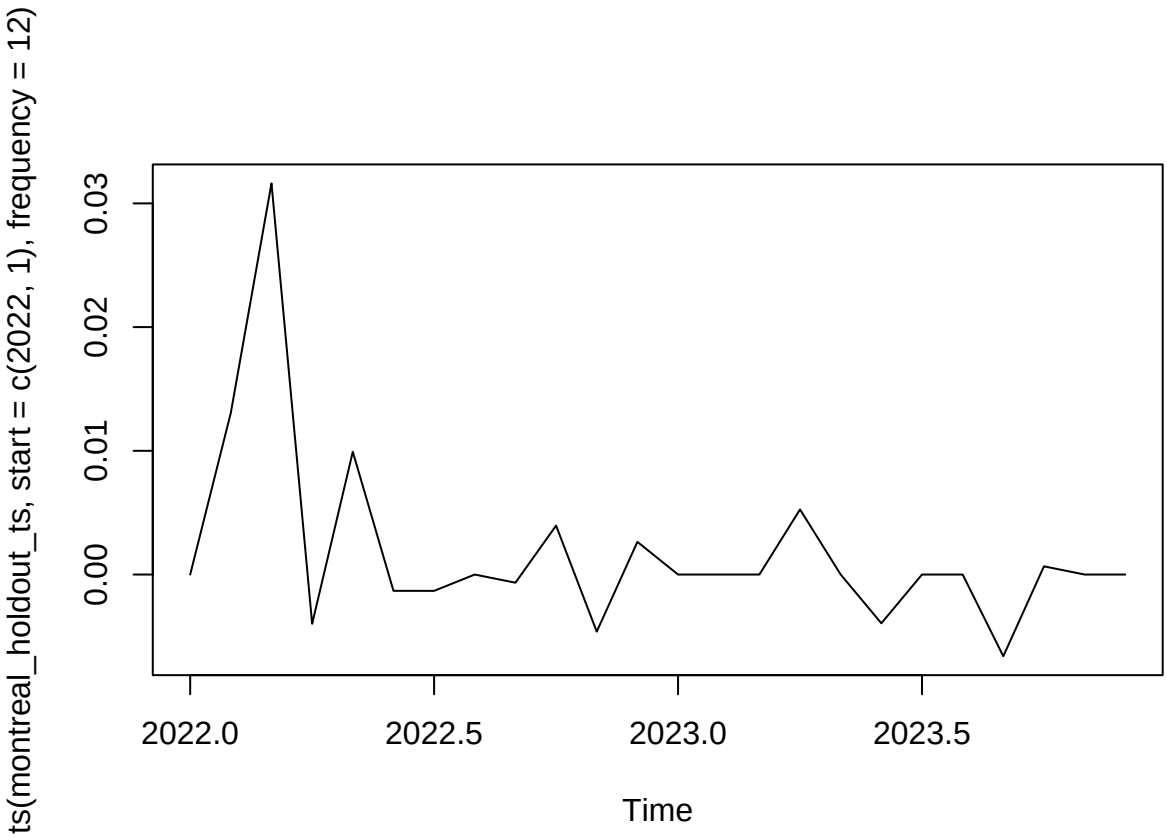

```r
# ts plot from arima
plot(ts(arima_results$fc, start = c(2022, 1), frequency = 12))
```



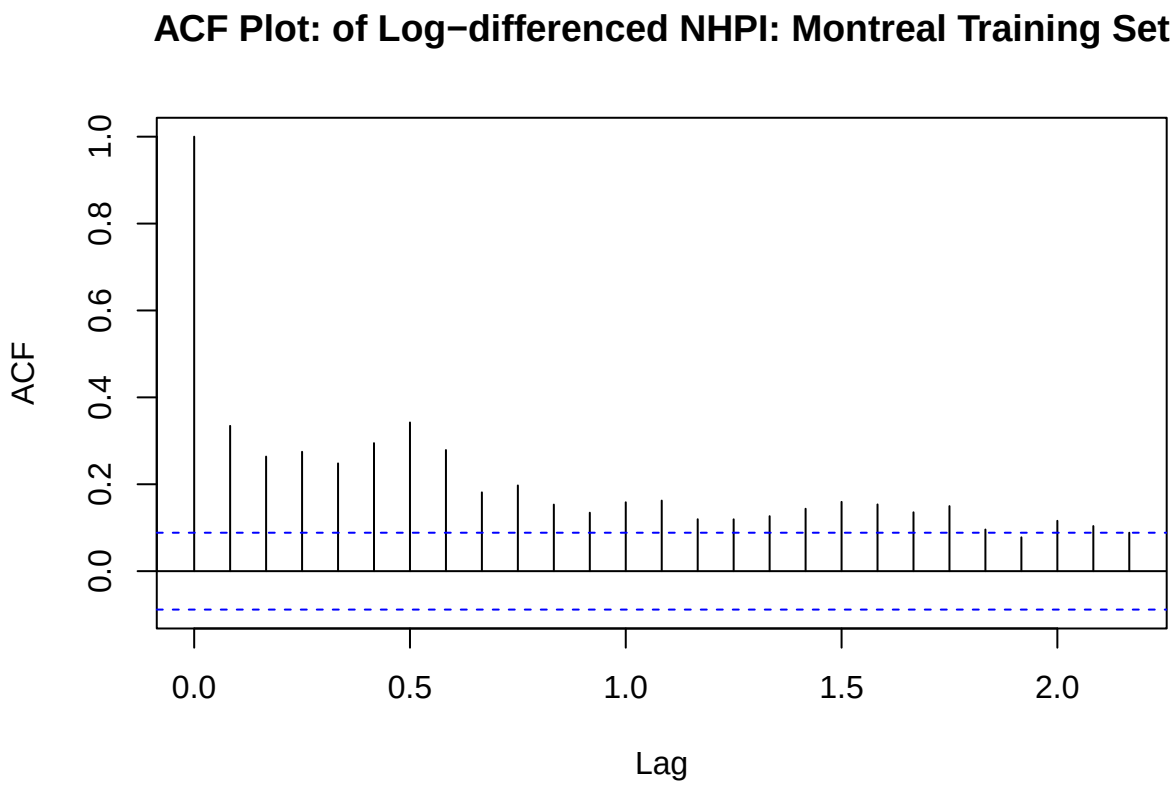
```r
# ts plot from arimax
plot(ts(arimax_results$fc, start = c(2022, 1), frequency = 12))
```



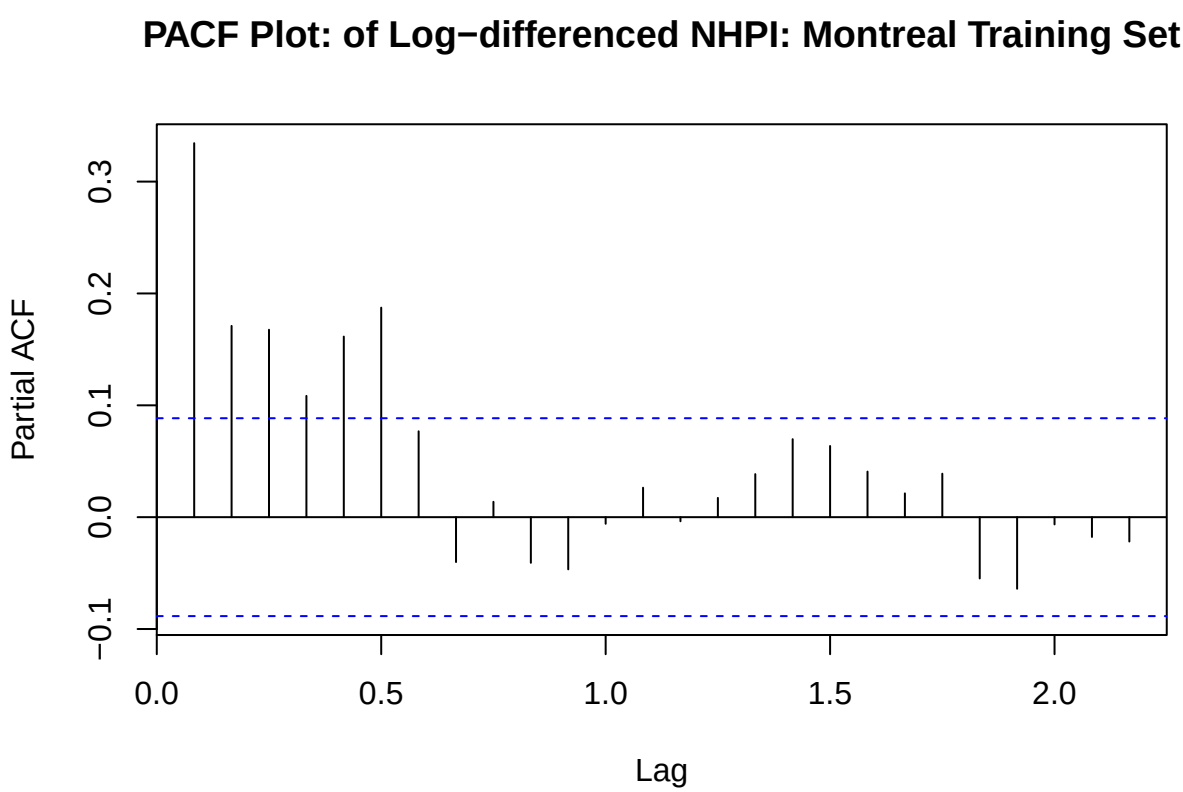

```r
# ts plot of holdout set
plot(ts(montreal_holdout_ts, start = c(2022, 1), frequency = 12))
```



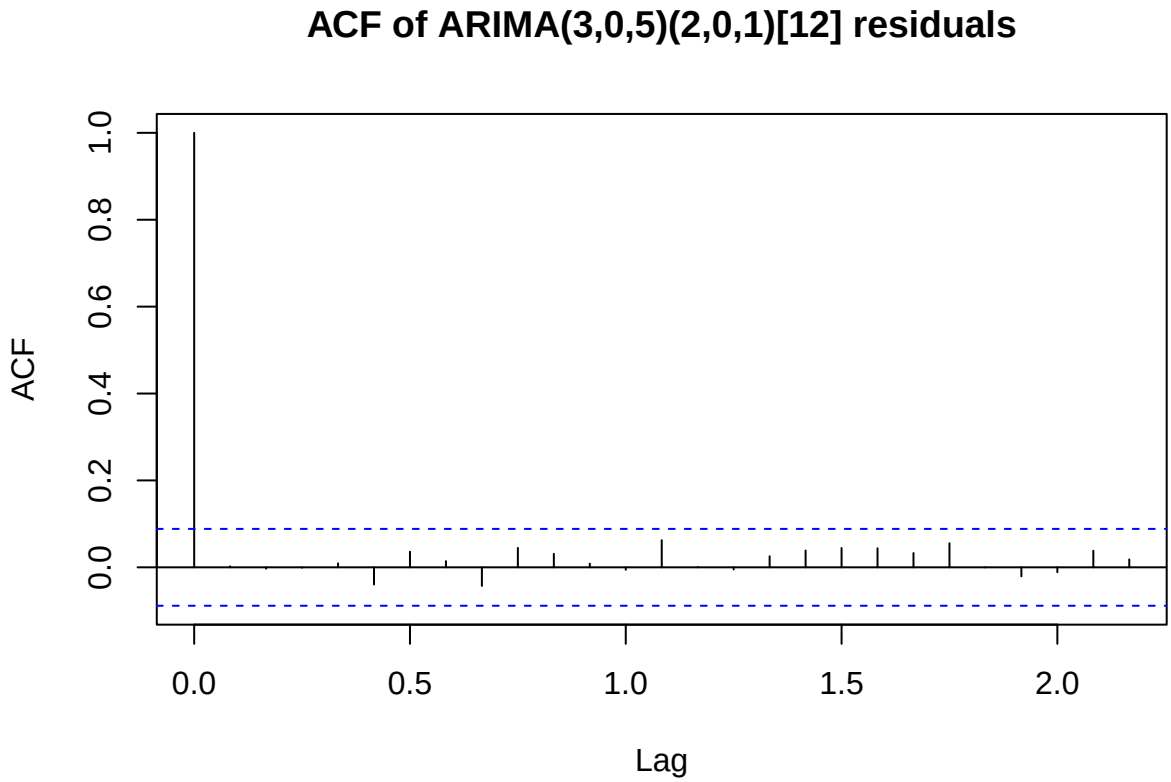

```
# 2. ACF PLOT
# ACF and PACF of training set
acf(montreal_train_ts, main = "ACF Plot: of Log-differenced NHPI: Montreal Training Set")
```



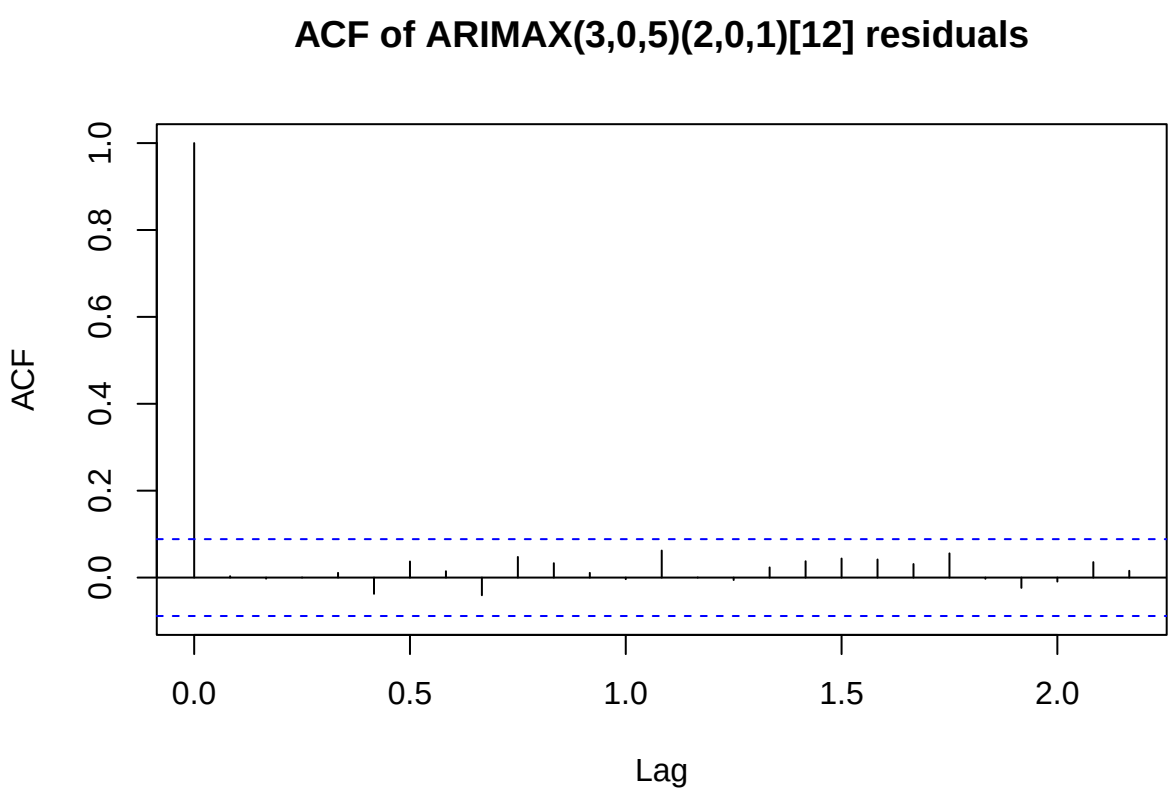

```
pacf(montreal_train_ts, main = "PACF Plot: of Log-differenced NHPI: Montreal Training Set")
```




```
# ACF from best ARIMA model
resid_fit <- residuals(montreal_transformed_nphi_auto)
acf(resid_fit, main = "ACF of ARIMA(3,0,5)(2,0,1)[12] residuals")
```




```r
# ACF from best ARIMAX model
resid_fit <- residuals(montreal_fit_arimax)
acf(resid_fit, main = "ACF of ARIMAX(3,0,5)(2,0,1)[12] residuals")
```



```r
# 3. SUMMARY STATISTICS
summary(montreal_train_ts)
```

```
##      Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
## -0.012308  0.000000  0.001907  0.003305  0.005096  0.071182
```

```r
summary(montreal_holdout_ts)
```

```
##       Min.    1st Qu.     Median       Mean    3rd Qu.       Max.
## -0.0066050 -0.0008246  0.0000000  0.0018622  0.0011569  0.0316205
```

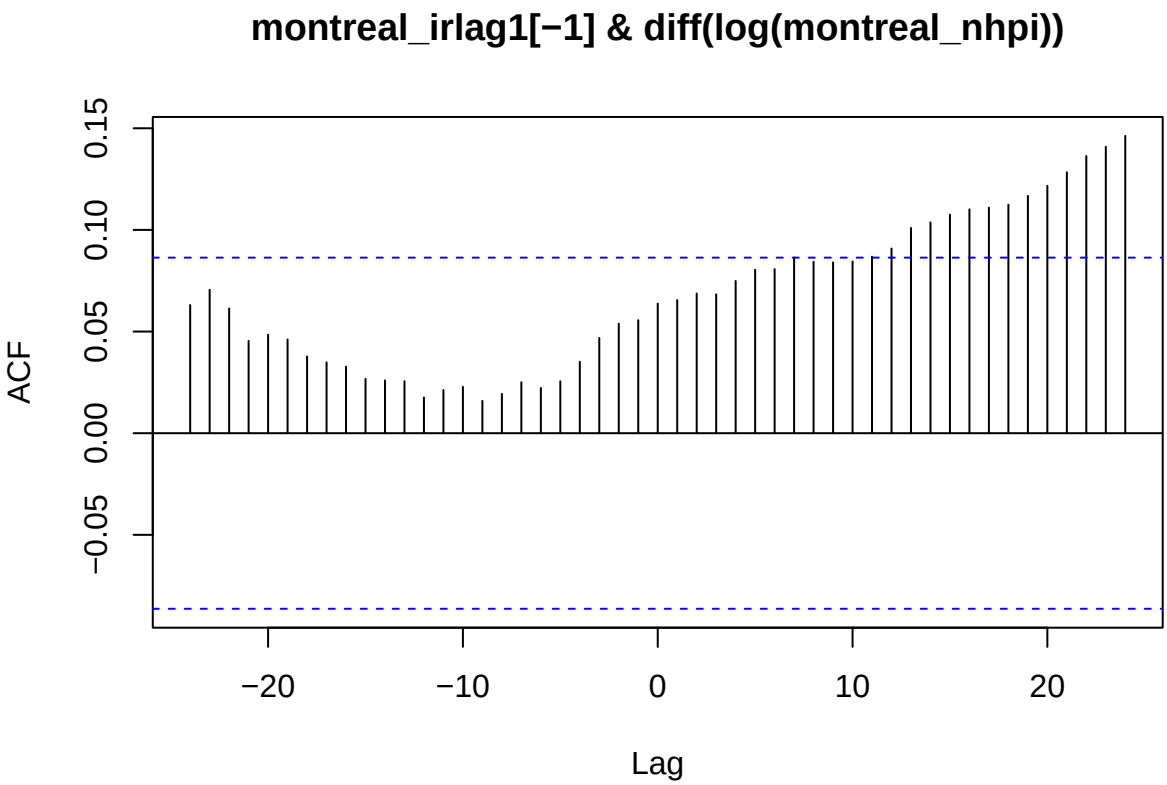
```r
# summary of best ARIMA model
summary(montreal_transformed_nphi_auto)
```

```
## Series: montreal_train_ts
## ARIMA(2,0,5)(2,0,1)[12] with non-zero mean
##
## Coefficients:
##          ar1      ar2      ma1     ma2     ma3      ma4     ma5    sar1    sar2
##       1.2933  -0.4683  -1.1324  0.3576  0.0775  -0.0112  0.1526  0.7129  0.0270
## s.e.  0.1615   0.1643   0.1629  0.1522  0.0680   0.0678  0.0628  0.2571  0.0723
##          sma1    mean
##       -0.6604  0.0034
## s.e.   0.2622  0.0007
##
## sigma^2 = 2.434e-05:  log likelihood = 1916.27
## AIC=-3808.54   AICc=-3807.89   BIC=-3758.19
##
## Training set error measures:
##                        ME        RMSE         MAE MPE MAPE      MASE
## Training set -4.22165e-05 0.004878312 0.002960714 NaN  Inf 0.7689245
##                     ACF1
## Training set 0.002456102
```

```r
# summary of best ARIMAX model
summary(montreal_fit_arimax)
```

```
## Series: montreal_train_ts
## Regression with ARIMA(2,0,5)(2,0,1)[12] errors
##
## Coefficients:
##          ar1      ar2      ma1     ma2     ma3      ma4     ma5    sar1    sar2
##       1.2998  -0.4778  -1.1411  0.3668  0.0793  -0.0121  0.1545  0.7148  0.0216
## s.e.  0.1554   0.1577   0.1572  0.1477  0.0684   0.0681  0.0629  0.2831  0.0728
##          sma1  intercept   xreg
##       -0.6608     0.0028  1e-04
## s.e.   0.2890     0.0010  1e-04
##
## sigma^2 = 2.435e-05:  log likelihood = 1916.72
## AIC=-3807.45   AICc=-3806.69   BIC=-3752.9
##
## Training set error measures:
##                         ME        RMSE         MAE MPE MAPE     MASE
## Training set -2.873017e-05 0.004873876 0.002945387 NaN  Inf 0.764944
##                     ACF1
## Training set 0.003074693
```

```r
# CCF between log-differenced NHPI and the lagged interest rate
ccf(montreal_irlag1[-1], diff(log(montreal_nhpi)))
```




```r
# 4. RMSE
rmse = cbind(montreal_persist_fc$rmse, montreal_iid_fc$rmse,
             montreal_esm_fc$rmse, arima_results$rmse, arimax_results$rmse)
colnames(rmse) = c("persist", "iid", "esm", "arima", "arimax")
print(round(rmse, 3))
```

```
##   persist   iid   esm arima arimax
## a    0.01 0.008 0.008 0.008  0.008
```

Analyzing Toronto

Setting up

```r
# Reading the data
toronto_nhpi_data <- read.csv("Full_Cities/toronto_full.csv")
toronto_nhpi_ir_data <- read.csv("Extracted_Cities/toronto.csv")

# Extracting the NHPI
toronto_nhpi <- toronto_nhpi_data$VALUE

# Extracting the one-month lag interest rate
toronto_irlag1 <- toronto_nhpi_ir_data$IR_lag1

# Setting the time window from 1981-2023
toronto_ts <- ts(toronto_nhpi, start = c(1981,1), end = c(2023, 12), frequency = 12)
toronto_ir_ts <- ts(toronto_irlag1, start = c(1981,1), end = c(2023, 12), frequency = 12)
```

```r
# Applying a transformation for NHPI
toronto_transformed_nhpi <- diff(log(toronto_ts))

# Setting a training set from 1981-2021
toronto_train_ts <- window(toronto_transformed_nhpi, end = c(2021,12))
toronto_train_ir_xreg <- window(toronto_ir_ts, start = c(1981,2), end = c(2021,12))

# Setting a holdout set from 2022-2023
toronto_holdout_ts <- window(toronto_transformed_nhpi, start = c(2022,1))
toronto_holdout_ir_xreg <- window(toronto_ir_ts, start = c(2022,1))
```

Doing the baseline forecasts

```r
# Baseline 1
toronto_persist_fc <- persist_fc(toronto_train_ts, toronto_holdout_ts, iprint = F)

# Baseline 2
toronto_iid_fc <- iid_fc(toronto_train_ts, toronto_holdout_ts, iprint = F)
```

Doing the exponential smoothing forecasts

```r
# Method 1
ntrain <- length(toronto_train_ts)
ntotal <- length(toronto_ts)-1

toronto_transformed_nhpi_train <- toronto_transformed_nhpi[1:ntrain]
toronto_transformed_nhpi_holdout <- toronto_transformed_nhpi[(ntrain+1):ntotal]

# Simple Exponential Smoothing
fit_expsmo <- HoltWinters(toronto_train_ts, beta = F, gamma = F)
toronto_esm_fc <- esm_fc(toronto_transformed_nhpi_train, toronto_transformed_nhpi_holdout,
                         alpha = fit_expsmo$alpha, level = fit_expsmo$coefficients["a"],
                         iprint = F)

# Linear Exponential Smoothing
fit_hw <- HoltWinters(toronto_train_ts, gamma = F)
toronto_holt_fc <- lholt_fc(toronto_transformed_nhpi_train, toronto_transformed_nhpi_holdout,
                            alpha = fit_hw$alpha, beta = fit_hw$beta,
                            level = fit_hw$coefficients["a"],
                            slope = fit_hw$coefficients["b"], iprint = F)

# Additive Seasonal Exponential Smoothing
wafit <- HoltWinters(toronto_train_ts, seasonal = "additive")
toronto_aseason_fc <- aseason_fc(toronto_transformed_nhpi_train,
                                 toronto_transformed_nhpi_holdout,
                                 alpha = wafit$alpha, beta = wafit$beta,
                                 gamma = wafit$gamma, level = wafit$coef[1],
                                 slope = wafit$coef[2], season = wafit$coefficient[3:14],
                                 iprint = F)

# Comparing RMSE
rmse_esm <- cbind(toronto_esm_fc$rmse, toronto_holt_fc$rmse, toronto_aseason_fc$rmse)
```

```r
colnames(rmse_esm) <- c("esm", "lholt", "aseason")
rmse_esm
```

```
##           esm       lholt     aseason
## a 0.002166508 0.002334658 0.002612987
```

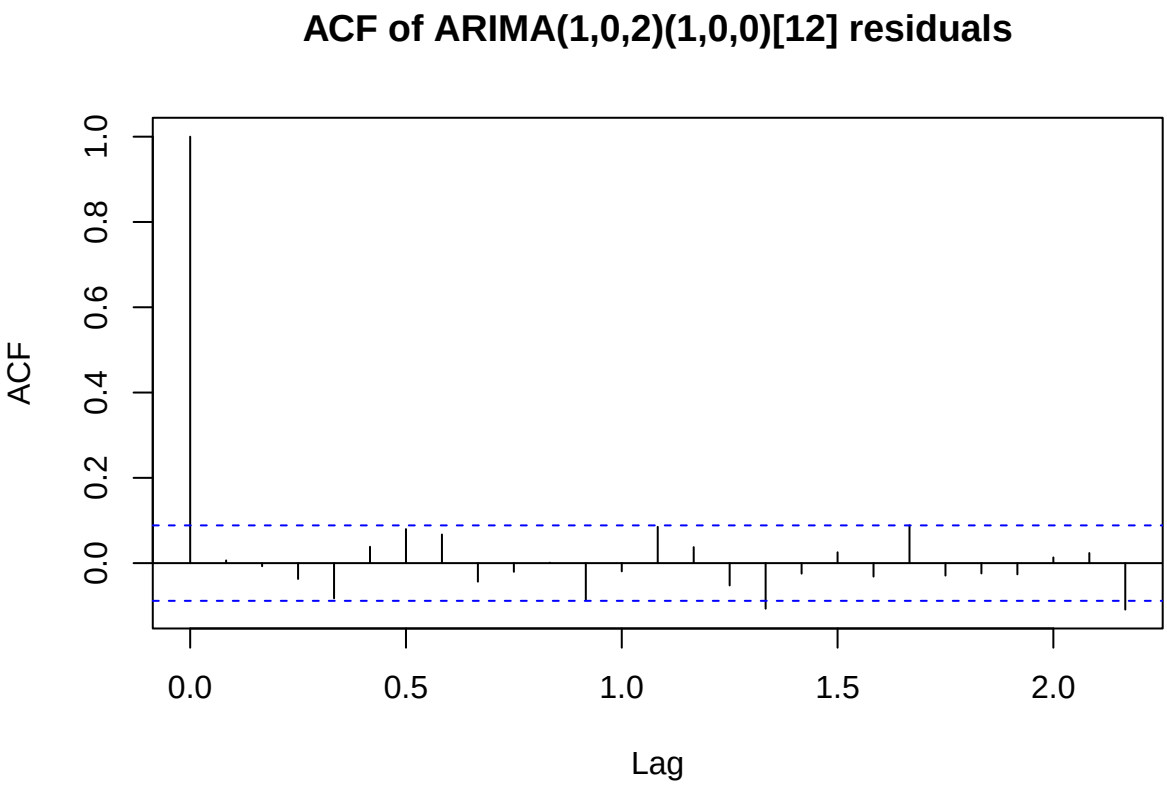
```r
# Best exponential smoothing method: Simple Exponential Smoothing
```

Doing the ARIMA forecasts

```r
# Method 2
toronto_transformed_nphi_auto <- forecast::auto.arima(toronto_train_ts)
summary(toronto_transformed_nphi_auto) # 0.0064057
```

```
## Series: toronto_train_ts
## ARIMA(1,0,2)(1,0,0)[12] with non-zero mean
##
## Coefficients:
##          ar1      ma1      ma2    sar1    mean
##       0.9161  -0.4317  -0.1386  0.1316  0.0034
## s.e.  0.0302   0.0551   0.0490  0.0487  0.0017
##
## sigma^2 = 4.146e-05:  log likelihood = 1782.61
## AIC=-3553.23   AICc=-3553.05   BIC=-3528.05
##
## Training set error measures:
##                        ME        RMSE         MAE MPE MAPE      MASE
## Training set -0.0001299236 0.006405744 0.003868432 NaN  Inf 0.6292045
##                     ACF1
## Training set 0.006342322
```

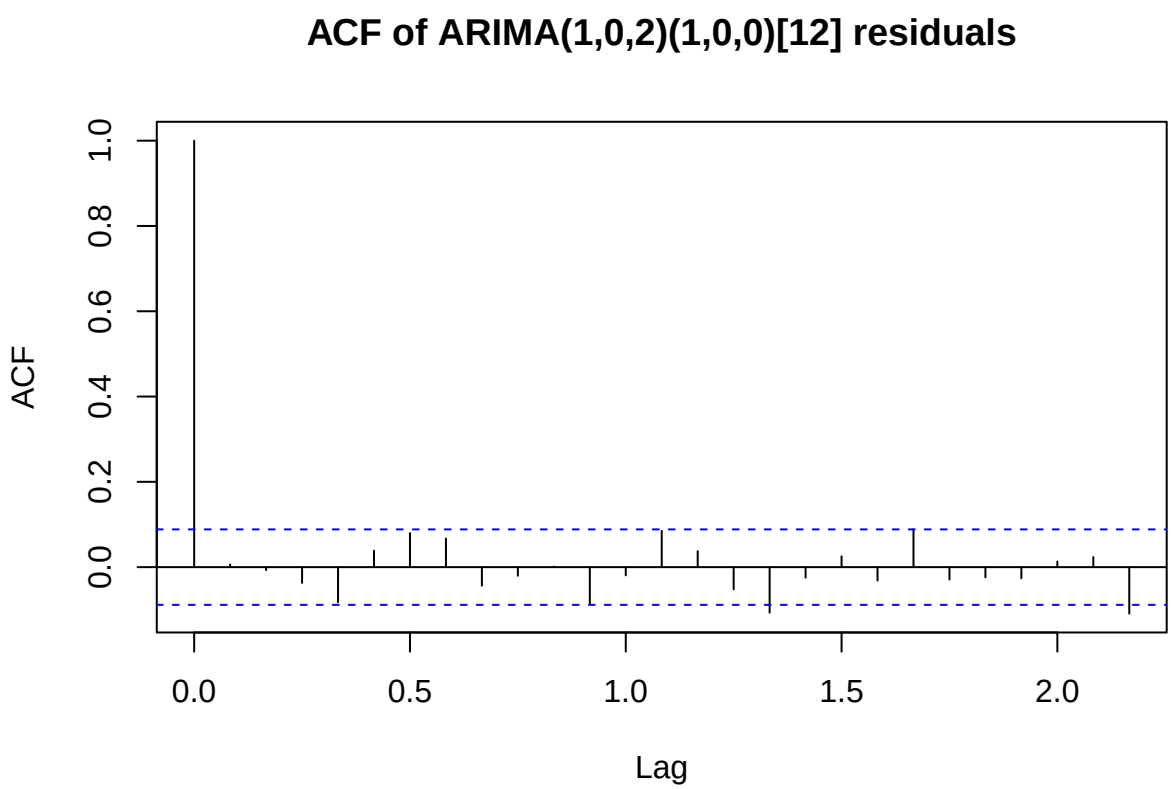
```r
resid_fit <- residuals(toronto_transformed_nphi_auto)
acf(resid_fit, main = "ACF of ARIMA(1,0,2)(1,0,0)[12] residuals")
```




```r
# Comparing with the other ARIMA models
manual_fit <- arima(toronto_train_ts, order = c(1,0,2),
                    seasonal = list(order = c(1,0,0), period = 12), method = "ML")
summary(manual_fit) # RMSE: 0.006405747
```

```
##
## Call:
## arima(x = toronto_train_ts, order = c(1, 0, 2), seasonal = list(order = c(1,
##     0, 0), period = 12), method = "ML")
##
## Coefficients:
##          ar1      ma1      ma2    sar1  intercept
##       0.9160  -0.4315  -0.1388  0.1316     0.0034
## s.e.  0.0302   0.0551   0.0490  0.0487     0.0017
##
## sigma^2 estimated as 4.103e-05:  log likelihood = 1782.61,  aic = -3553.23
##
## Training set error measures:
##                        ME        RMSE         MAE MPE MAPE      MASE
## Training set -0.0001276383 0.006405747 0.003868164 NaN  Inf 0.6291608
##                     ACF1
## Training set 0.006235644
```

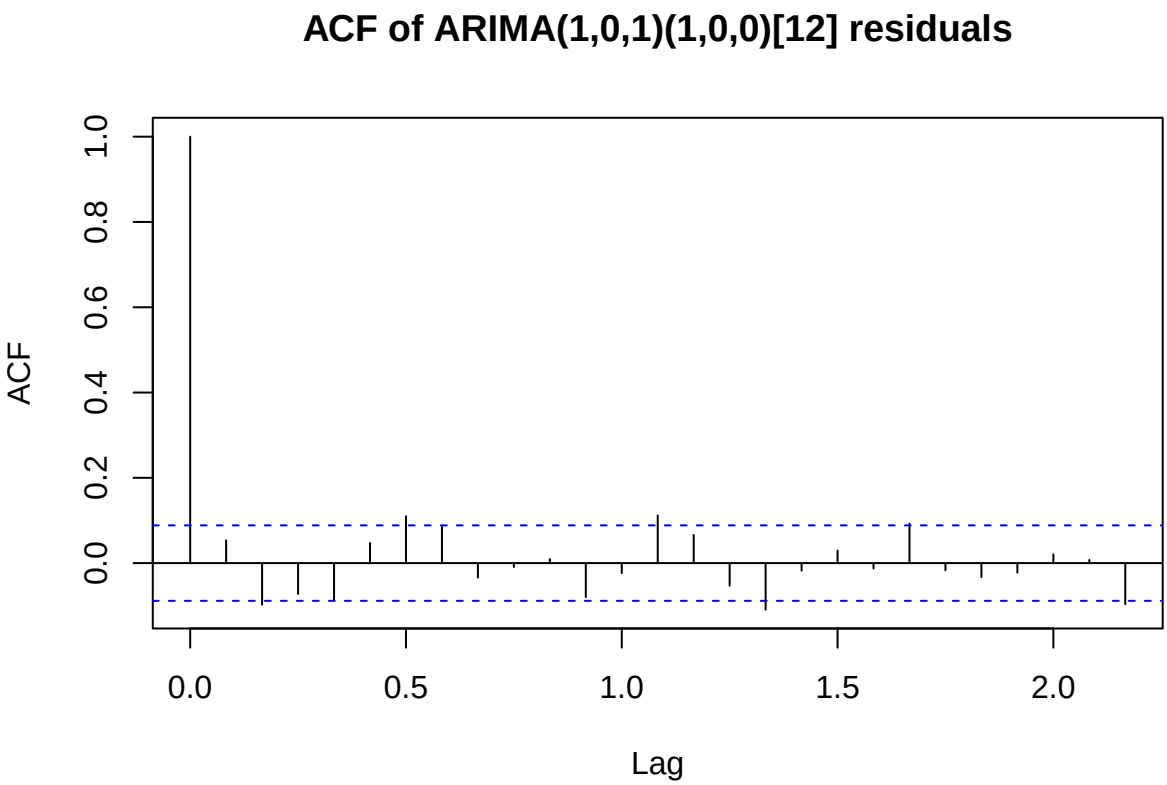
```r
acf(residuals(manual_fit), main = "ACF of ARIMA(1,0,2)(1,0,0)[12] residuals")
```




```r
test1 <- arima(toronto_train_ts, order = c(1,0,1),
               seasonal = list(order = c(1,0,0), period = 12), method = "ML")
summary(test1) # RMSE: 0.006456026
```

```
##
## Call:
## arima(x = toronto_train_ts, order = c(1, 0, 1), seasonal = list(order = c(1,
##     0, 0), period = 12), method = "ML")
##
## Coefficients:
##          ar1      ma1    sar1  intercept
##       0.8636  -0.4245  0.1466     0.0033
## s.e.  0.0434   0.0829  0.0482     0.0014
##
## sigma^2 estimated as 4.168e-05:  log likelihood = 1778.8,  aic = -3547.61
##
## Training set error measures:
##                        ME        RMSE         MAE MPE MAPE     MASE       ACF1
## Training set -0.0001074825 0.006456026 0.003832997 NaN  Inf 0.623441 0.05363762
```

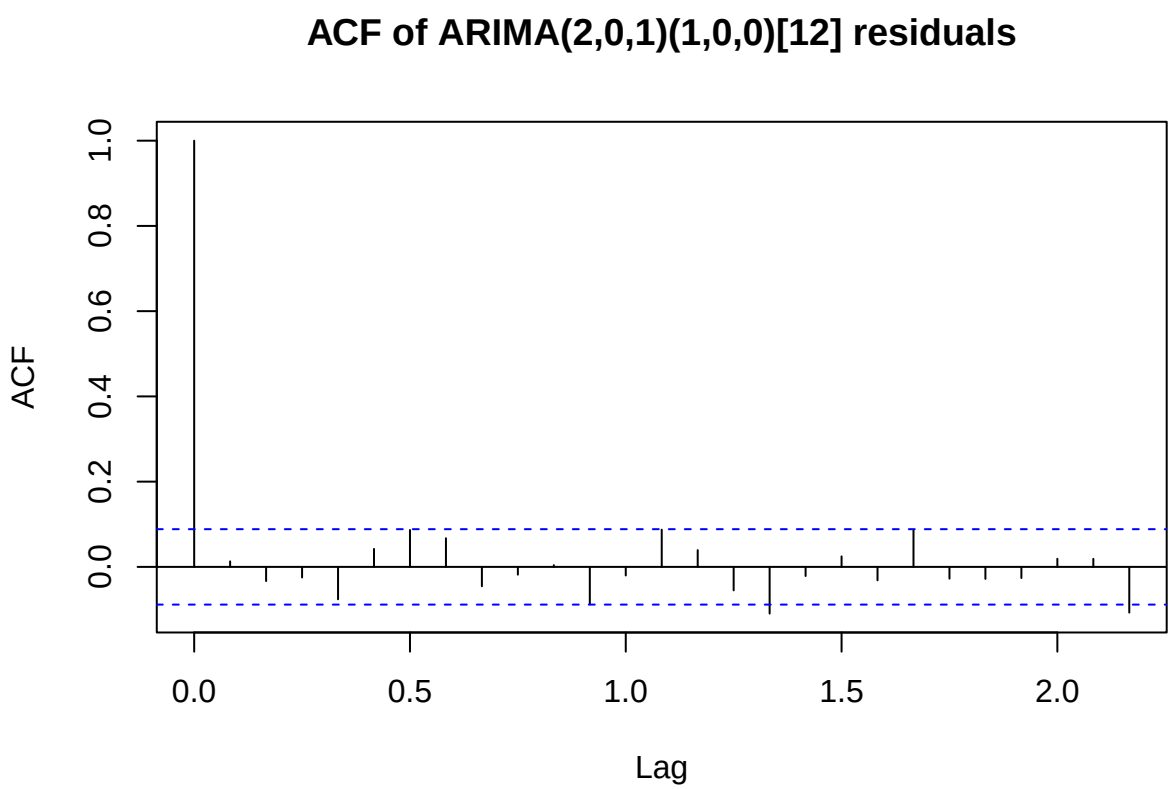
```r
acf(residuals(test1), main = "ACF of ARIMA(1,0,1)(1,0,0)[12] residuals")
```




```r
test2 <- arima(toronto_train_ts, order = c(2,0,1),
               seasonal = list(order = c(1,0,0), period = 12), method = "ML")
summary(test2) # RMSE: 0.006410017
```

```
##
## Call:
## arima(x = toronto_train_ts, order = c(2, 0, 1), seasonal = list(order = c(1,
##     0, 0), period = 12), method = "ML")
##
## Coefficients:
##          ar1      ar2      ma1    sar1  intercept
##       1.1623  -0.2244  -0.6839  0.1292     0.0034
## s.e.  0.0959   0.0782   0.0792  0.0489     0.0017
##
## sigma^2 estimated as 4.109e-05:  log likelihood = 1782.3,  aic = -3552.6
##
## Training set error measures:
##                        ME        RMSE         MAE MPE MAPE      MASE       ACF1
## Training set -0.000128604 0.006410017 0.003868191 NaN  Inf 0.6291652 0.01298083
```

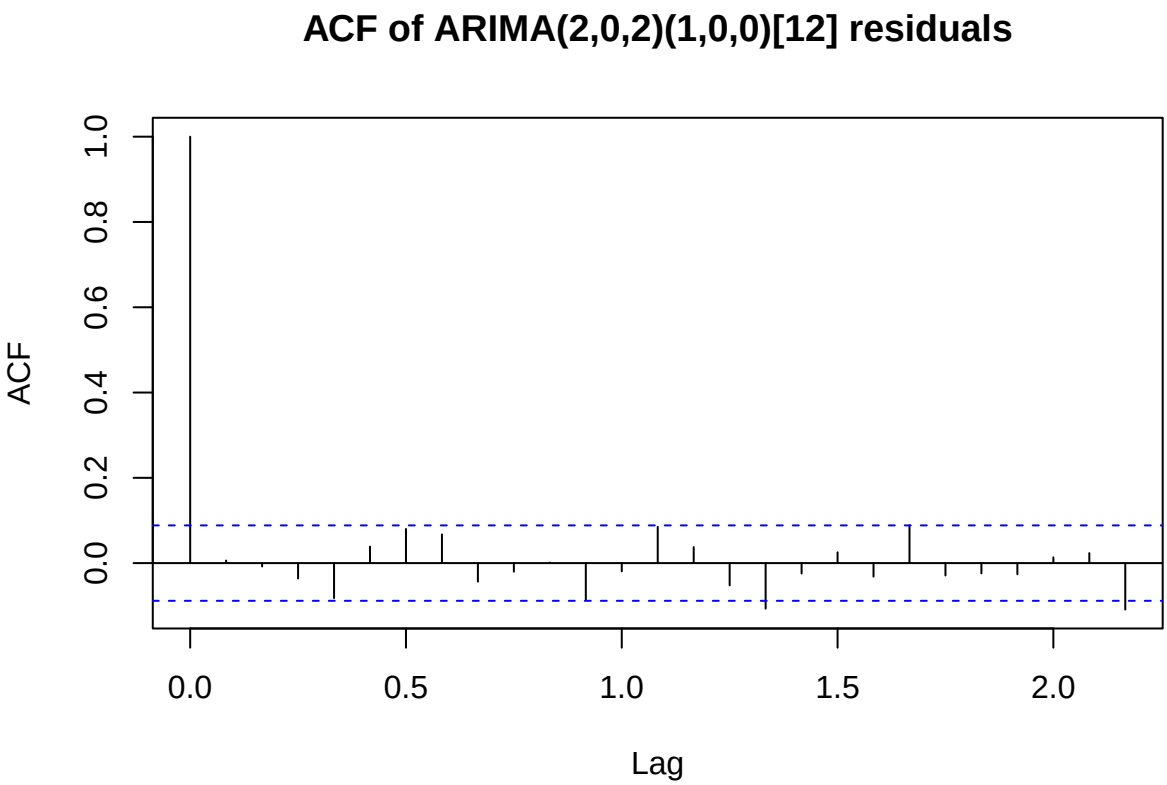
```r
acf(residuals(test2), main = "ACF of ARIMA(2,0,1)(1,0,0)[12] residuals")
```




```r
test3 <- arima(toronto_train_ts, order = c(2,0,2),
               seasonal = list(order = c(1,0,0), period = 12), method = "ML")
summary(test3) # RMSE: 0.006405754
```

```
##
## Call:
## arima(x = toronto_train_ts, order = c(2, 0, 2), seasonal = list(order = c(1,
##     0, 0), period = 12), method = "ML")
##
## Coefficients:
##          ar1      ar2      ma1      ma2    sar1  intercept
##       0.9255  -0.0089  -0.4408  -0.1336  0.1316     0.0035
## s.e.  0.3544   0.3170   0.3512   0.1762  0.0491     0.0017
##
## sigma^2 estimated as 4.103e-05:  log likelihood = 1782.61,  aic = -3551.23
##
## Training set error measures:
##                        ME        RMSE        MAE MPE MAPE      MASE
## Training set -0.0001365834 0.006405754 0.00386938 NaN  Inf 0.6293587
##                     ACF1
## Training set 0.006071536
```

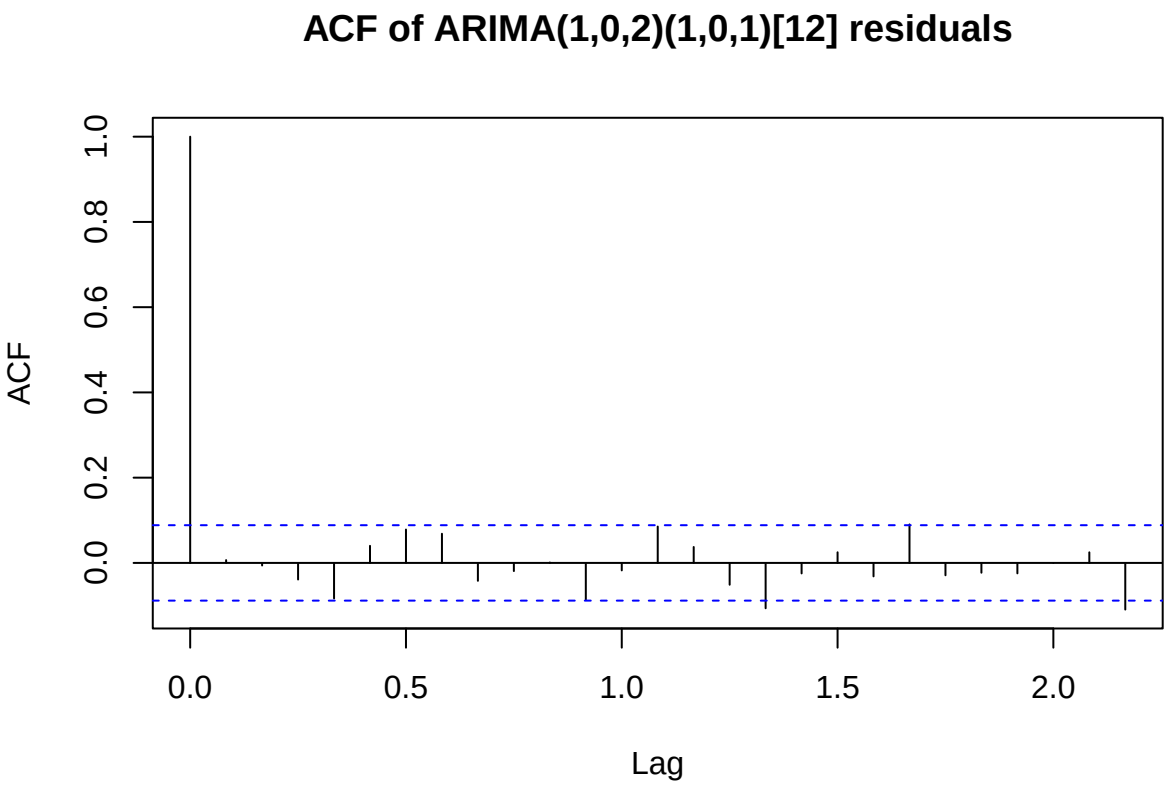
```r
acf(residuals(test3), main = "ACF of ARIMA(2,0,2)(1,0,0)[12] residuals")
```




```r
test4 <- arima(toronto_train_ts, order = c(1,0,2),
               seasonal = list(order = c(1,0,1), period = 12), method = "ML")
summary(test4) # RMSE: 0.006404998
```

```
##
## Call:
## arima(x = toronto_train_ts, order = c(1, 0, 2), seasonal = list(order = c(1,
##     0, 1), period = 12), method = "ML")
##
## Coefficients:
##          ar1      ma1      ma2    sar1     sma1  intercept
##       0.9160  -0.4317  -0.1376  0.2437  -0.1140     0.0035
## s.e.  0.0303   0.0551   0.0491  0.3689   0.3775     0.0017
##
## sigma^2 estimated as 4.102e-05:  log likelihood = 1782.67,  aic = -3551.33
##
## Training set error measures:
##                        ME        RMSE         MAE MPE MAPE      MASE
## Training set -0.0001376316 0.006404998 0.003866659 NaN  Inf 0.6289161
##                     ACF1
## Training set 0.006611957
```

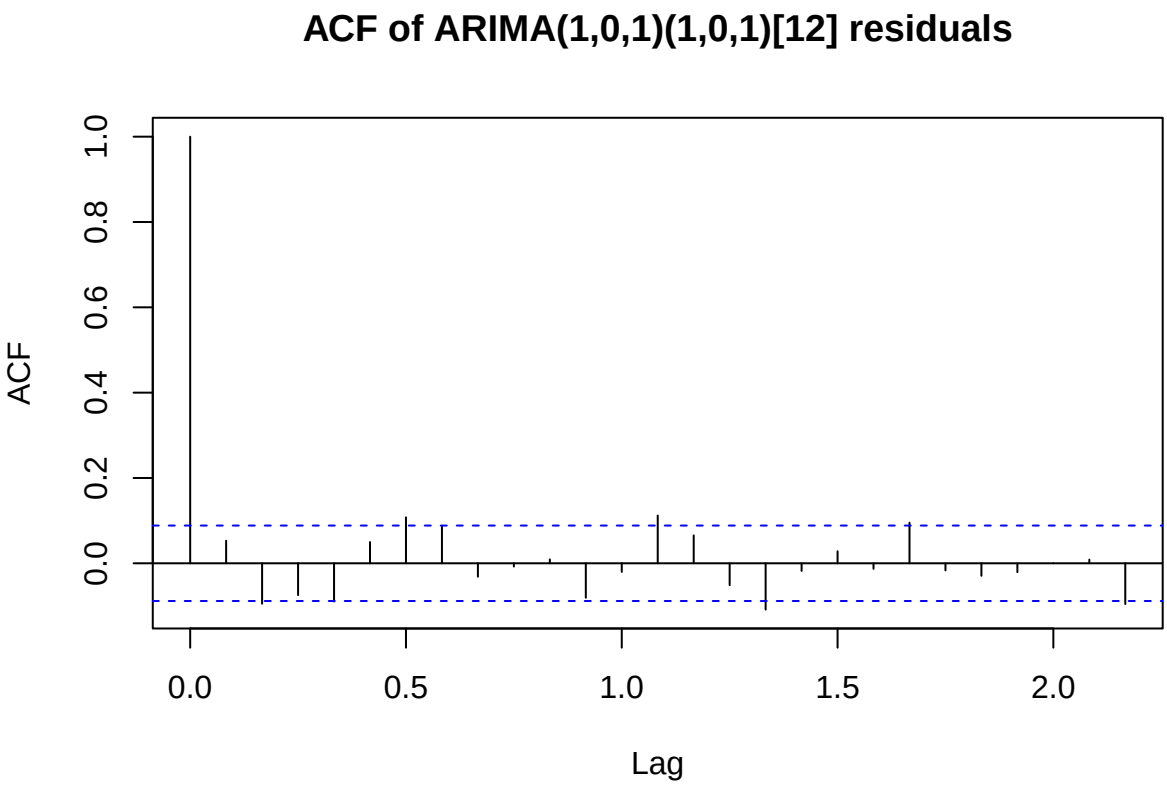
```r
acf(residuals(test4), main = "ACF of ARIMA(1,0,2)(1,0,1)[12] residuals")
```




```r
test5 <- arima(toronto_train_ts, order = c(1,0,1),
               seasonal = list(order = c(1,0,1), period = 12), method = "ML")
summary(test5) # RMSE: 0.006454211
```

```
##
## Call:
## arima(x = toronto_train_ts, order = c(1, 0, 1), seasonal = list(order = c(1,
##     0, 1), period = 12), method = "ML")
##
## Coefficients:
##          ar1      ma1    sar1     sma1  intercept
##       0.8634  -0.4227  0.3043  -0.1614     0.0033
## s.e.  0.0434   0.0830  0.2853   0.2956     0.0015
##
## sigma^2 estimated as 4.166e-05:  log likelihood = 1778.93,  aic = -3545.87
##
## Training set error measures:
##                        ME        RMSE         MAE MPE MAPE      MASE
## Training set -0.0001083458 0.006454211 0.003830568 NaN  Inf 0.6230459
##                    ACF1
## Training set 0.05289892
```

```r
acf(residuals(test5), main = "ACF of ARIMA(1,0,1)(1,0,1)[12] residuals")
```




```r
# Comparing AIC values
aic = cbind(manual_fit$aic, test1$aic, test2$aic, test3$aic, test4$aic, test5$aic)
colnames(aic) = c("manual_fit", "test1", "test2", "test3", "test4", "test5")
aic
```

```
##      manual_fit     test1     test2    test3     test4     test5
## [1,]  -3553.228 -3547.608 -3552.598 -3551.23 -3551.334 -3545.868
```

```r
# Best (based on lowest AIC and ACF plot): manual_fit, ARIMA(1,0,2)(1,0,0)[12]

order <- c(1,0,2)
seasonal <- list(order = c(1,0,0), period = 12)
# Fitting the model to the training data
train_arima_model <- arima(toronto_train_ts, order = order,
                           seasonal = seasonal, method = "ML", include.mean = T)
# Getting the result of the moving 1-step ahead forecast for holdout set
arima_results <- arma_fc(tsdata = toronto_transformed_nhpi,
                         ntrain = length(toronto_train_ts), order = order,
                         seasonal = seasonal, method = "ML",
                         traincoef = coef(train_arima_model), include.mean = T)
arima_results$rmse
```

```
## [1] 0.00204875
```

Doing the ARIMAX forecasts with lagged interest

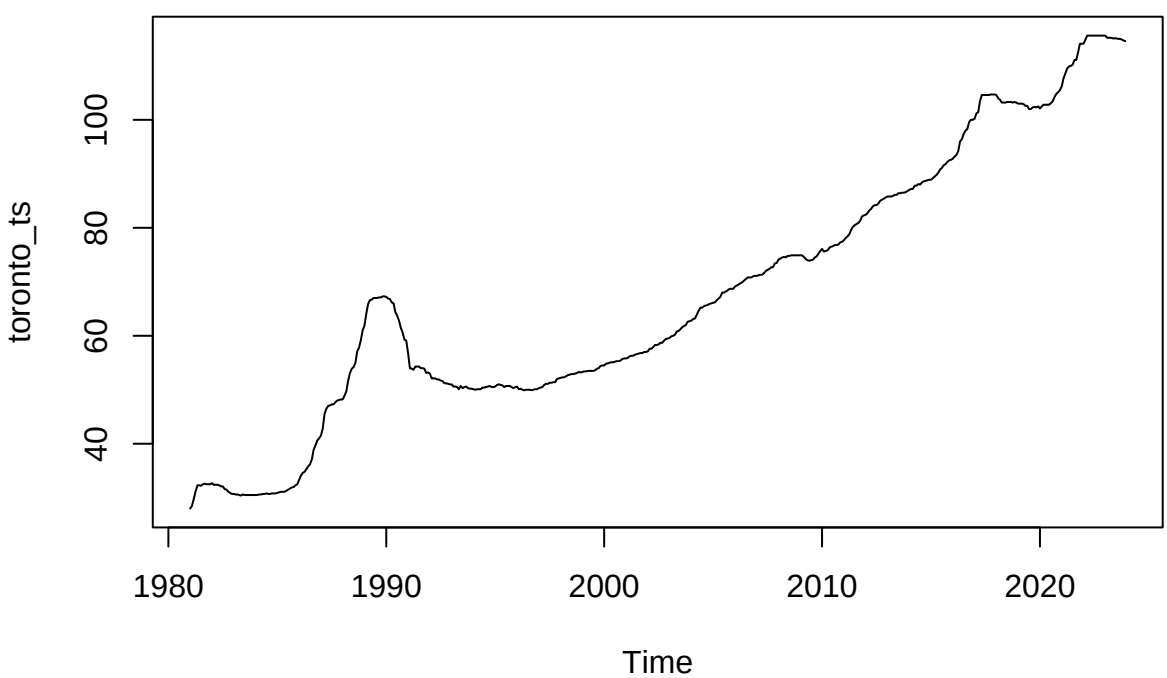
```r
# Method 3
toronto_fit_arimax <- forecast::auto.arima(toronto_train_ts, xreg = toronto_train_ir_xreg)
summary(toronto_fit_arimax)
```

```
## Series: toronto_train_ts
## Regression with ARIMA(1,0,2)(1,0,0)[12] errors
##
## Coefficients:
##          ar1      ma1      ma2    sar1  intercept    xreg
##       0.9257  -0.4415  -0.1406  0.1330     0.0051  -3e-04
## s.e.  0.0329   0.0563   0.0488  0.0489     0.0028   4e-04
##
## sigma^2 = 4.147e-05:  log likelihood = 1783
## AIC=-3551.99   AICc=-3551.76   BIC=-3522.62
##
## Training set error measures:
##                         ME        RMSE         MAE MPE MAPE      MASE
## Training set -0.0001599343 0.006400238 0.003875959 NaN  Inf 0.6304288
##                     ACF1
## Training set 0.006084236
```

```
order <- c(1,0,2)
seasonal <- list(order = c(1,0,0), period = 12)
# Fitting a model to the training data
train_arimax_model <- arima(toronto_train_ts, order = order, seasonal = seasonal, xreg = toronto_train_
# Getting the result of the moving 1-step ahead forecast for holdout set
arimax_results <- armax_fc(tsdata = toronto_transformed_nhpi, ntrain = length(toronto_train_ts), order =
toronto_arima_results <- arima_results
toronto_arimax_results <- arimax_results
```

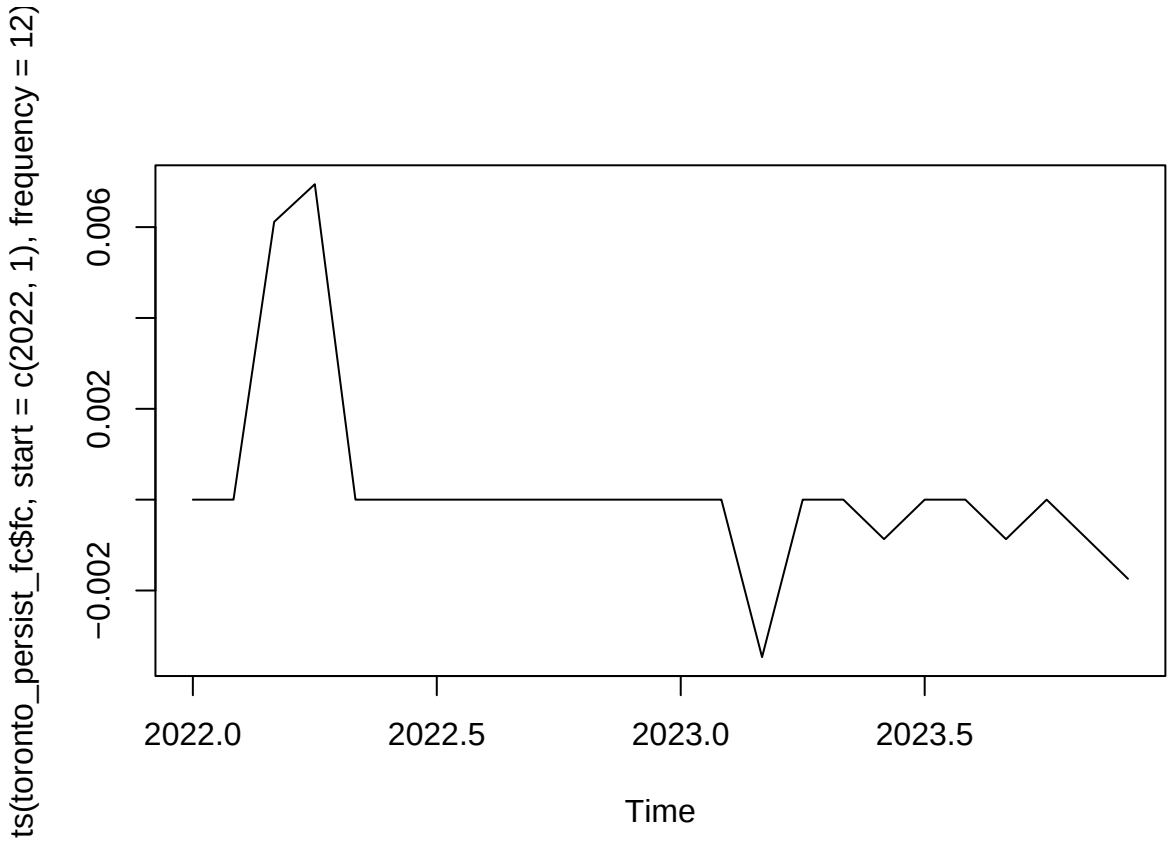
Viewing the results of Toronto's analyses

```
# 1. TIME SERIES PLOT
plot(toronto_ts)
```

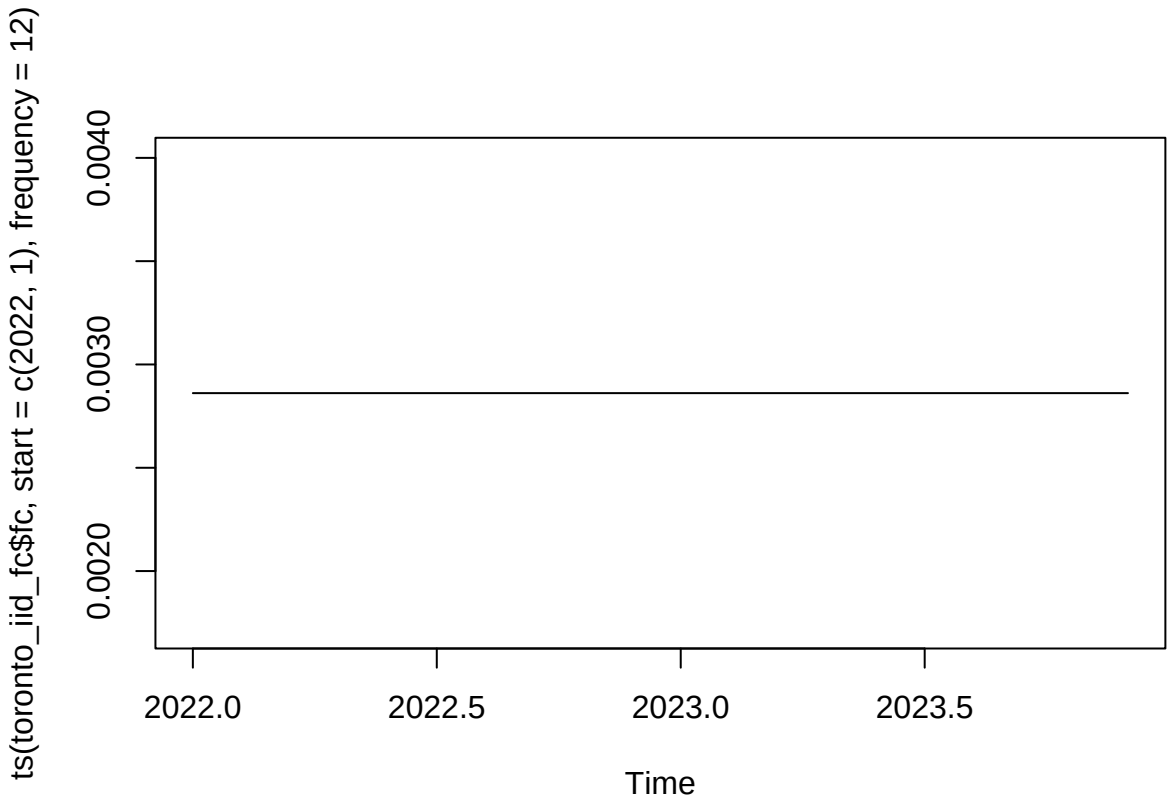



```
# log-diff transformation
# plot(toronto_transformed_ts) # NOTE: this object doesn't exist
```

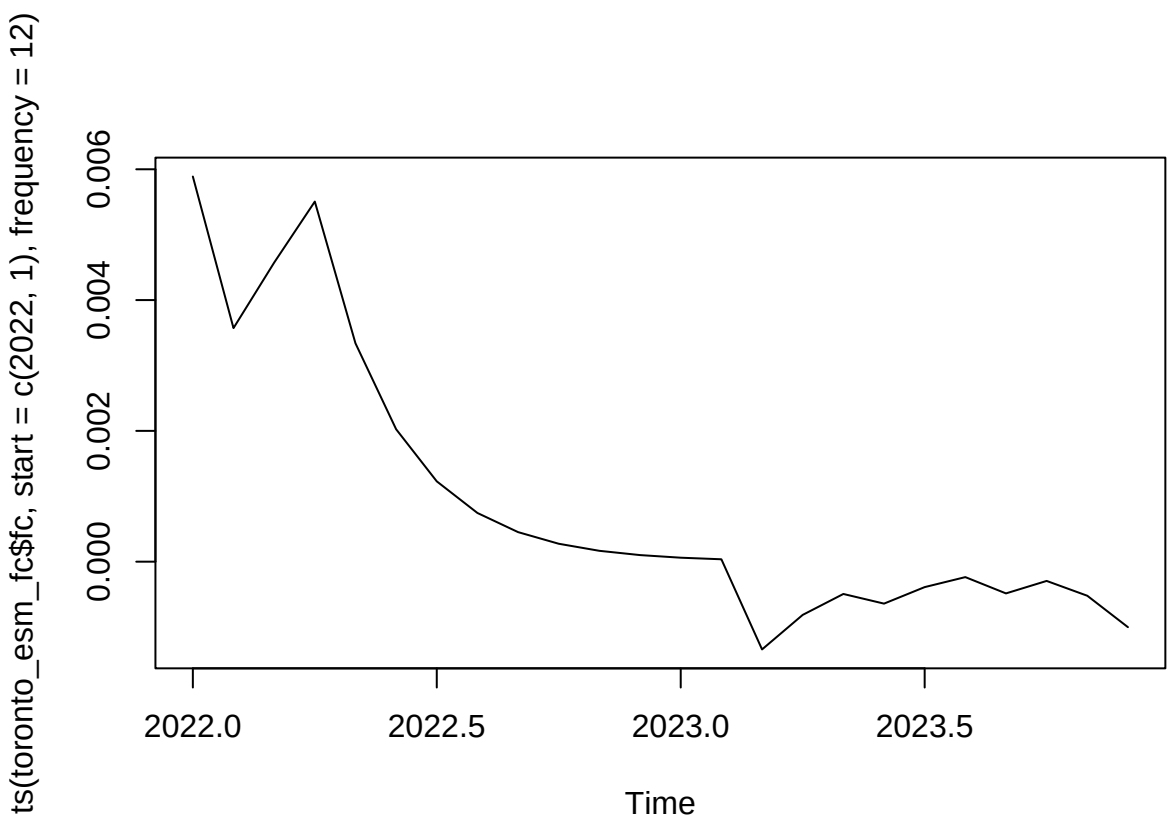
```r
# ts plot from persistence
plot(ts(toronto_persist_fc$fc, start = c(2022, 1), frequency = 12))
```



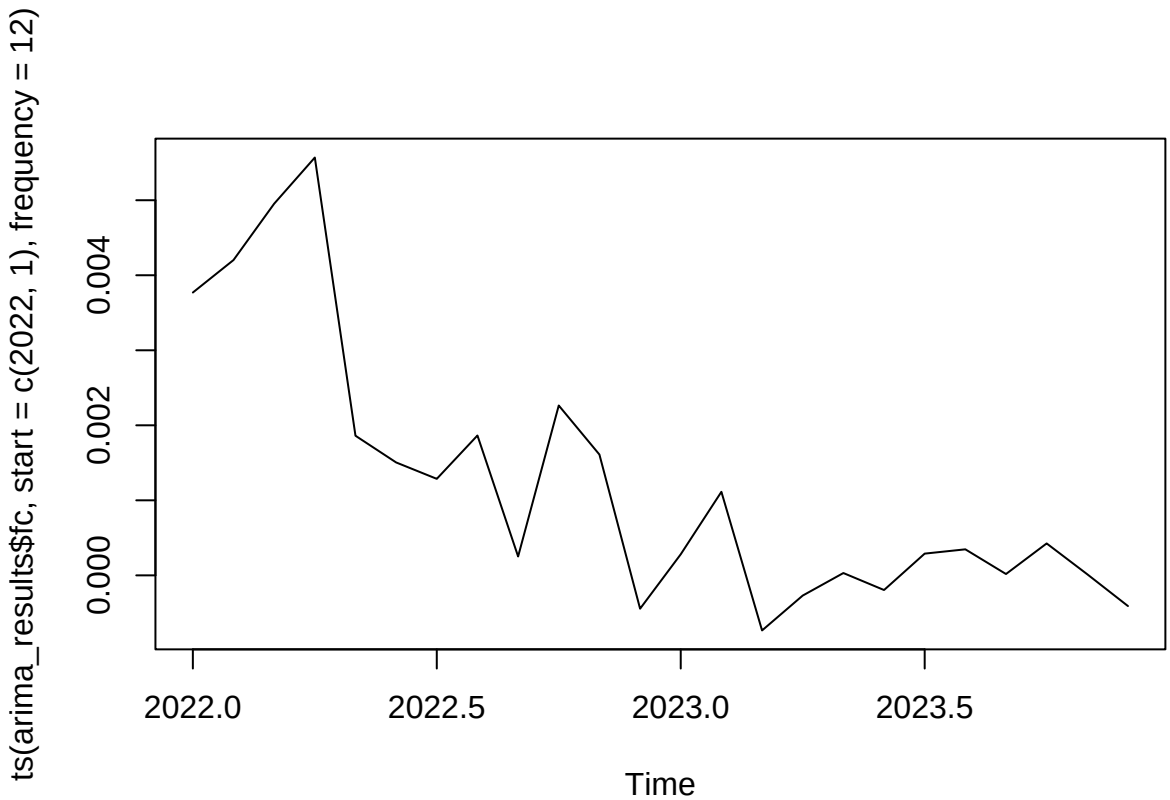

```r
# ts plot from average
plot(ts(toronto_iid_fc$fc, start = c(2022, 1), frequency = 12))
```



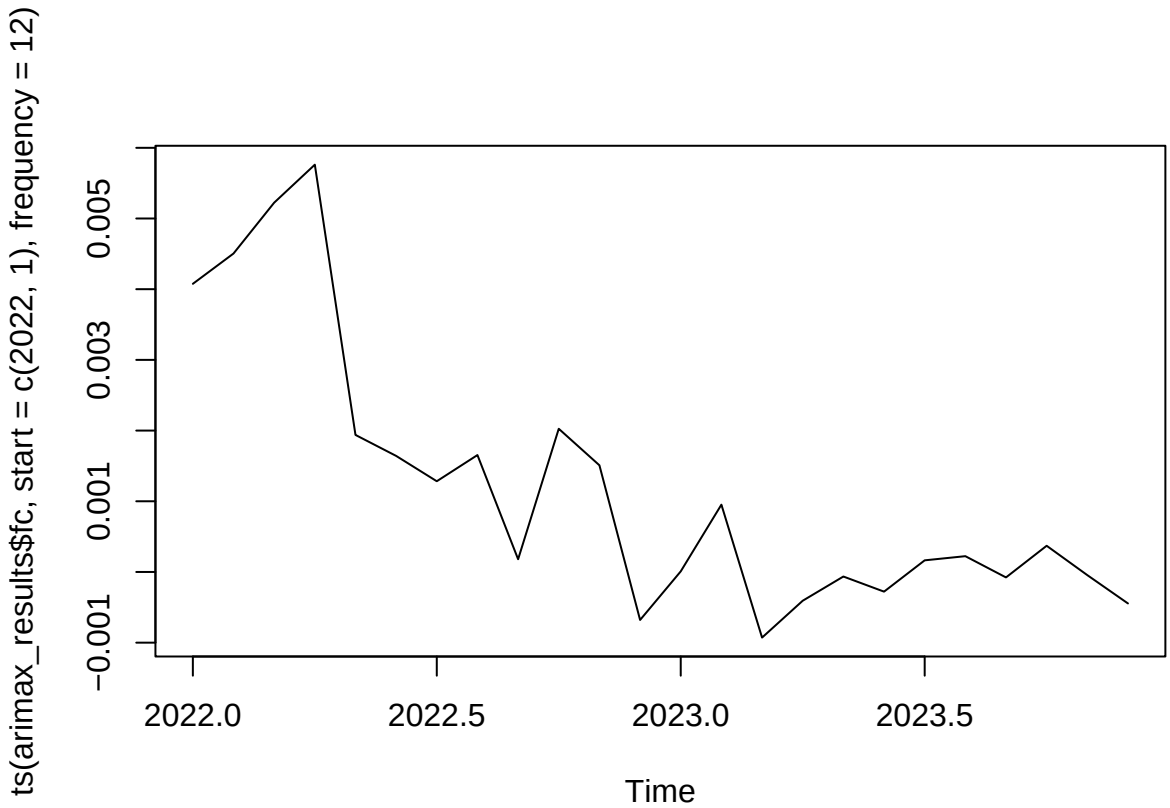

```r
# ts plot from simple exponential smoothing
plot(ts(toronto_esm_fc$fc, start = c(2022, 1), frequency = 12))
```



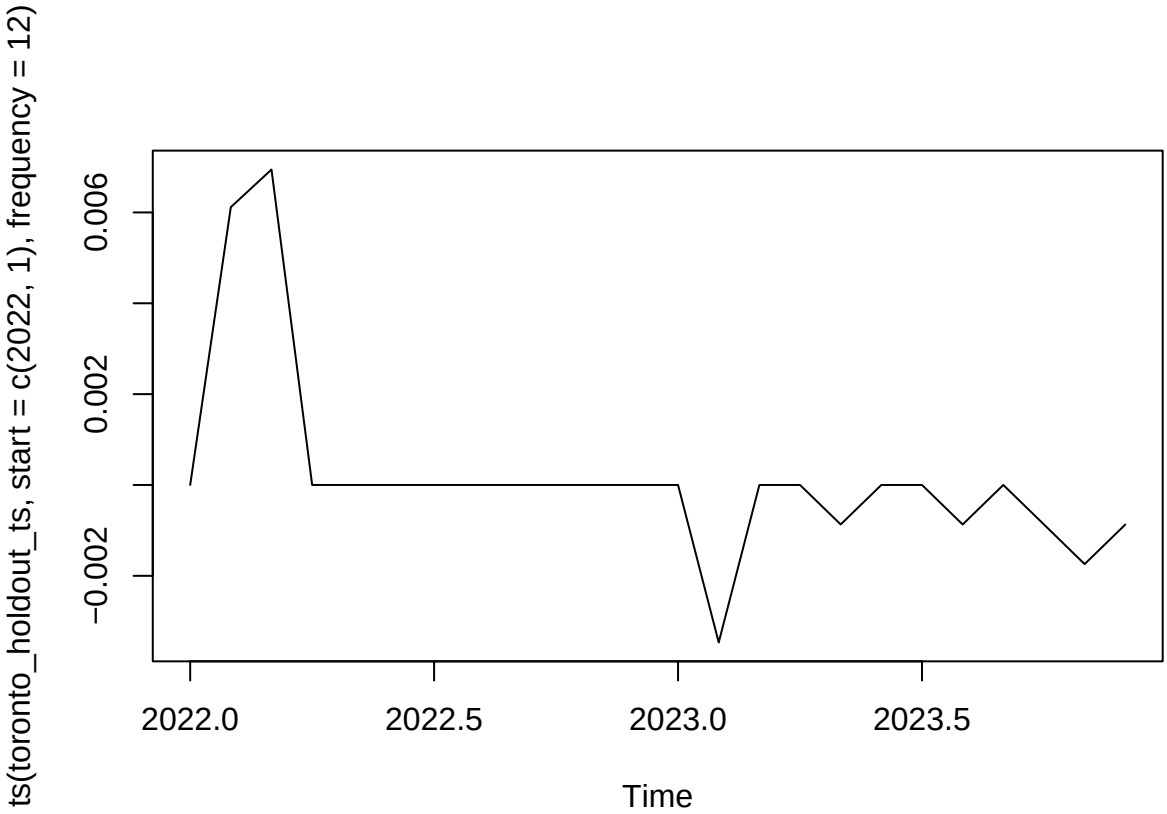
```r
# ts plot from ARIMA
plot(ts(arima_results$fc, start = c(2022, 1), frequency = 12))
```



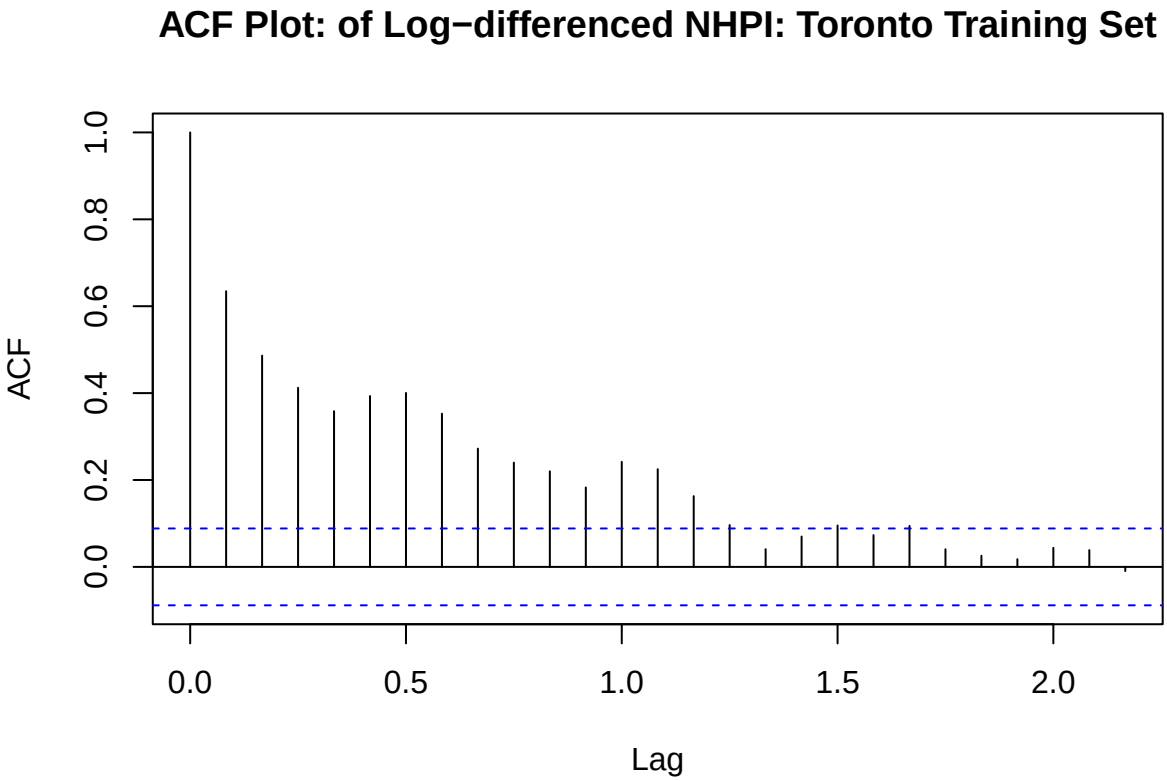

```r
# ts plot from ARIMAX
plot(ts(arimax_results$fc, start = c(2022, 1), frequency = 12))
```



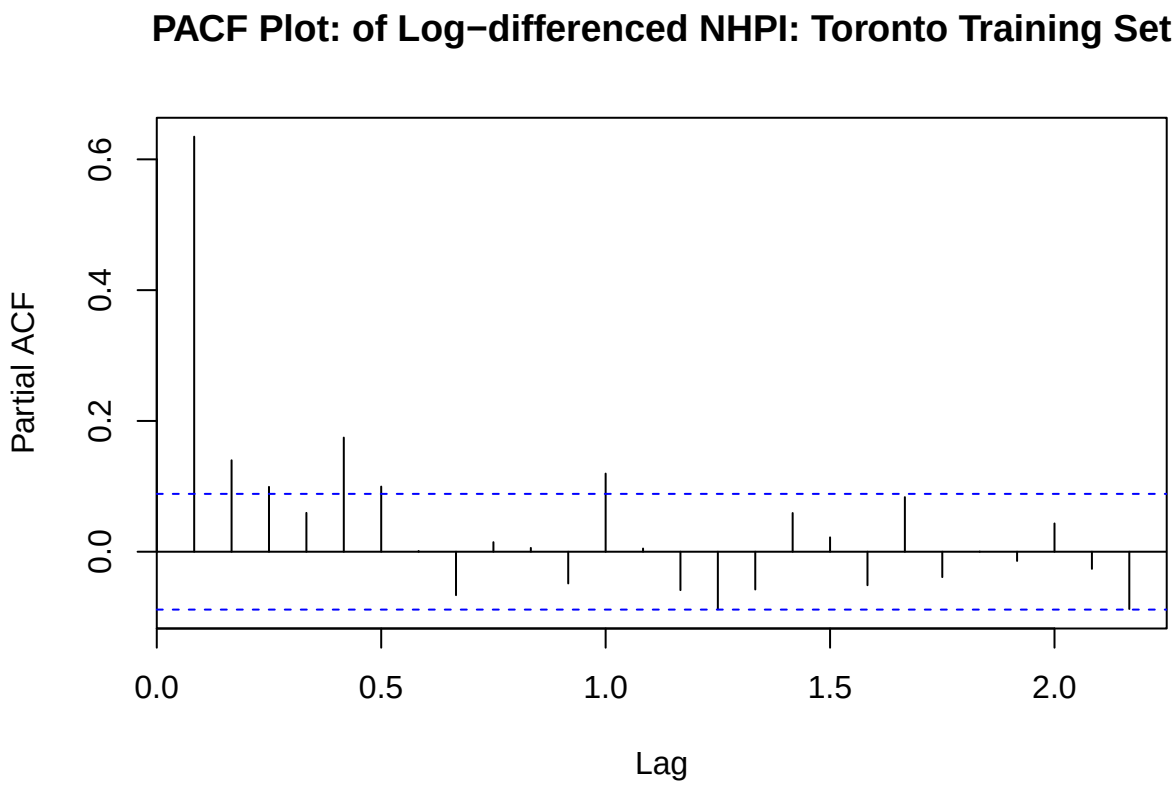

```r
# ts plot of holdout set
plot(ts(toronto_holdout_ts, start = c(2022, 1), frequency = 12))
```



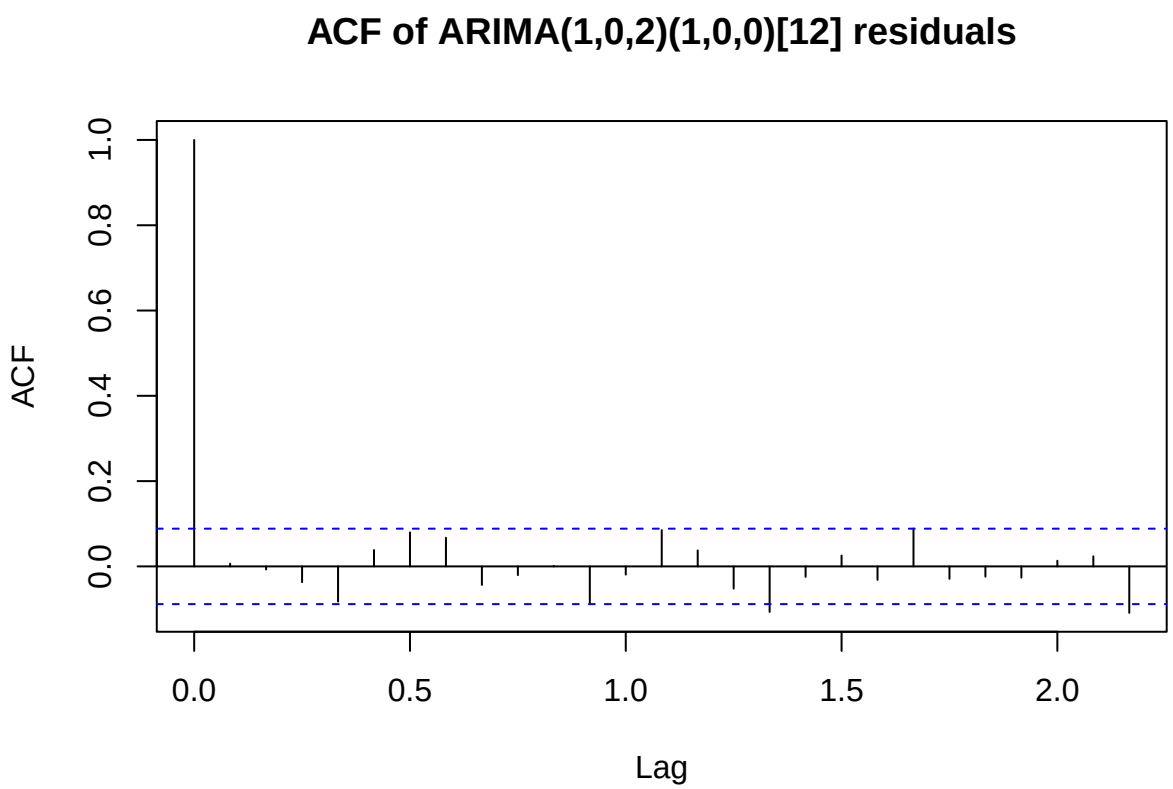
```r
# 2. ACF PLOT
# ACF and PACF of training set
acf(toronto_train_ts, main = "ACF Plot: of Log-differenced NHPI: Toronto Training Set")
```



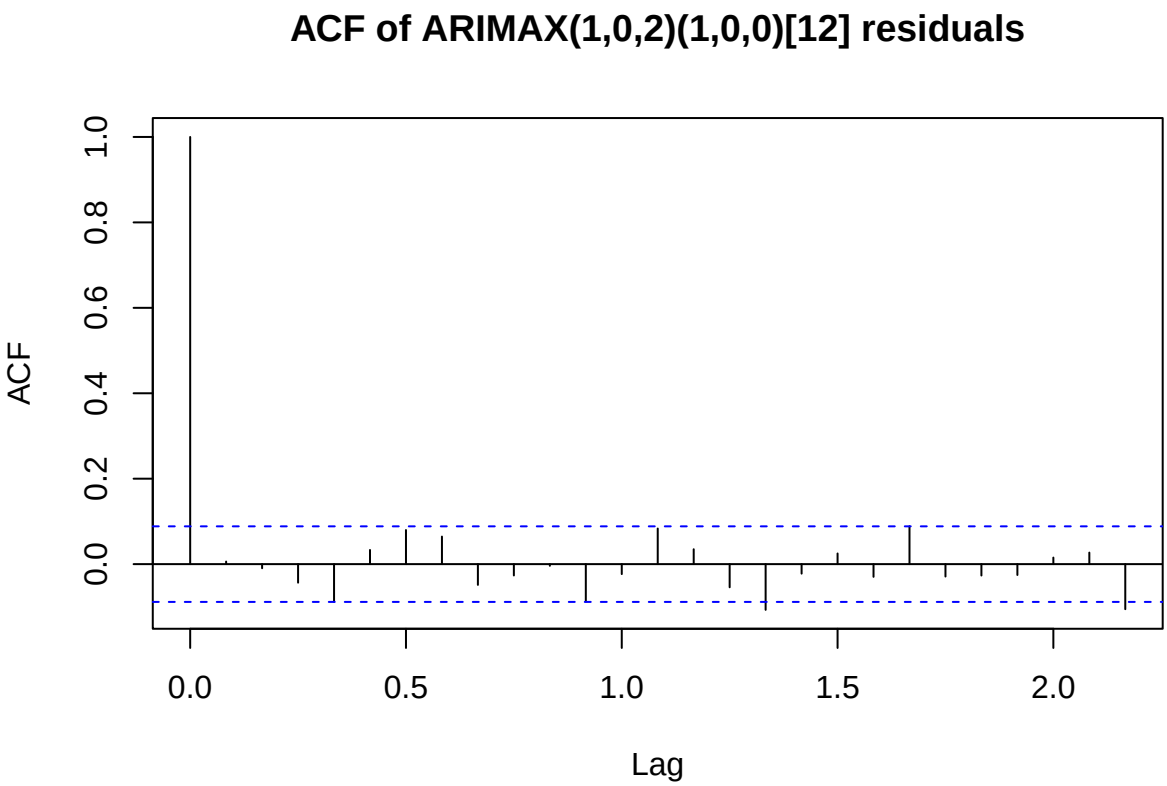

```r
pacf(toronto_train_ts, main = "PACF Plot: of Log-differenced NHPI: Toronto Training Set")
```




```r
# ACF from best ARIMA model
resid_fit <- residuals(toronto_transformed_nphi_auto)
acf(resid_fit, main = "ACF of ARIMA(1,0,2)(1,0,0)[12] residuals")
```



```r
# ACF from best ARIMAX model
resid_fit <- residuals(toronto_fit_arimax)
acf(resid_fit, main = "ACF of ARIMAX(1,0,2)(1,0,0)[12] residuals")
```




```r
# 3. SUMMARY STATISTICS
summary(toronto_train_ts)
```

```
##      Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
## -0.050552  0.000000  0.001820  0.002861  0.004292  0.061174
```

```r
summary(toronto_holdout_ts)
```

```
##       Min.    1st Qu.     Median       Mean    3rd Qu.       Max.
## -0.0034662 -0.0002171  0.0000000  0.0001822  0.0000000  0.0069445
```

```r
# summary of best ARIMA model
summary(toronto_transformed_nphi_auto)
```

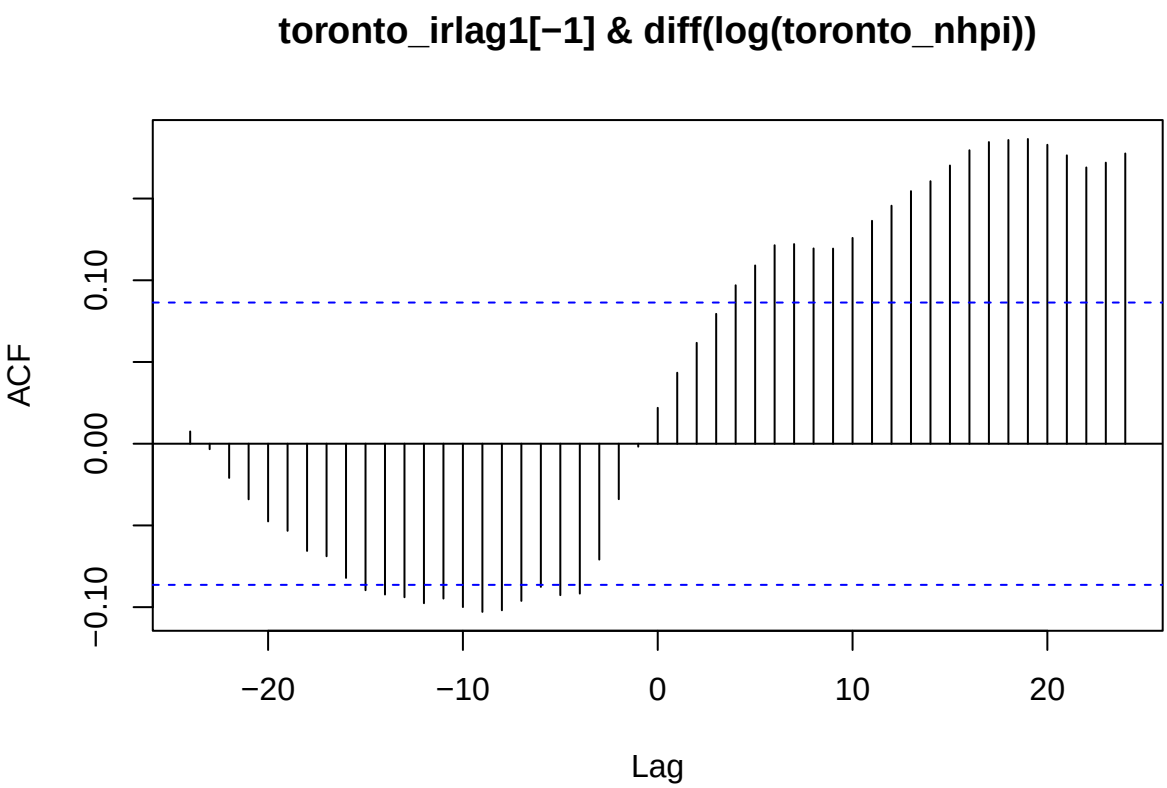
```
## Series: toronto_train_ts
## ARIMA(1,0,2)(1,0,0)[12] with non-zero mean
##
## Coefficients:
##          ar1      ma1      ma2    sar1    mean
##       0.9161  -0.4317  -0.1386  0.1316  0.0034
## s.e.  0.0302   0.0551   0.0490  0.0487  0.0017
##
## sigma^2 = 4.146e-05:  log likelihood = 1782.61
## AIC=-3553.23   AICc=-3553.05   BIC=-3528.05
##
## Training set error measures:
##                        ME        RMSE         MAE MPE MAPE      MASE
```

```
## Training set -0.0001299236 0.006405744 0.003868432 NaN  Inf 0.6292045
##                     ACF1
## Training set 0.006342322
```

```r
# summary of best ARIMAX model
summary(toronto_fit_arimax)
```

```
## Series: toronto_train_ts
## Regression with ARIMA(1,0,2)(1,0,0)[12] errors
##
## Coefficients:
##          ar1      ma1      ma2    sar1  intercept    xreg
##       0.9257  -0.4415  -0.1406  0.1330     0.0051  -3e-04
## s.e.  0.0329   0.0563   0.0488  0.0489     0.0028   4e-04
##
## sigma^2 = 4.147e-05:  log likelihood = 1783
## AIC=-3551.99   AICc=-3551.76   BIC=-3522.62
##
## Training set error measures:
##                        ME        RMSE         MAE MPE MAPE      MASE
## Training set -0.0001599343 0.006400238 0.003875959 NaN  Inf 0.6304288
##                     ACF1
## Training set 0.006084236
```

```r
# ccf between log-differenced NHPI and the lagged interest rate
ccf(toronto_irlag1[-1], diff(log(toronto_nhpi)))
```




```r
# 4. RMSE
rmse = cbind(toronto_persist_fc$rmse, toronto_iid_fc$rmse,
             toronto_esm_fc$rmse, arima_results$rmse, arimax_results$rmse)
colnames(rmse) = c("persist", "iid", "esm", "arima", "arimax")
```

```r
print(round(rmse, 3))
```

```
##   persist   iid   esm arima arimax
## a   0.002 0.003 0.002 0.002  0.002
```

Analyzing Vancouver

Setting up

```r
# Reading the data
vancouver_nhpi_data <- read.csv("Full_Cities/vancouver_full.csv")
vancouver_nhpi_ir_data <- read.csv("Extracted_Cities/vancouver.csv")

# Extracting the NHPI
vancouver_nhpi <- vancouver_nhpi_data$VALUE

# Extracting the one-month lag interest rate
vancouver_irlag1 <- vancouver_nhpi_ir_data$IR_lag1

# Setting the time window from 1981-2023
vancouver_ts <- ts(vancouver_nhpi, start = c(1981,1), end = c(2023,12), frequency = 12)
vancouver_ir_ts <- ts(vancouver_irlag1, start = c(1981,1), end = c(2023,12), frequency = 12)

# Applying a transformation for NHPI
vancouver_transformed_nhpi <- diff(log(vancouver_ts))

# Setting a training set from 1981-2021
vancouver_train_ts <- window(vancouver_transformed_nhpi, end = c(2021,12))
vancouver_train_ir_xreg <- window(vancouver_ir_ts, start = c(1981,2), end = c(2021,12))

# Setting a holdout set from 2022-2023
vancouver_holdout_ts <- window(vancouver_transformed_nhpi, start = c(2022,1))
vancouver_holdout_ir_xreg <- window(vancouver_ir_ts, start = c(2022,1))
```

Doing the baseline forecasts

```r
# Baseline 1: Naive/Persistence
vancouver_persist_fc <- persist_fc(vancouver_train_ts,vancouver_holdout_ts, iprint = FALSE)

# Baseline 2: Training-Set Average
vancouver_iid_fc <- iid_fc(vancouver_train_ts, vancouver_holdout_ts, iprint = FALSE)
```

Doing the exponential smoothing forecasts

```r
library(forecast)
```

```
## Warning: package 'forecast' was built under R version 4.4.3
```

```r
# Method 1
ntrain <- length(vancouver_train_ts)
ntotal <- length(vancouver_transformed_nhpi)

vancouver_transformed_nhpi_train <- vancouver_transformed_nhpi[1:ntrain]
vancouver_transformed_nhpi_holdout <- vancouver_transformed_nhpi[(ntrain+1):ntotal]
```

```r
# Simple Exponential Smoothing
fit_expsmo <- HoltWinters(vancouver_train_ts, beta = FALSE, gamma = FALSE)
vancouver_esm_fc <- esm_fc(vancouver_transformed_nhpi_train,
                           vancouver_transformed_nhpi_holdout,
                           alpha = fit_expsmo$alpha,
                           level = fit_expsmo$coefficients["a"],
                           iprint = FALSE)

# Linear Exponential Smoothing
fit_hw <- HoltWinters(vancouver_train_ts, gamma = FALSE)
vancouver_holt_fc <- lholt_fc(vancouver_transformed_nhpi_train,
                              vancouver_transformed_nhpi_holdout,
                              alpha = fit_hw$alpha,
                              beta = fit_hw$beta,
                              level = fit_hw$coefficients["a"],
                              slope = fit_hw$coefficients["b"],
                              iprint = FALSE)

# Additive Seasonal Exponential Smoothing (using ETS for stability)
# We switched to ets() instead of HoltWinters() because it failed to converage for the Vancouver series
vancouver_aseason_fit <- ets(vancouver_train_ts, model = "AAA")
summary(vancouver_aseason_fit)
```

```
## ETS(A,Ad,A)
##
## Call:
## ets(y = vancouver_train_ts, model = "AAA")
##
##   Smoothing parameters:
##     alpha = 0.239
##     beta  = 1e-04
##     gamma = 1e-04
##     phi   = 0.98
##
##   Initial states:
##     l = -0.0072
##     b = 1e-04
##     s = 0.0014 -0.0011 -0.002 -0.0019 0.001 -0.0031
##            0.0026 -8e-04 -7e-04 0.0017 0 0.0031
##
##   sigma:  0.0079
##
##       AIC      AICc       BIC
## -1687.583 -1686.134 -1612.047
##
## Training set error measures:
##                        ME        RMSE         MAE MPE MAPE      MASE      ACF1
## Training set 9.313466e-05 0.007801818 0.005032244 NaN  Inf 0.6916746 0.1151713
```

```r
vancouver_aseason_forecast <- forecast(vancouver_aseason_fit,
                                       h = length(vancouver_holdout_ts))

vancouver_aseason_rmse <- sqrt(mean((vancouver_holdout_ts -
                                     vancouver_aseason_forecast$mean)^2))
```

```r
# Comparing RMSE
rmse_esm_vancouver <- cbind(vancouver_esm_fc$rmse,
                            vancouver_holt_fc$rmse,
                            vancouver_aseason_rmse)
colnames(rmse_esm_vancouver) <- c("esm", "lholt", "aseason")
rmse_esm_vancouver
```

```
##           esm     lholt     aseason
## a 0.004590305 0.0049477 0.007257523
```

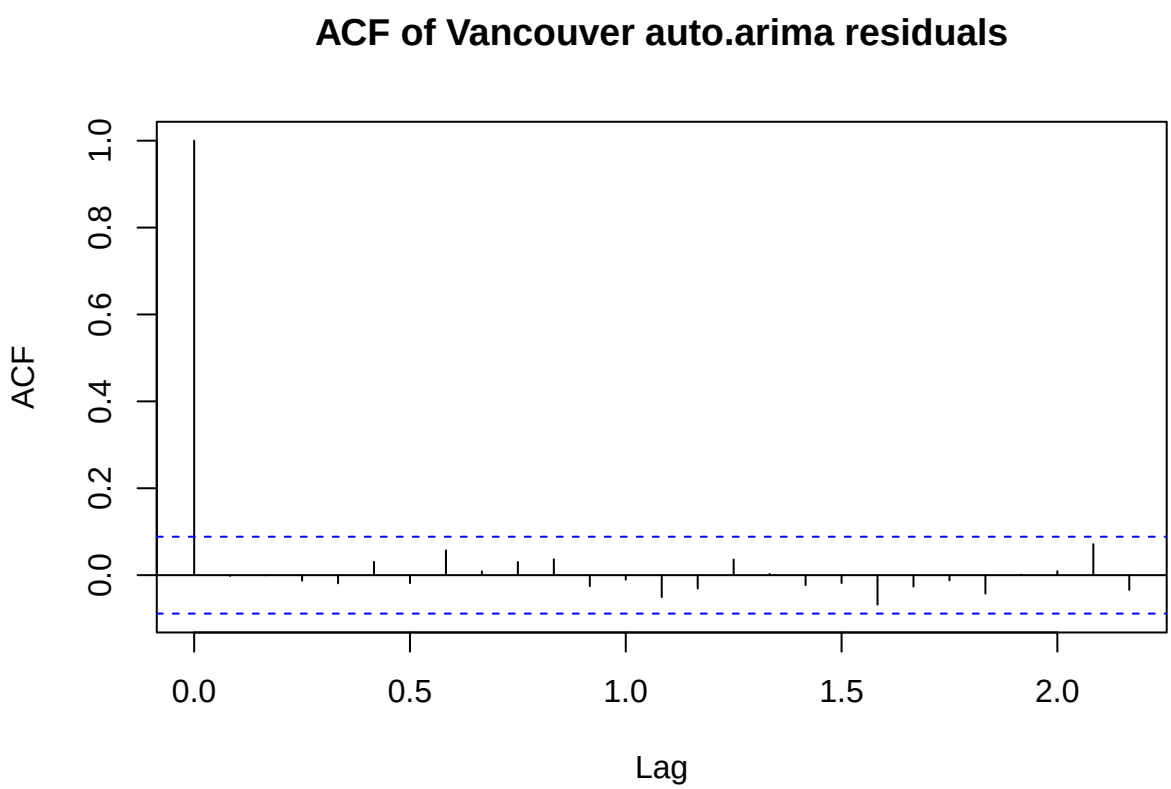
```r
# Best exponential smoothing method: Simple Exponential Smoothing
```

Doing the ARIMA forecasts

```r
# Method 2
vancouver_auto_arima <- forecast::auto.arima(vancouver_train_ts)
summary(vancouver_auto_arima)
```

```
## Series: vancouver_train_ts
## ARIMA(2,0,1)(2,0,0)[12] with zero mean
##
## Coefficients:
##          ar1     ar2      ma1    sar1     sar2
##       0.5860  0.1859  -0.2732  0.0847  -0.0397
## s.e.  0.1429  0.0876   0.1451  0.0517   0.0535
##
## sigma^2 = 5.782e-05:  log likelihood = 1701.14
## AIC=-3390.28   AICc=-3390.1   BIC=-3365.1
##
## Training set error measures:
##                       ME        RMSE         MAE MPE MAPE      MASE        ACF1
## Training set 0.00019155 0.007565047 0.004533597 NaN  Inf 0.6231362 -0.00185311
```

```r
# Check residual ACF
acf(residuals(vancouver_auto_arima),
    main = "ACF of Vancouver auto.arima residuals")
```

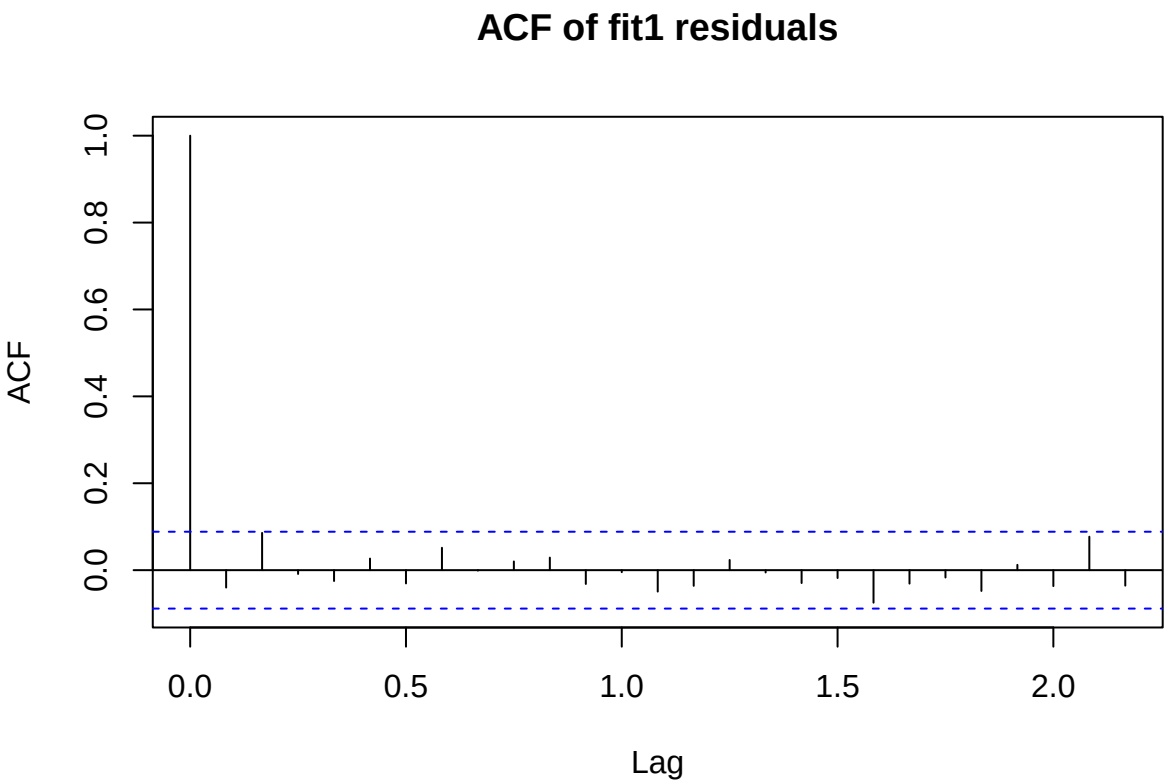



```
# Fit several nearby candidate ARIMA models manually

fit1 <- arima(vancouver_train_ts,
              order = c(1,0,1),
              seasonal = list(order = c(1,0,0), period = 12),
              method = "ML")

fit2 <- arima(vancouver_train_ts,
              order = c(1,0,2),
              seasonal = list(order = c(1,0,0), period = 12),
              method = "ML")

fit3 <- arima(vancouver_train_ts,
              order = c(2,0,1),
              seasonal = list(order = c(1,0,0), period = 12),
              method = "ML")

fit4 <- arima(vancouver_train_ts,
              order = c(2,0,2),
              seasonal = list(order = c(1,0,0), period = 12),
              method = "ML")

fit5 <- arima(vancouver_train_ts,
              order = c(1,0,2),
              seasonal = list(order = c(1,0,1), period = 12),
              method = "ML")

# Compare AIC values
aic_vancouver <- cbind(fit1$aic, fit2$aic, fit3$aic, fit4$aic, fit5$aic)
```

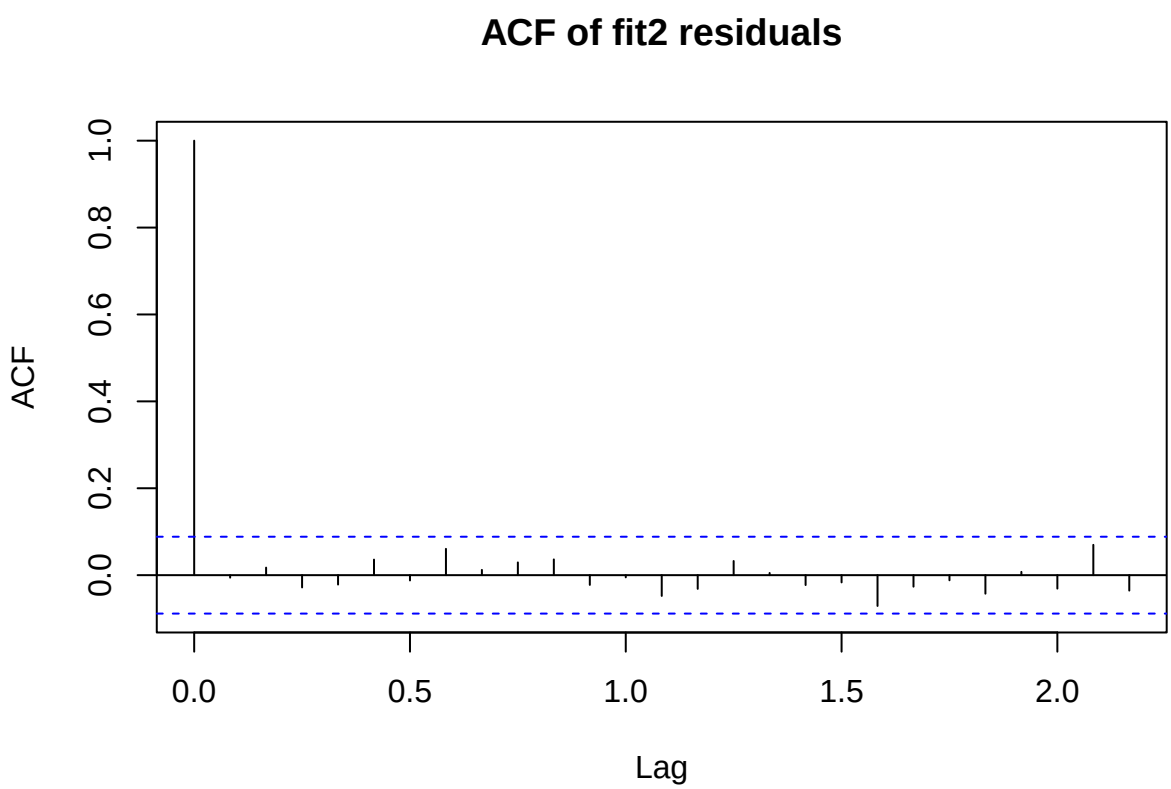
```r
colnames(aic_vancouver) <- c("fit1", "fit2", "fit3", "fit4", "fit5")
aic_vancouver
```

```
##           fit1     fit2      fit3      fit4      fit5
## [1,] -3387.783 -3389.46 -3390.148 -3388.239 -3387.991
```

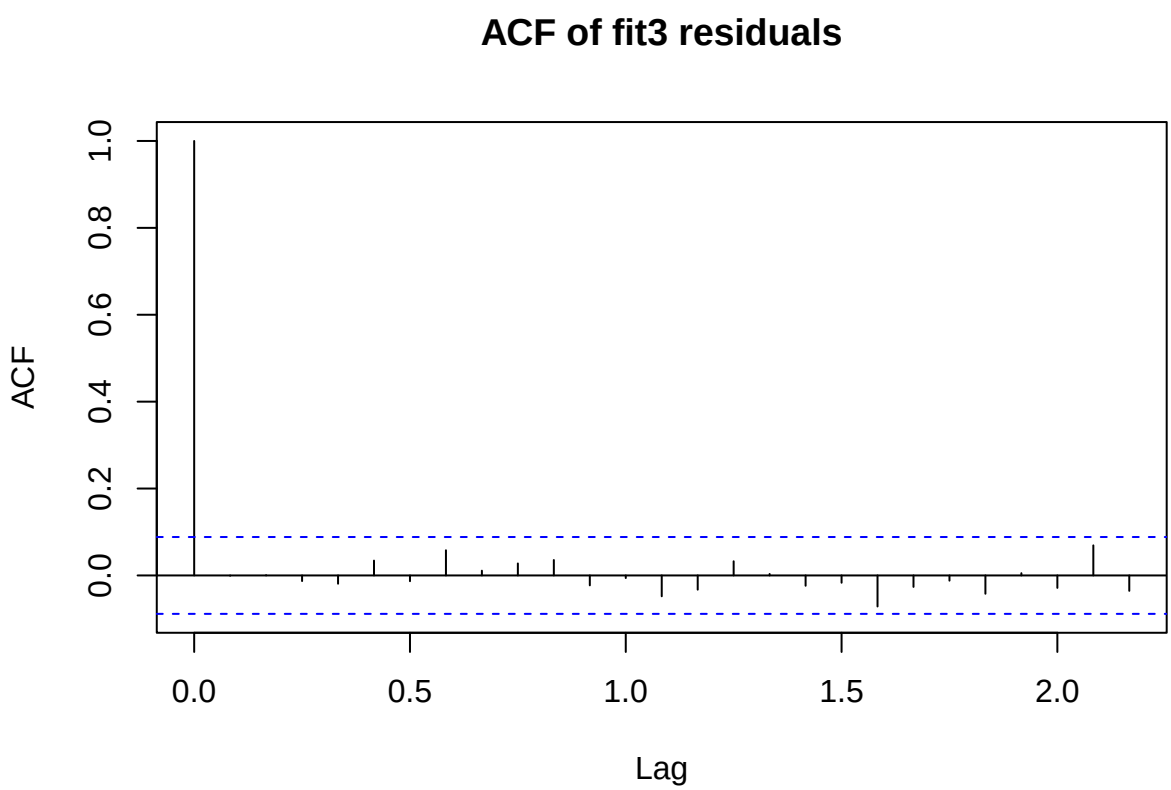
```r
# Check residual ACFs
acf(residuals(fit1), main = "ACF of fit1 residuals")
```



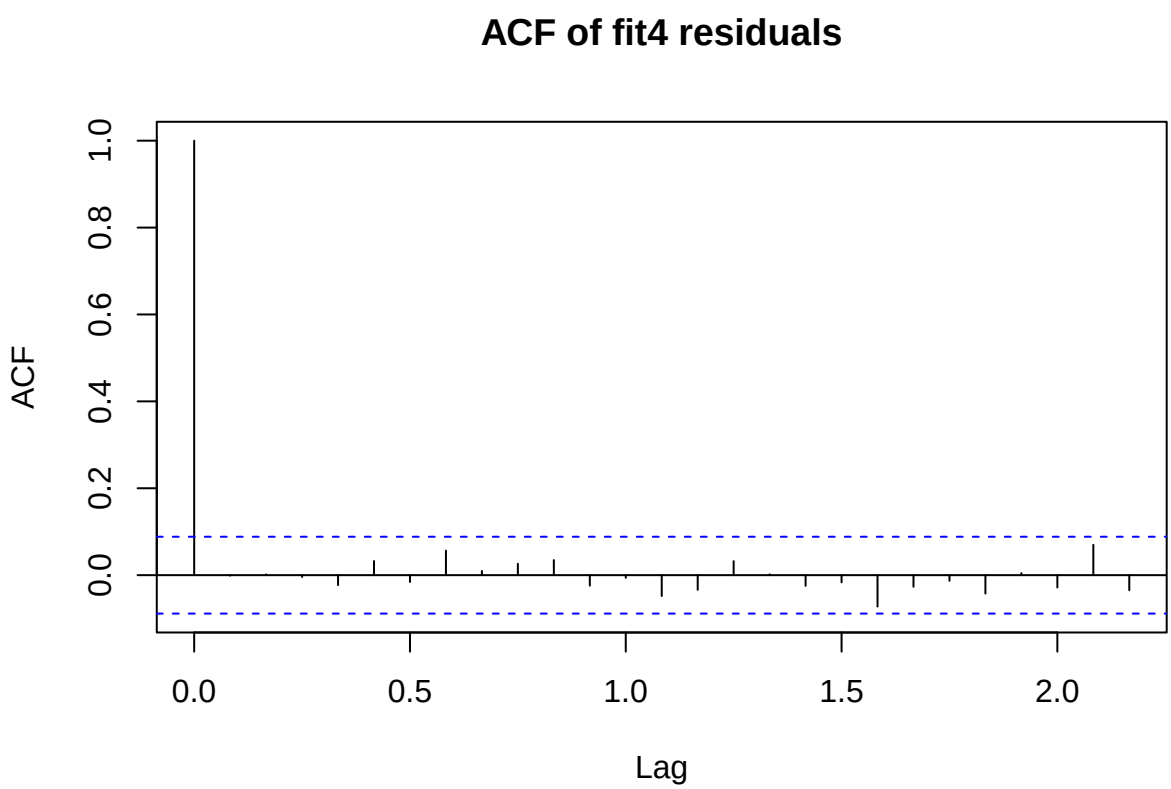

```r
acf(residuals(fit2), main = "ACF of fit2 residuals")
```



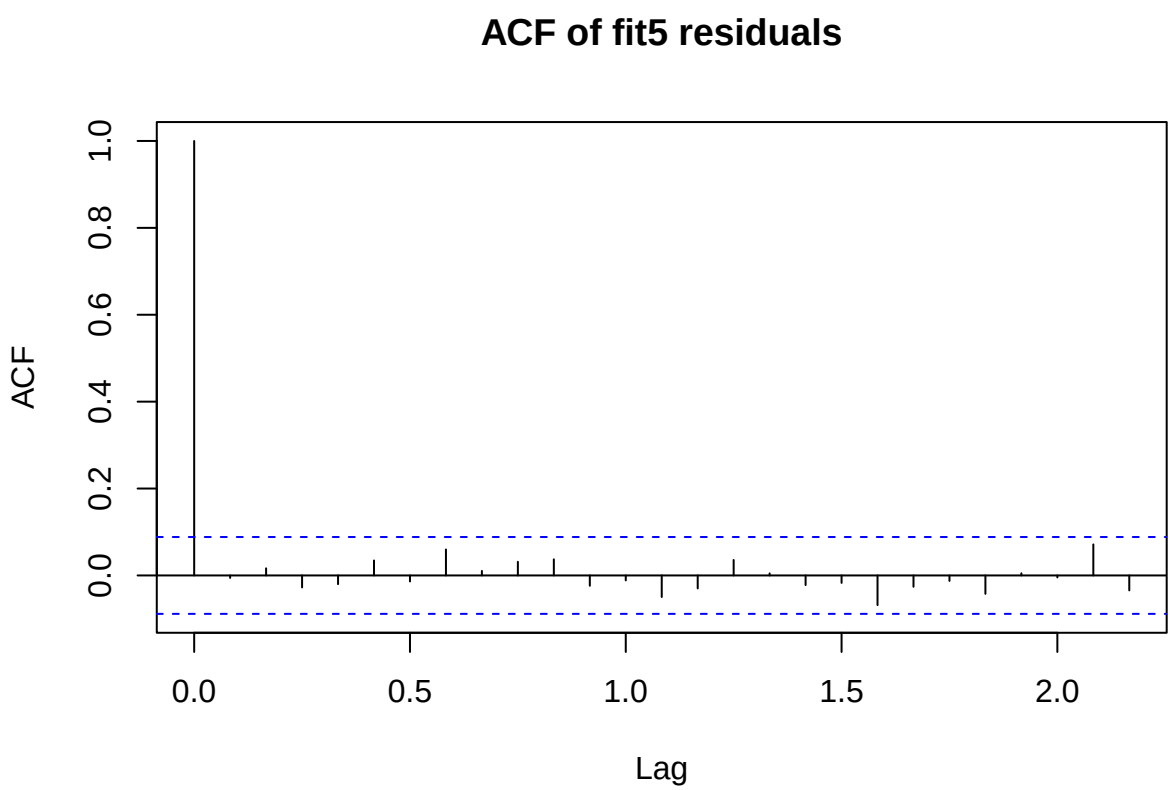

```
acf(residuals(fit3), main = "ACF of fit3 residuals")
```




```
acf(residuals(fit4), main = "ACF of fit4 residuals")
```




```r
acf(residuals(fit5), main = "ACF of fit5 residuals")
```




```r
# Choose the best candidate based on lowest AIC and ACF plot
# fit3 has smallest aic, so we choose fit3
order <- c(2,0,1)
```

```r
seasonal <- list(order = c(1,0,0), period = 12)

# Fit the selected model on the training set
train_arima_model <- arima(vancouver_train_ts,
                           order = order,
                           seasonal = seasonal,
                           method = "ML",
                           include.mean = TRUE)

# Moving 1-step ahead forecasts for the holdout set
vancouver_arima_results <- arma_fc(tsdata = vancouver_transformed_nhpi,
                                   ntrain = length(vancouver_train_ts),
                                   order = order,
                                   seasonal = seasonal,
                                   method = "ML",
                                   traincoef = coef(train_arima_model),
                                   include.mean = TRUE,
                                   iprint = FALSE)

vancouver_arima_results$rmse
```

```
## [1] 0.004518064
```

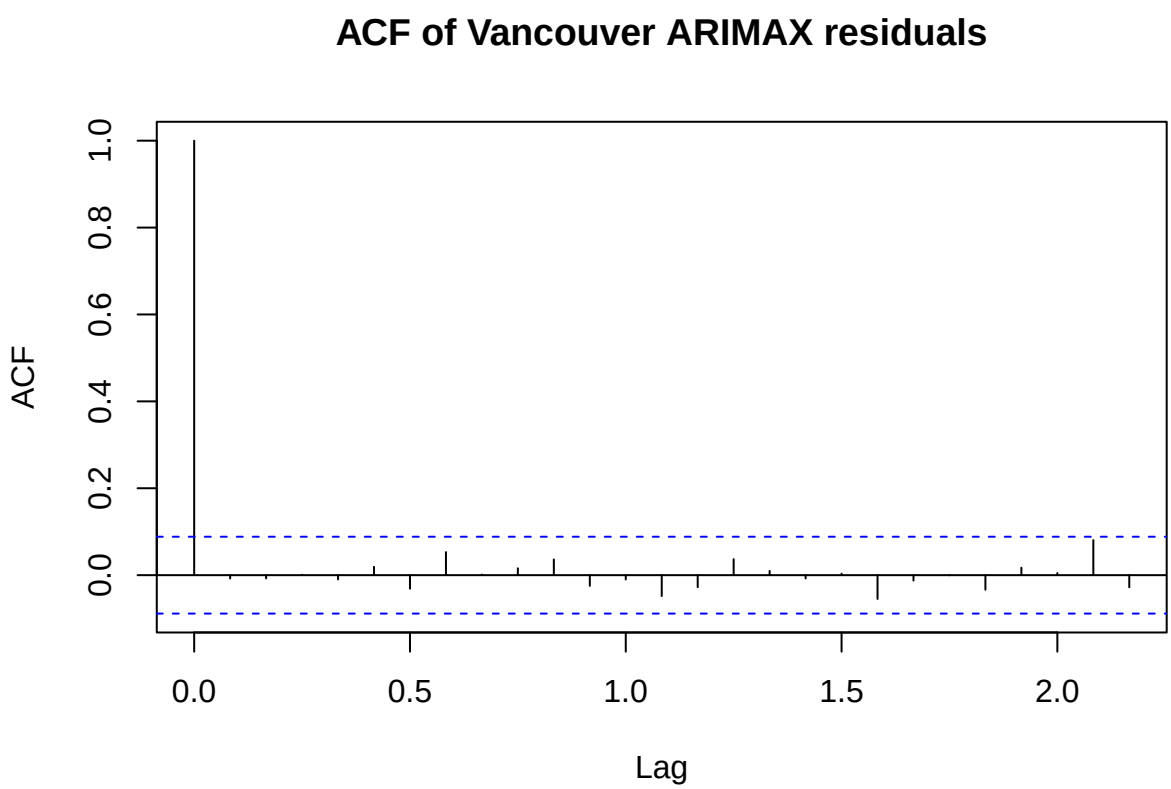
```r
#0.004518064
```

Doing the ARIMAX forecasts with lagged interest

```r
# Method 3
vancouver_fit_arimax <- forecast::auto.arima(vancouver_train_ts,
                                             xreg = vancouver_train_ir_xreg)
summary(vancouver_fit_arimax)
```

```
## Series: vancouver_train_ts
## Regression with ARIMA(1,0,3)(2,0,0)[12] errors
##
## Coefficients:
##          ar1      ma1     ma2      ma3    sar1     sar2  intercept    xreg
##       0.8510  -0.5583  0.1035  -0.0536  0.0849  -0.0216     0.0049  -8e-04
## s.e.  0.0591   0.0764  0.0541   0.0536  0.0522   0.0540     0.0019   3e-04
##
## sigma^2 = 5.695e-05:  log likelihood = 1706.4
## AIC=-3394.81   AICc=-3394.43   BIC=-3357.04
##
## Training set error measures:
##                        ME        RMSE         MAE MPE MAPE      MASE
## Training set -6.790793e-05 0.007484566 0.004592843 NaN  Inf 0.6312796
##                      ACF1
## Training set -0.007682121
```

```r
acf(residuals(vancouver_fit_arimax),
    main = "ACF of Vancouver ARIMAX residuals")
```



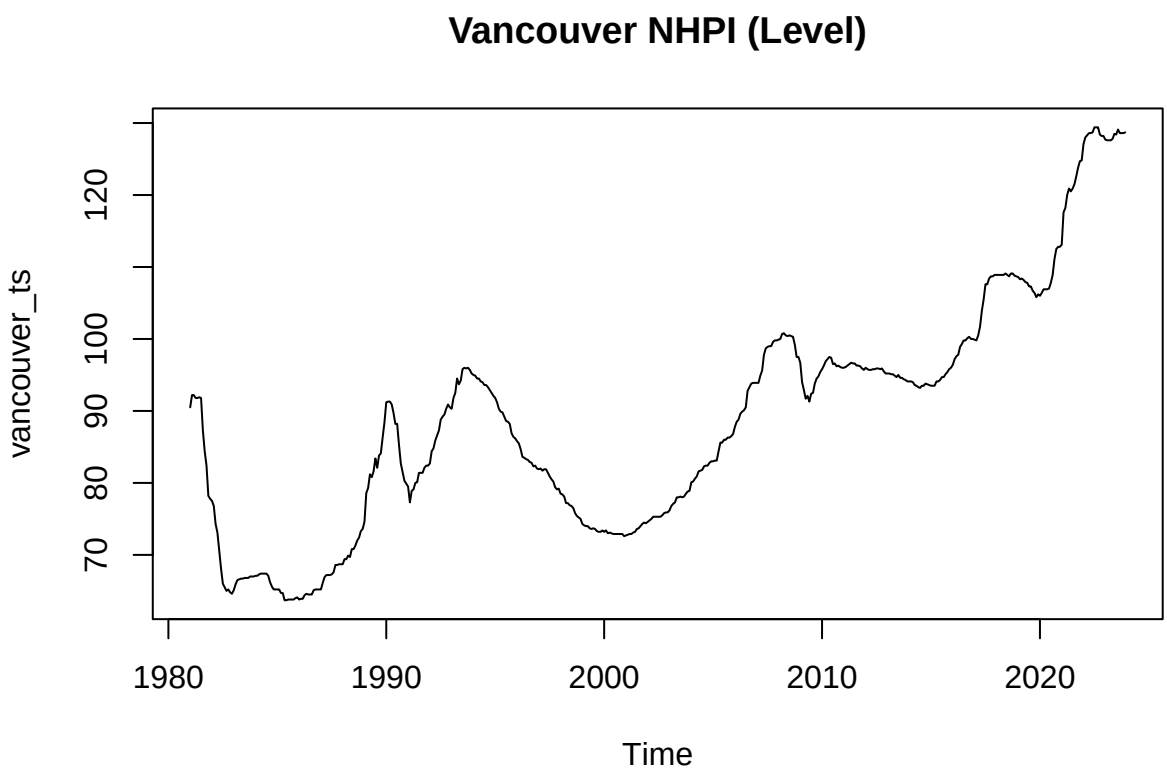

```
order <- c(2,0,1)
seasonal <- list(order = c(1,0,0), period = 12)

# Use the same ARIMA order as chosen above as a starting point
train_arimax_model_vancouver <- arima(vancouver_train_ts,
                                       order = order,
                                       seasonal = seasonal,
                                       xreg = vancouver_train_ir_xreg,
                                       method = "ML",
                                       include.mean = TRUE)

# Getting the result of the moving 1-step ahead forecast for holdout set
vancouver_arimax_results <- armax_fc(tsdata = vancouver_transformed_nhpi, ntrain = length(vancouver_trai
                                      order = order,
                                      seasonal = seasonal,
                                      xreg = vancouver_irlag1[-1],
                                      method = "ML",
                                      traincoef = coef(train_arimax_model_vancouver), include.mean = TRU

vancouver_arimax_results$rmse
```

```
## [1] 0.004444058
```

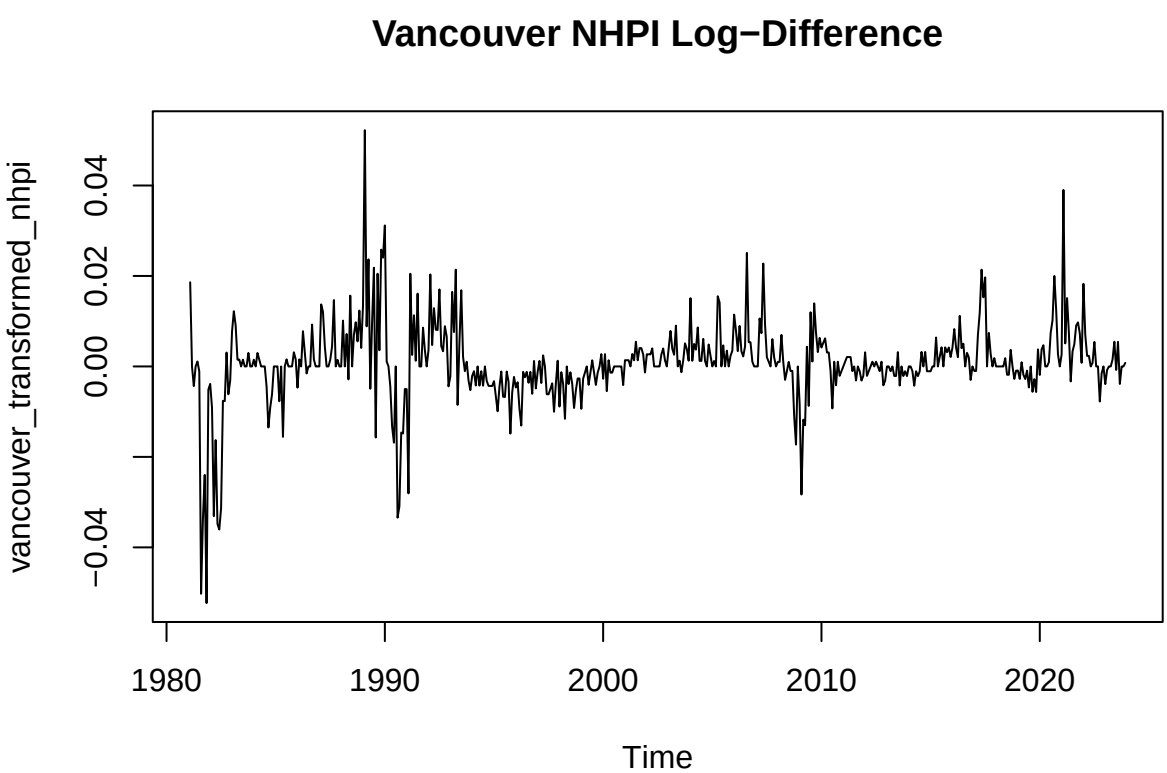
```
#0.004444058
```

Viewing the results of Vancouver's analyses

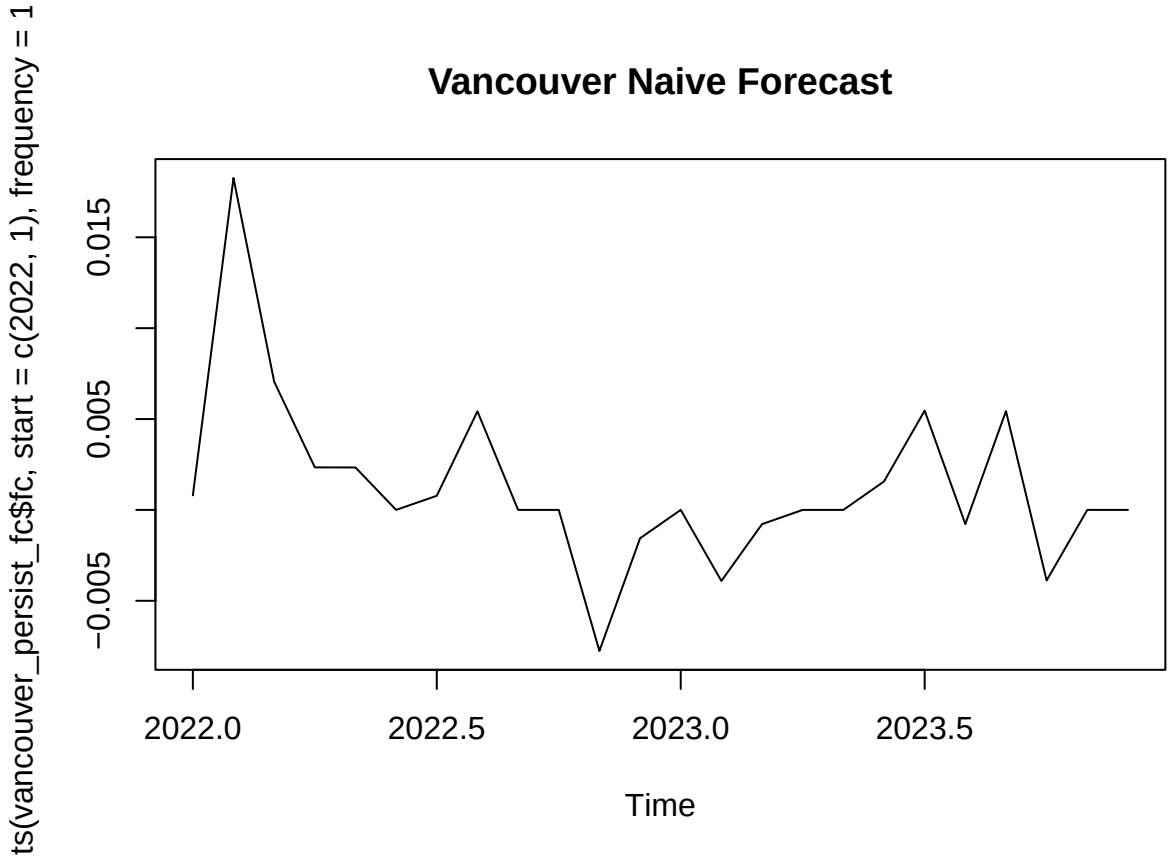
```
# 1. TIME SERIES PLOT
plot(vancouver_ts, main = "Vancouver NHPI (Level)")
```



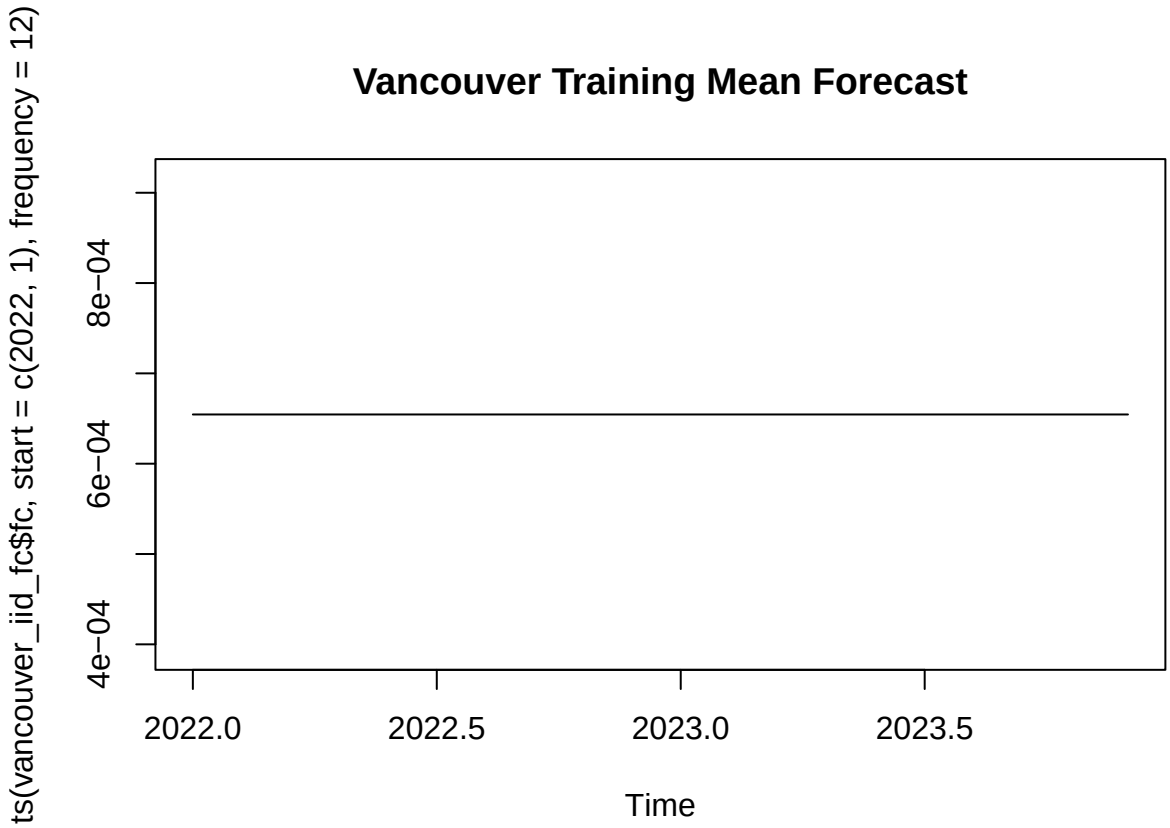

```r
plot(vancouver_transformed_nhpi, main = "Vancouver NHPI Log-Difference")
```



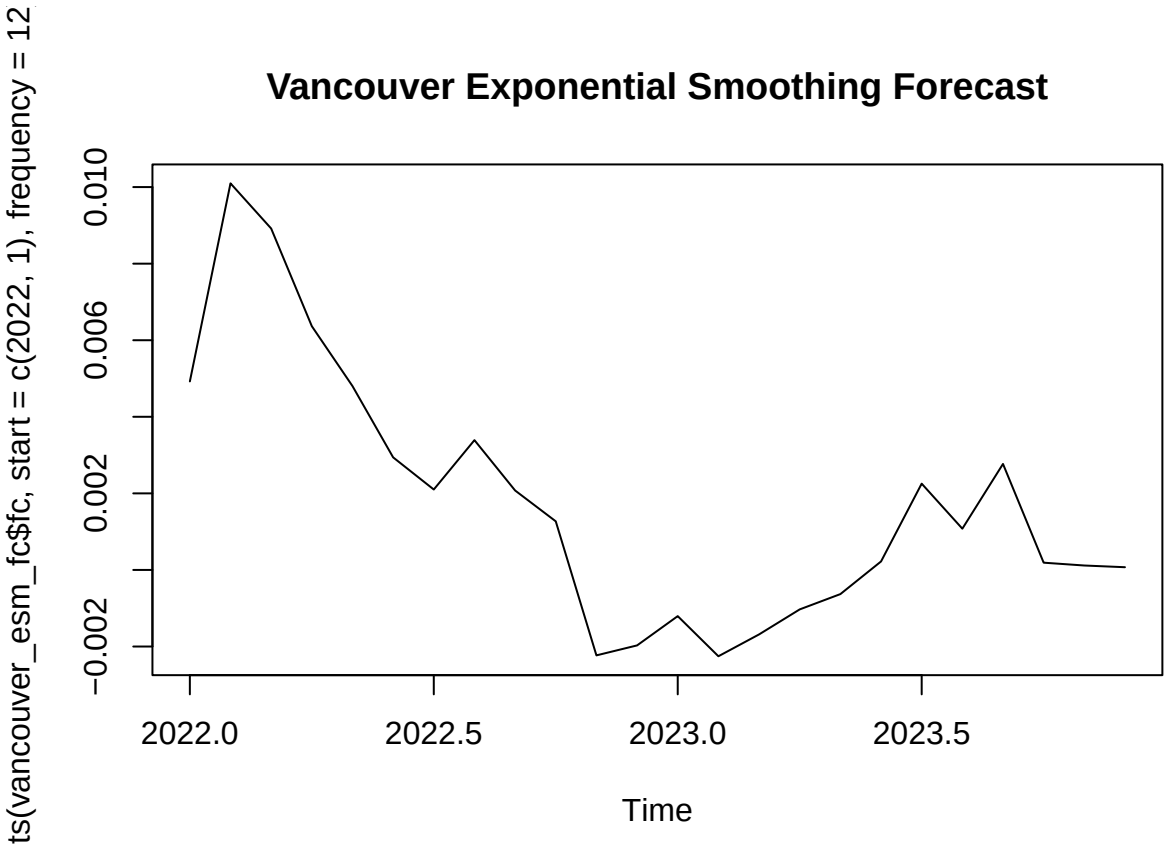

```r
# Forecast plots
plot(ts(vancouver_persist_fc$fc, start = c(2022,1), frequency = 12),
     main = "Vancouver Naive Forecast")
```



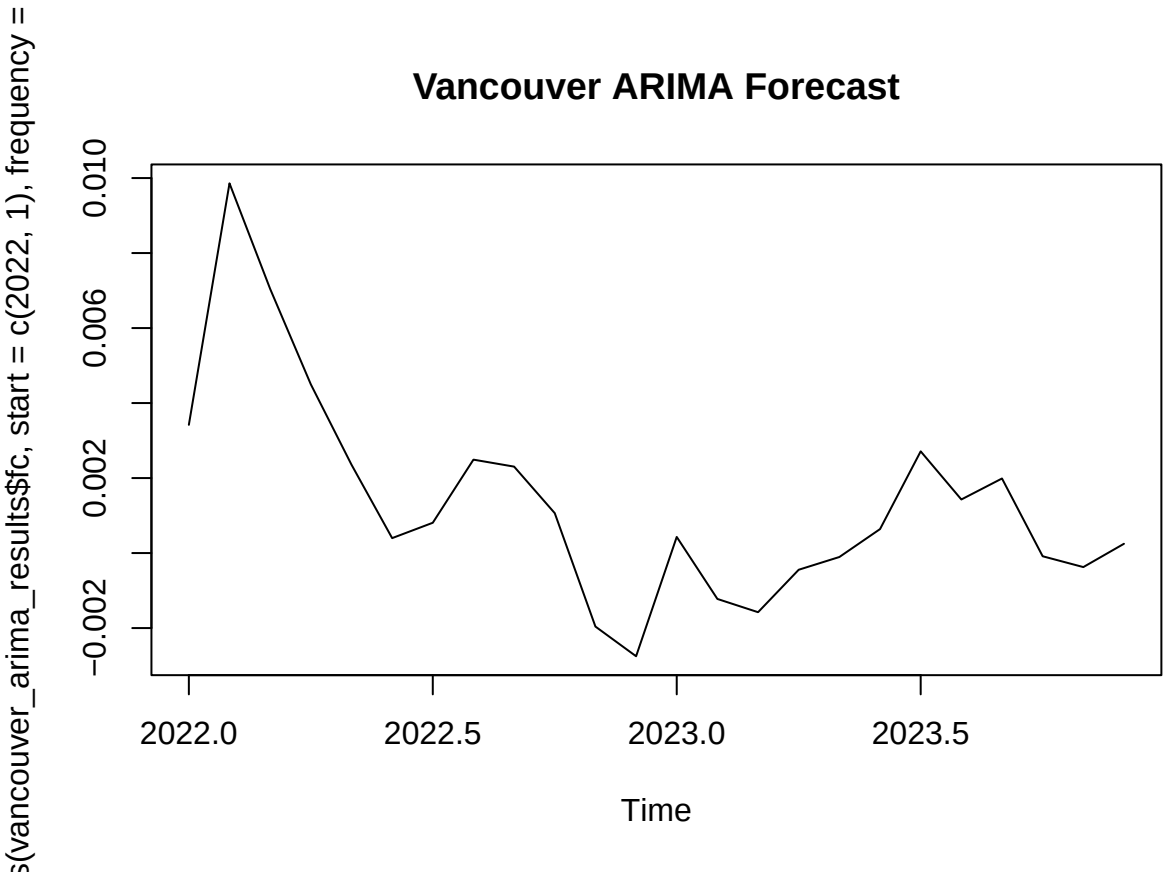

```r
plot(ts(vancouver_iid_fc$fc, start = c(2022,1), frequency = 12),
     main = "Vancouver Training Mean Forecast")
```



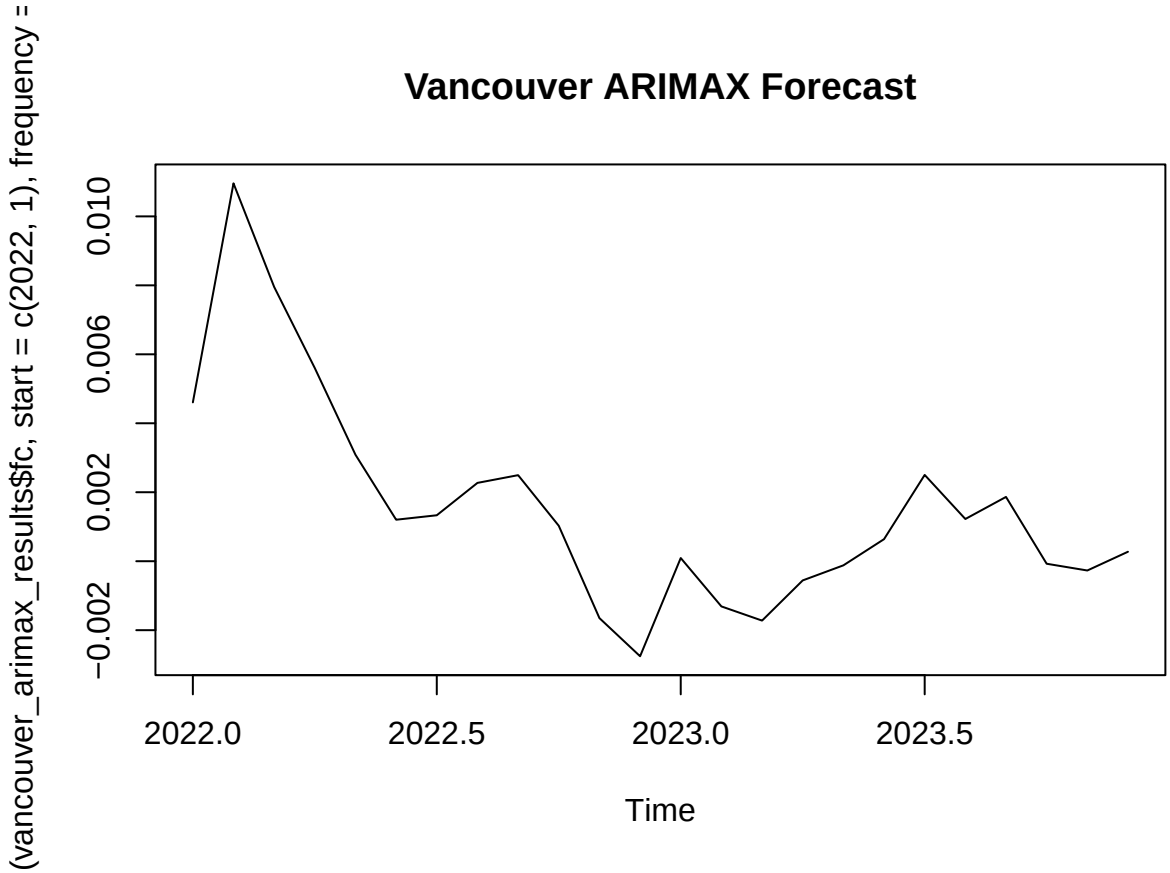

```r
plot(ts(vancouver_esm_fc$fc, start = c(2022,1), frequency = 12),
     main = "Vancouver Exponential Smoothing Forecast")
```



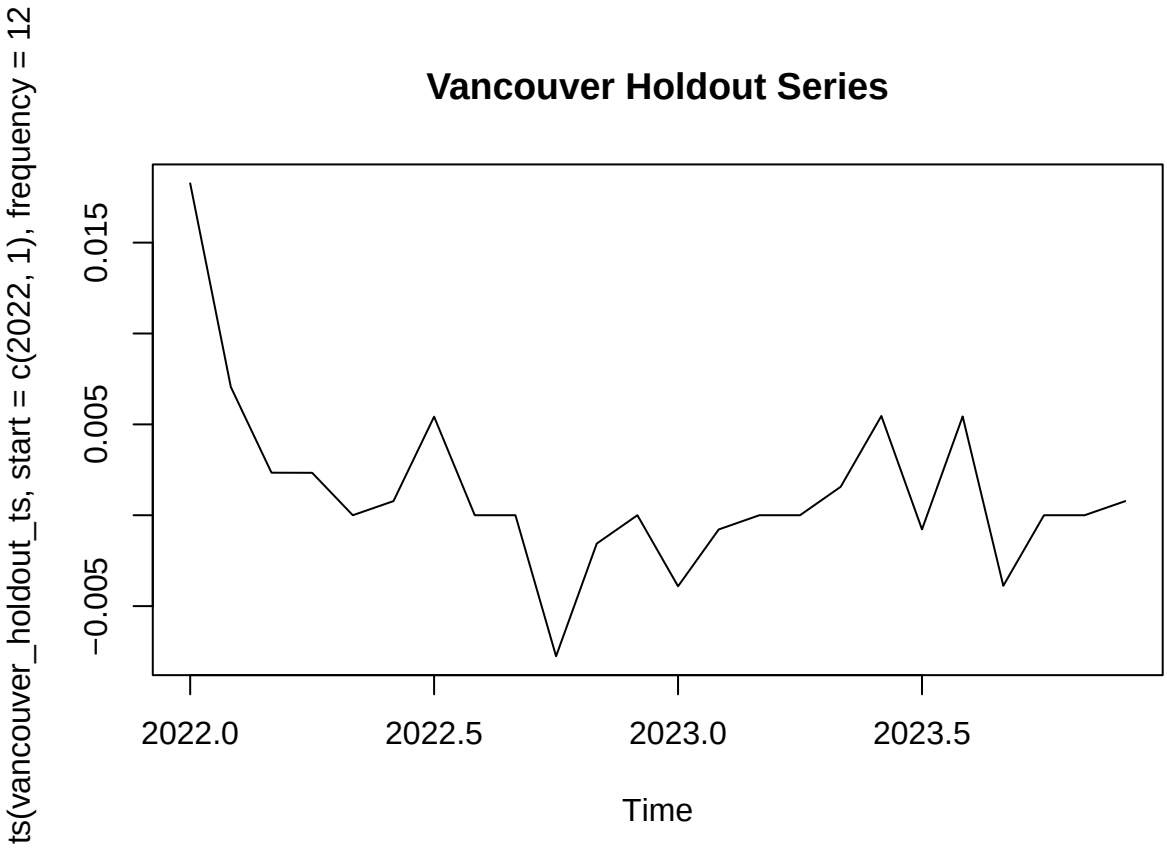

```r
plot(ts(vancouver_arima_results$fc, start = c(2022,1), frequency = 12),
     main = "Vancouver ARIMA Forecast")
```



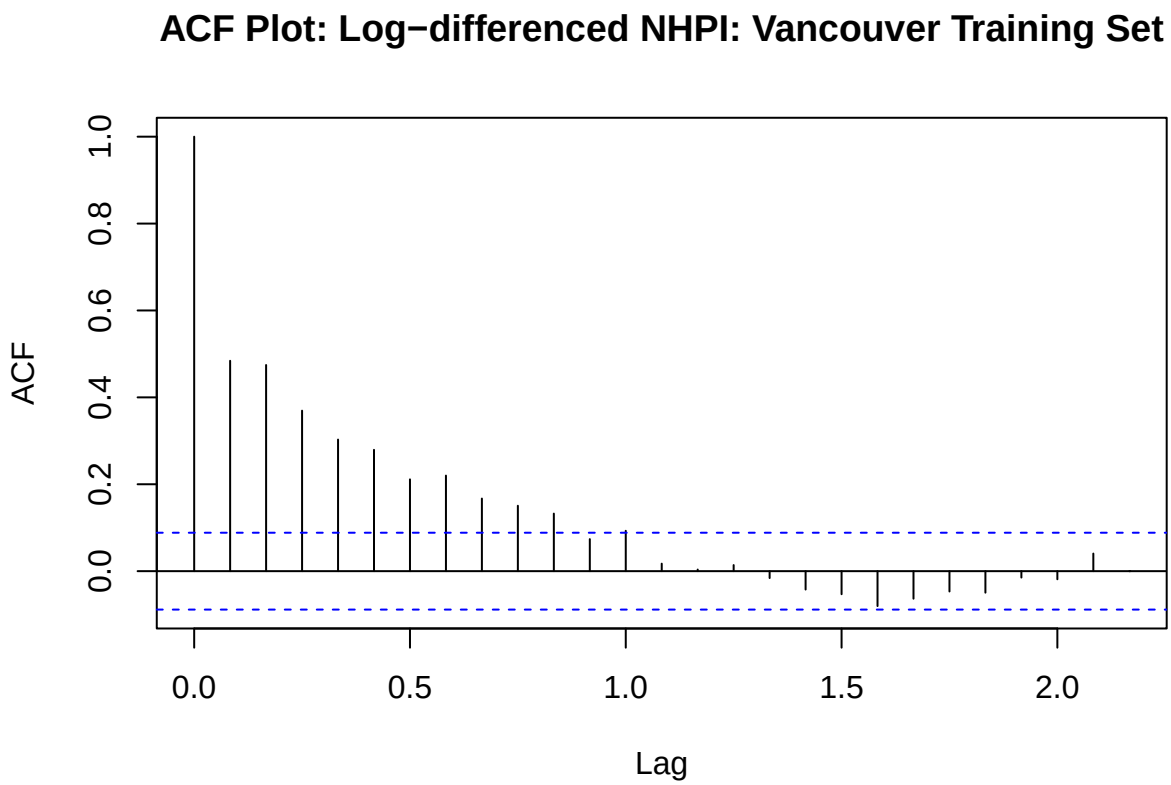

```r
plot(ts(vancouver_arimax_results$fc, start = c(2022,1), frequency = 12),
     main = "Vancouver ARIMAX Forecast")
```



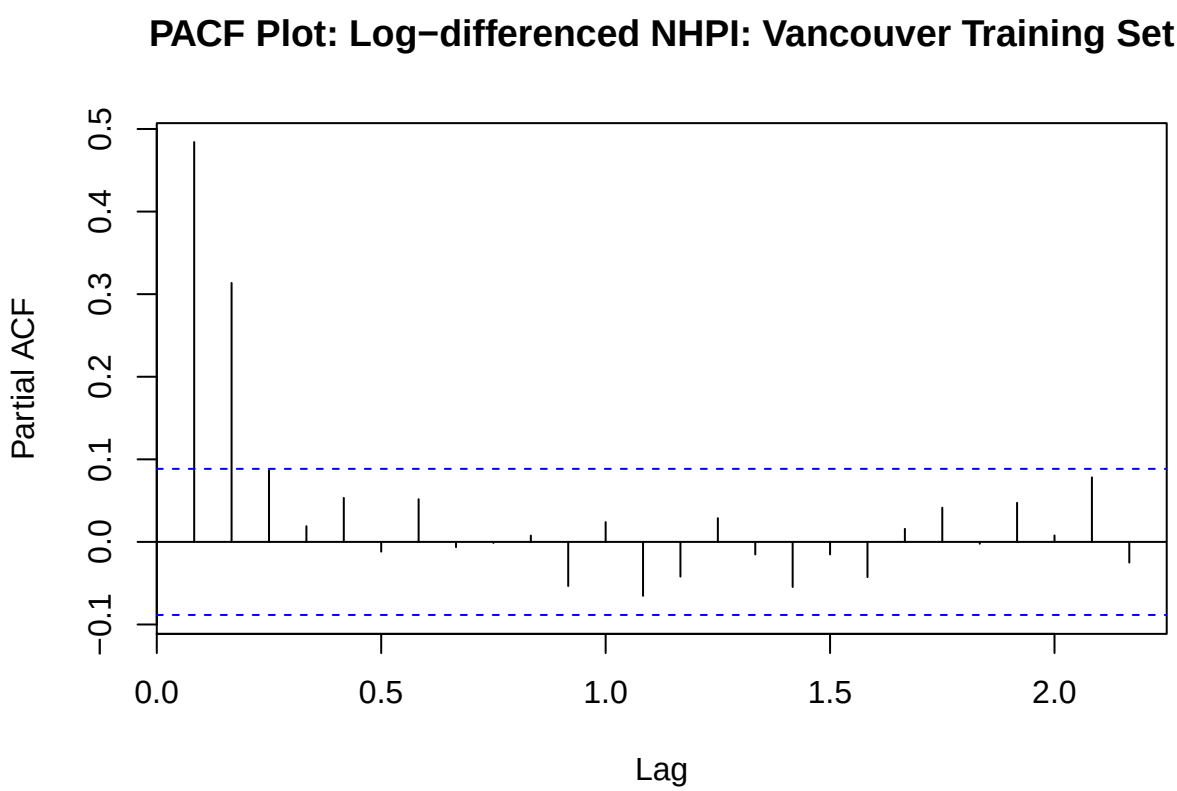

```r
plot(ts(vancouver_holdout_ts, start = c(2022,1), frequency = 12),
     main = "Vancouver Holdout Series")
```




```r
# 2. ACF PLOT
acf(vancouver_train_ts,
    main = "ACF Plot: Log-differenced NHPI: Vancouver Training Set")
```

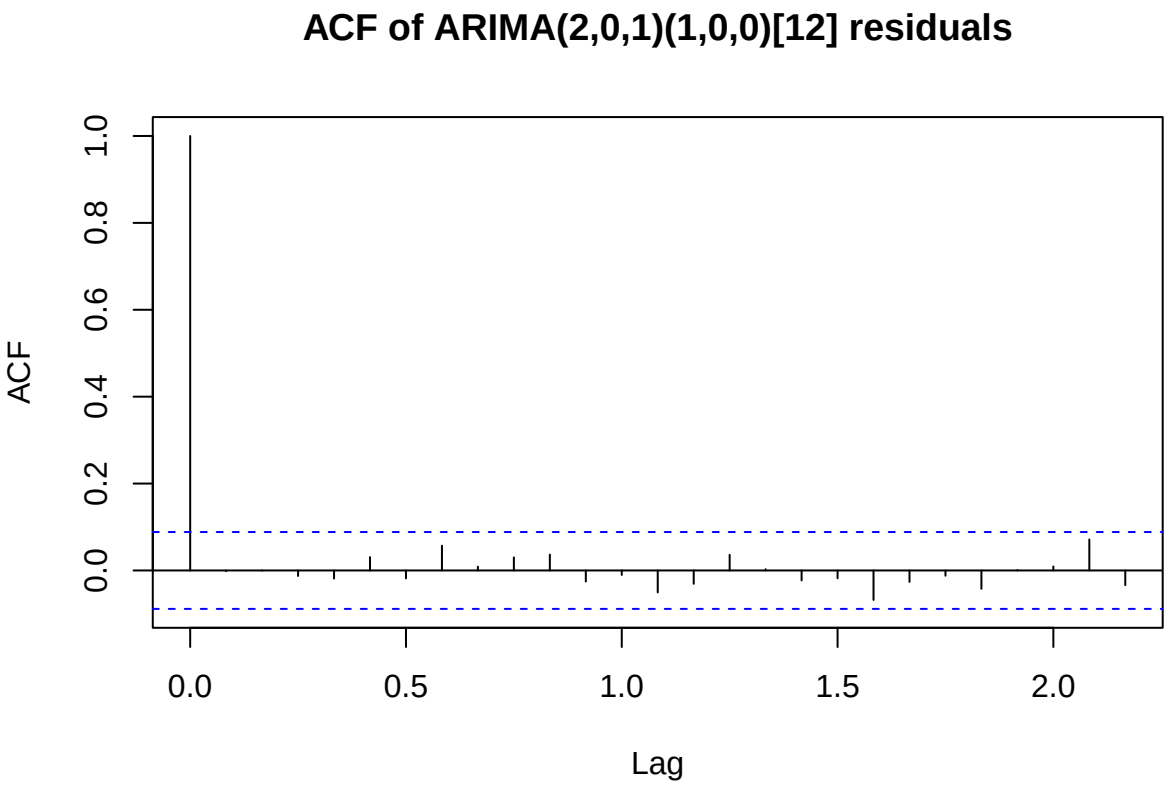



```
pacf(vancouver_train_ts,
     main = "PACF Plot: Log-differenced NHPI: Vancouver Training Set")
```

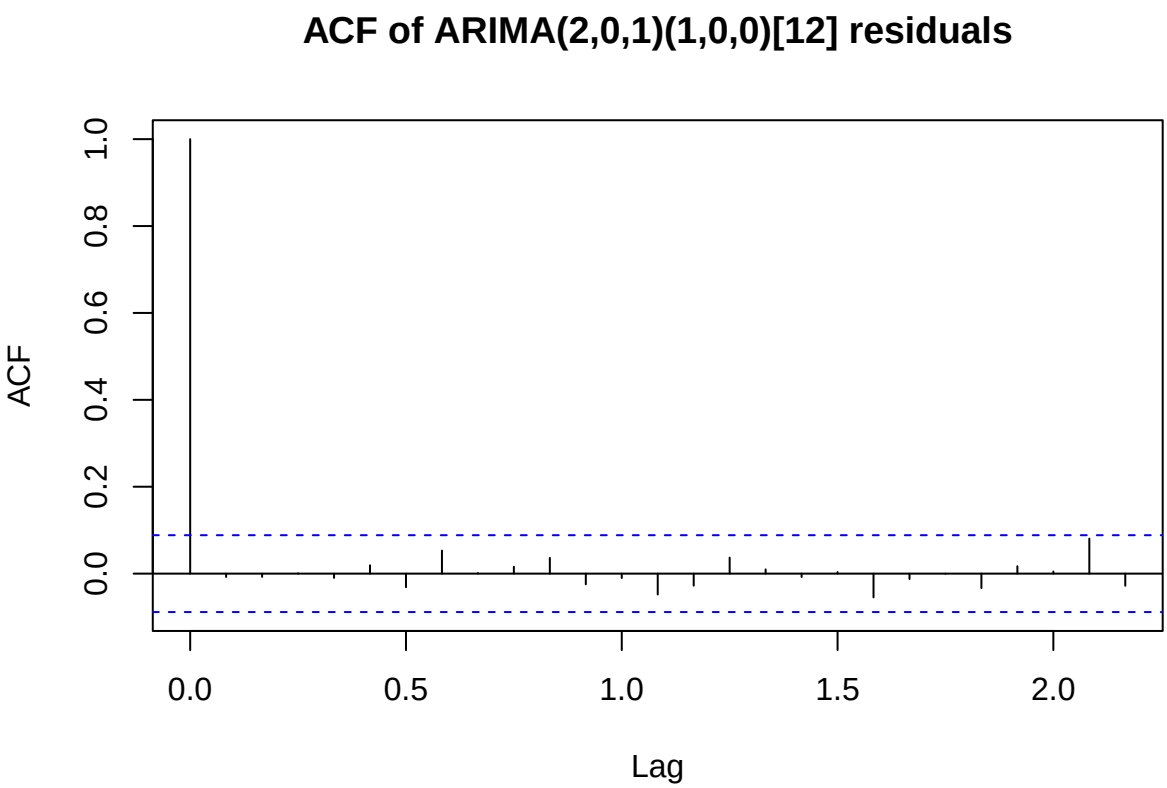



```
acf(residuals(vancouver_auto_arima),
```

```
    main = "ACF of ARIMA(2,0,1)(1,0,0)[12] residuals")
```




```
acf(residuals(vancouver_fit_arimax),
    main = "ACF of ARIMA(2,0,1)(1,0,0)[12] residuals")
```



```r
# 3. SUMMARY STATISTICS
summary(vancouver_train_ts)
```

```
##       Min.    1st Qu.     Median       Mean    3rd Qu.       Max.
## -0.0523158 -0.0020822  0.0000000  0.0006545  0.0035279  0.0522312
```

```r
summary(vancouver_holdout_ts)
```

```
##       Min.    1st Qu.     Median       Mean    3rd Qu.       Max.
## -0.0077580 -0.0001946  0.0000000  0.0012822  0.0023369  0.0182617
```

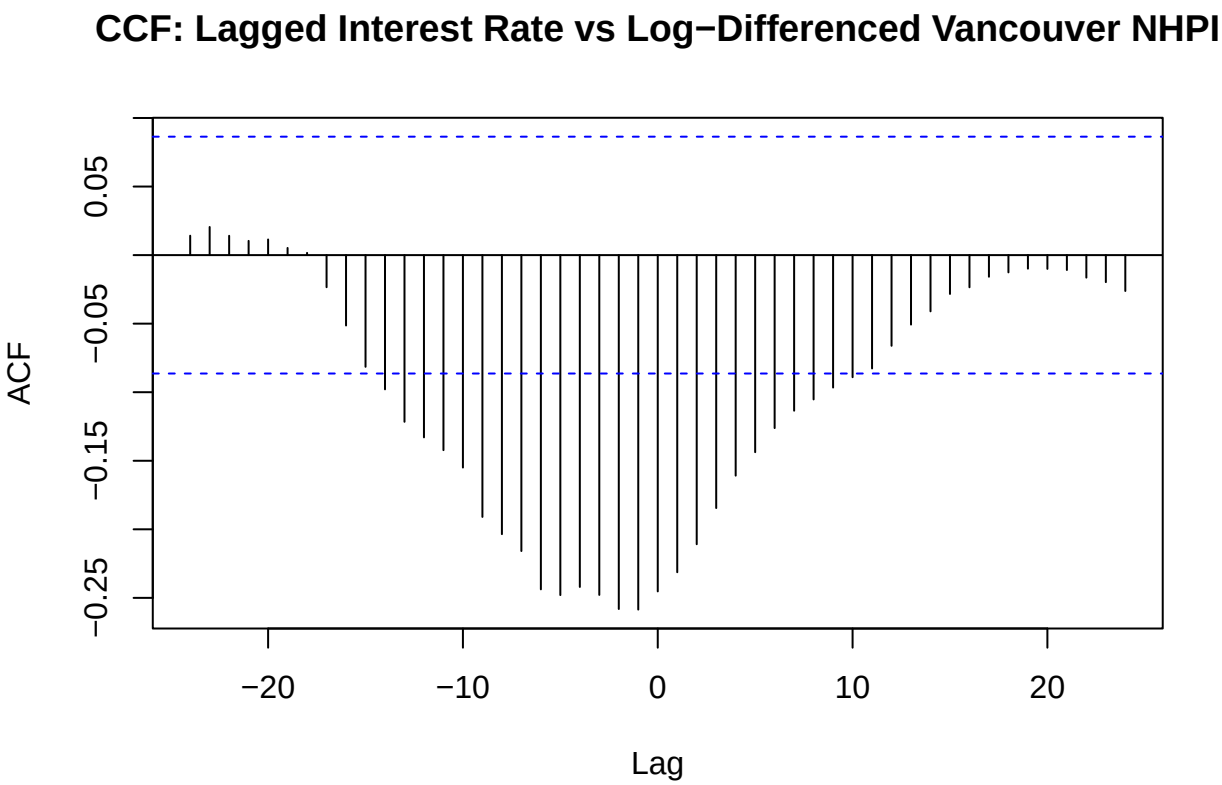
```r
summary(vancouver_auto_arima)
```

```
## Series: vancouver_train_ts
## ARIMA(2,0,1)(2,0,0)[12] with zero mean
##
## Coefficients:
##          ar1     ar2      ma1    sar1     sar2
##       0.5860  0.1859  -0.2732  0.0847  -0.0397
## s.e.  0.1429  0.0876   0.1451  0.0517   0.0535
##
## sigma^2 = 5.782e-05:  log likelihood = 1701.14
## AIC=-3390.28   AICc=-3390.1   BIC=-3365.1
##
## Training set error measures:
##                      ME        RMSE         MAE MPE MAPE      MASE        ACF1
## Training set 0.00019155 0.007565047 0.004533597 NaN  Inf 0.6231362 -0.00185311
```

```r
summary(vancouver_fit_arimax)
```

```
## Series: vancouver_train_ts
## Regression with ARIMA(1,0,3)(2,0,0)[12] errors
##
## Coefficients:
##          ar1      ma1     ma2      ma3    sar1     sar2  intercept    xreg
##       0.8510  -0.5583  0.1035  -0.0536  0.0849  -0.0216     0.0049  -8e-04
## s.e.  0.0591   0.0764  0.0541   0.0536  0.0522   0.0540     0.0019   3e-04
##
## sigma^2 = 5.695e-05:  log likelihood = 1706.4
## AIC=-3394.81   AICc=-3394.43   BIC=-3357.04
##
## Training set error measures:
##                         ME        RMSE         MAE MPE MAPE      MASE
## Training set -6.790793e-05 0.007484566 0.004592843 NaN  Inf 0.6312796
##                      ACF1
## Training set -0.007682121
```

```r
# CCF between log-differenced NHPI and lagged interest rate
ccf(vancouver_irlag1[-1], diff(log(vancouver_nhpi)),
    main = "CCF: Lagged Interest Rate vs Log-Differenced Vancouver NHPI")
```




```r
# 4. RMSE
vancouver_rmse <- cbind(vancouver_persist_fc$rmse,
                        vancouver_iid_fc$rmse,
                        vancouver_esm_fc$rmse,
                        vancouver_arima_results$rmse,
                        vancouver_arimax_results$rmse)
colnames(vancouver_rmse) <- c("persist", "iid", "esm", "arima", "arimax")
print(round(vancouver_rmse, 3))
```

```
##   persist   iid   esm arima arimax
## a   0.006 0.005 0.005 0.005  0.004
```

Final Comparison Table

```r
library(knitr)

results_table <- data.frame(
  City    = c("Montreal", "Toronto", "Vancouver"),
  Persist = c(montreal_persist_fc$rmse,
              toronto_persist_fc$rmse,
              vancouver_persist_fc$rmse),
  IID     = c(montreal_iid_fc$rmse,
             toronto_iid_fc$rmse,
             vancouver_iid_fc$rmse),
  ESM     = c(montreal_holt_fc$rmse,     # Montreal best: Linear Holt
             toronto_esm_fc$rmse,        # Toronto best: Simple ESM
             vancouver_esm_fc$rmse),     # Vancouver best: Simple ESM
  ARIMA   = c(montreal_arima_results$rmse,
             toronto_arima_results$rmse,
```

```r
              vancouver_arima_results$rmse),
  ARIMAX = c(montreal_arimax_results$rmse,
             toronto_arimax_results$rmse,
             vancouver_arimax_results$rmse)
)

# Average row
avg_row <- data.frame(
  City   = "Average",
  Persist = mean(results_table$Persist),
  IID    = mean(results_table$IID),
  ESM    = mean(results_table$ESM),
  ARIMA  = mean(results_table$ARIMA),
  ARIMAX = mean(results_table$ARIMAX)
)

final_table <- rbind(results_table, avg_row)

kable(final_table, digits = 4,
      caption = "One-Step-Ahead Rolling Forecast RMSE by City and Method")
```

Table 1: One-Step-Ahead Rolling Forecast RMSE by City and Method

City	Persist	IID	ESM	ARIMA	ARIMAX
Montreal	0.0101	0.0076	0.0077	0.0083	0.0084
Toronto	0.0022	0.0034	0.0022	0.0020	0.0020
Vancouver	0.0059	0.0048	0.0046	0.0045	0.0044
Average	0.0061	0.0053	0.0048	0.0049	0.0050

```r
# Best overall method
avg_rmse <- colMeans(results_table[, -1])
best_method <- names(which.min(avg_rmse))
cat("Best overall method:", best_method,
    "| Average RMSE =", round(min(avg_rmse), 4), "\n")
```

```
## Best overall method: ESM | Average RMSE = 0.0048
```